

Migração de Arquitetura

Como migramos **arquitetura** em ambientes reais

Quem sou eu?

Luiz Schons

- 23 anos
- PicPay
- Guarapuava PR
- Canionista / Montanhista
- Corrida de Aventura
- Dev Paraná
- UTFPR



Disclaimer!

Migração de arquitetura

O que **não** é arquitetura?

O que **não** é arquitetura?

Domain-Driven Design (DDD)

O que **não** é arquitetura?

Domain-Driven Design (DDD)

Design Patterns

O que **não** é arquitetura?

Domain-Driven Design (DDD)

Object Calisthenics

Design Patterns

O que **não** é arquitetura?

Domain-Driven Design (DDD)

Object Calisthenics

Design Patterns

SOLID

O que **não** é arquitetura?

Domain-Driven Design (DDD) Object Calisthenics

Design Patterns

SOLID

Estrutura de pastas

O que isso é exatamente?

Domain-Driven Design (DDD) Object Calisthenics

Design Patterns SOLID

Estimativa de pasta

O que é arquitetura?



A arquitetura de software é o conjunto de estruturas essenciais para a compreensão e análise do sistema.

A arquitetura de software é o **conjunto de estruturas** essenciais para a compreensão e análise do sistema.

Monolítica

A arquitetura de software é o **conjunto de estruturas** essenciais para a compreensão e análise do sistema.

Monolítica

Microservices

A arquitetura de software é o **conjunto de estruturas** essenciais para a compreensão e análise do sistema.

Monolítica

EDA

Microservices

A arquitetura de software é o **conjunto de estruturas** essenciais para a compreensão e análise do sistema.

Monolítica

EDA

SOA

Microservices

Estruturas arquiteturais

Monolítica

EDA

SOA

Microservices

Estruturas arquiteturais

Padrões de design de alto nível que definem a estrutura fundamental do sistema e impõem regras globais para garantir atributos de qualidade críticos, como escalabilidade e manutenibilidade.

Estruturas arquiteturais

É o que define a estrutura principal do sistema e as regras de montagem para que ele possa crescer muito e não quebre fácil.

Como escolher a melhor arquitetura?

Como escolher a melhor ~~arquitetura~~?
estrutura arquitetural

Para facilitar nossa compreensão durante a palestra:

**Vamos definir que arquitetura é
a escolha ou composição de
estruturas arquiteturais**

Como escolher a melhor arquitetura?

Como escolher a melhor arquitetura?

Time-to-market

Momento atual da codebase

Maturidade do time

Problema que o software resolve

Time-to-market

Tempo de entrega necessário para que um produto ou nova funcionalidade seja lançada no mercado

Precisamos lançar quando?

Precisamos lançar quando?

Antes do concorrentes

Precisamos lançar quando?

Antes do concorrentes

ou

Depois de validar o produto

Time-to-market ✓

Momento atual da codebase

Maturidade do time

Problema que o software resolve

Momento atual da codebase

Complexidade para evoluir o software

Dificuldade em fazer entregas de qualidade

Qualidade da codebase

Qualidade do código

Sem código de qualidade não
existe evolução **arquitetural**

**Sem código de qualidade não
existe evolução do produto**

Sem código de qualidade não
existe evolução **técnica**

- **Testes**
- **Pipelines**
- **DevX**
- **Observabilidade**

Time-to-market ✓

Momento atual da codebase ✓

Maturidade do time

Problema que o software resolve

O time **precisa** estar preparado para
atuar no cenário que o **software vive**

Não adianta o time entender de
microservices se a empresa precisa de
um **MVP**

O time **não deve** impor gostos
pessoais no software

"Criar uma estrutura mais complexa que o necessário, não faz de você um bom desenvolvedor só satisfaz o seu ego"

- Diego Borges

Time-to-market ✓

Momento atual da codebase ✓

Maturidade do time ✓

Problema que o software resolve

Problemas **simples** pedem resoluções **simples**

Problemas complexos pedem resoluções complexas

Problemas **complexos** pedem resoluções ~~**complexas**~~
simples

Problemas **complexos** pedem resoluções ~~**complexas**~~
~~**simples**~~
menos complexas

Simplicidade $\neq$ Mal feito

Quando o **problema** que o software resolve
é claro, fica mais fácil chegar em **soluções**
menos complexas

Time-to-market ✓

Momento atual da codebase ✓

Maturidade do time ✓

Problema que o software resolve ✓

Mas essa escolha é definitiva?

Definitivamente NÃO



Time-to-market

Momento atual da codebase

Maturidade do time

Problema que você resolve

Arquitetura é uma coisa viva e
evolui junto com o sistema

Arquitetura deve servir ao time!

Não o time a **arquitetura**

Eu **quero** ou eu **preciso** migrar?

Eu quero ou eu preciso migrar?

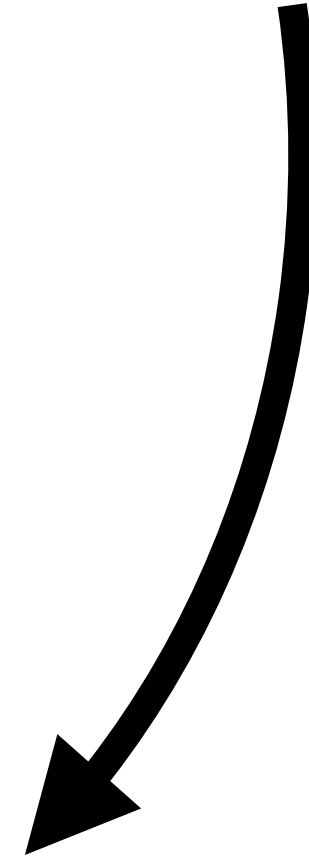
Como identificar isso?

Querer **!=** Precisar

Querer antes de precisar pode
ser uma jogada de mestre

Querer antes de **precisar** pode
ser uma jogada de mestre

Pode, não tem garantia de que será



Querer sem precisar poder ser
um checkmate em si mesmo

Seguir hypes
Ignorar seu problema
Decisões por ego
Vontade de usar uma tecnologia

Seguir hypes

Ignorar seu problema

Decisões por ego

Vontade de usar uma tecnologia

Tudo isso aí é **querer!**

Problemas para escalar
Dificuldades com manutenção
Decisões visando o projeto
Resolver travas técnicas

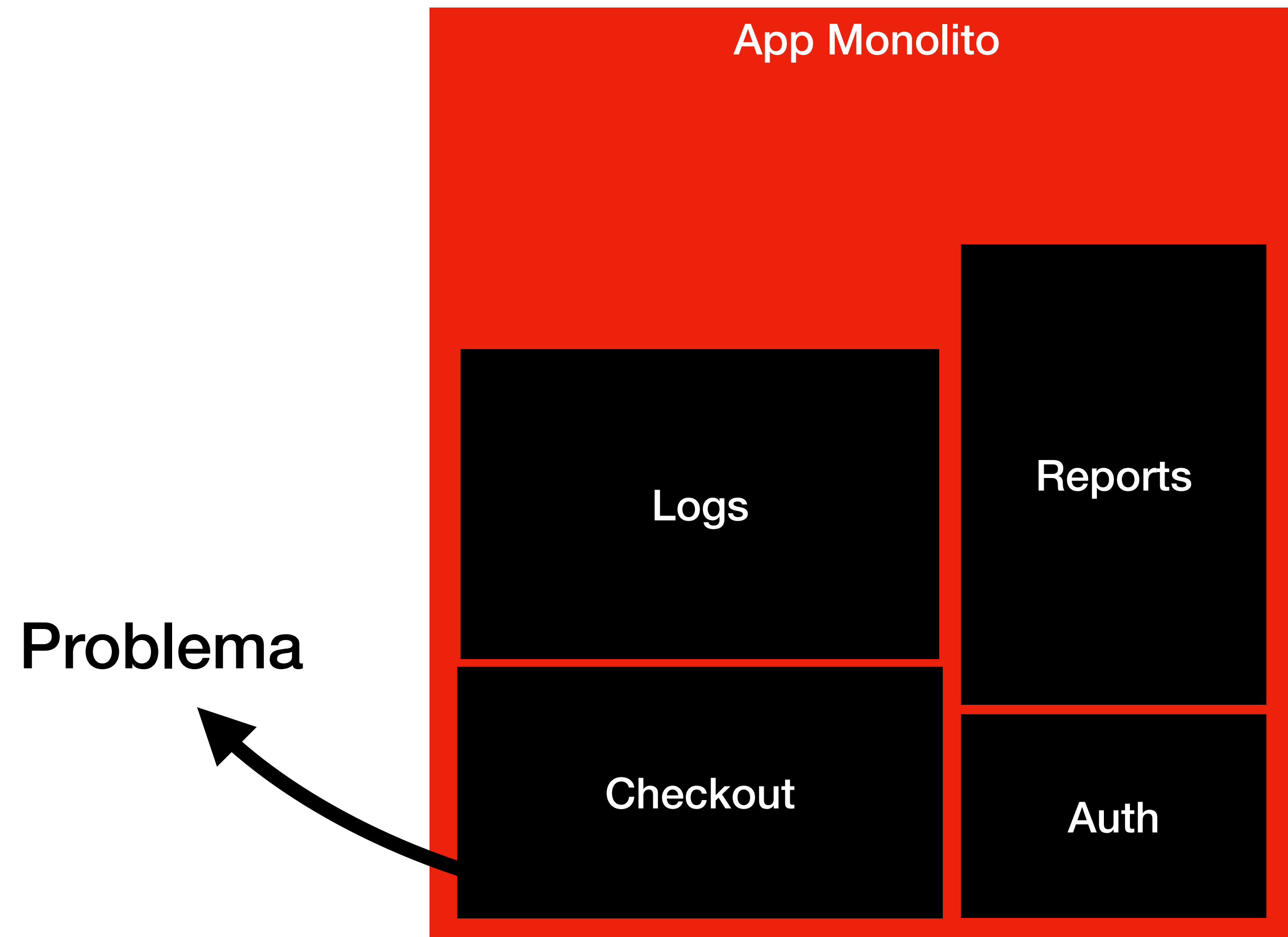
Problemas para escalar
Dificuldades com manutenção
Decisões visando o projeto
Resolver travas técnicas

Tudo isso aí é **precisar!**

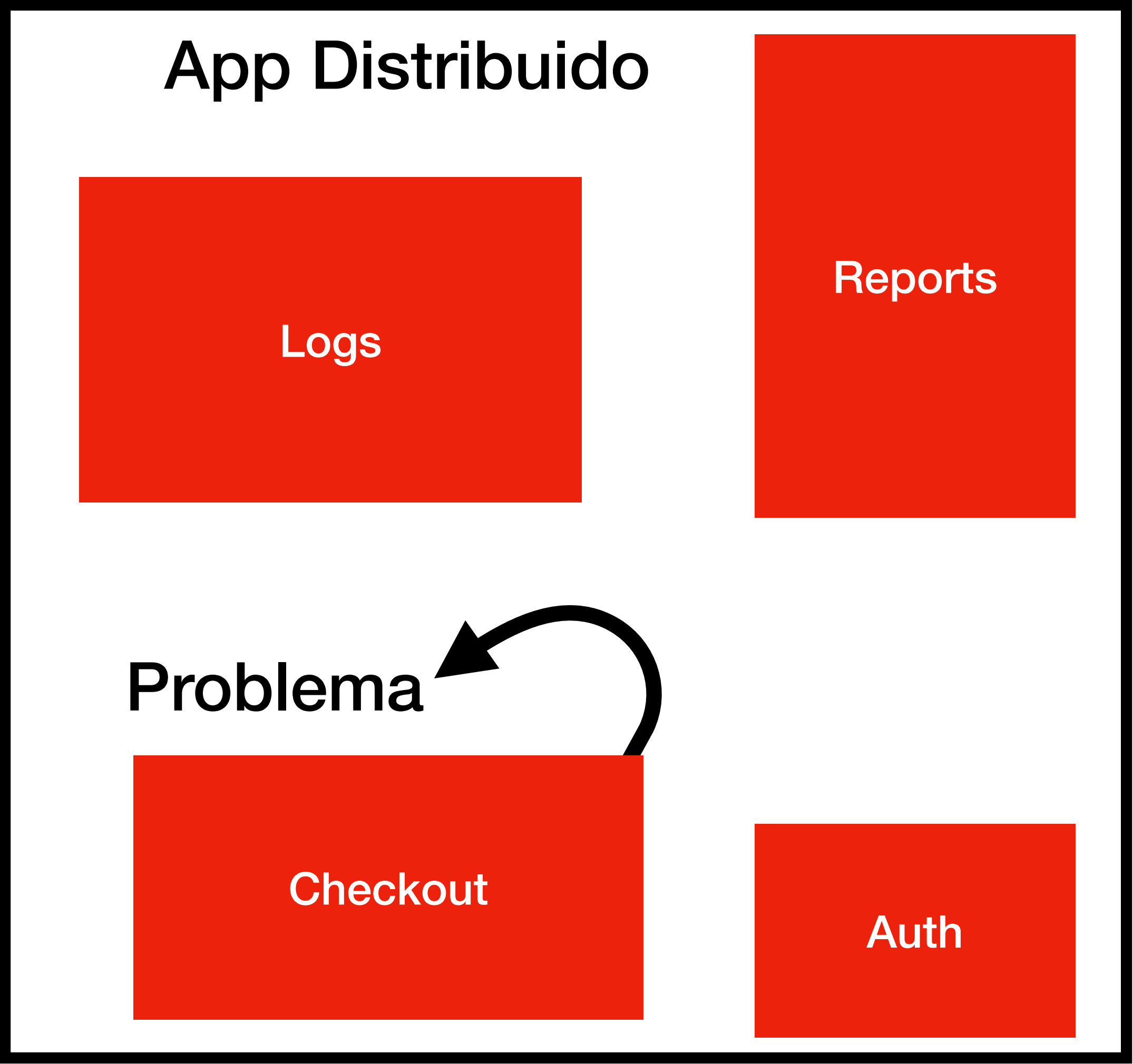
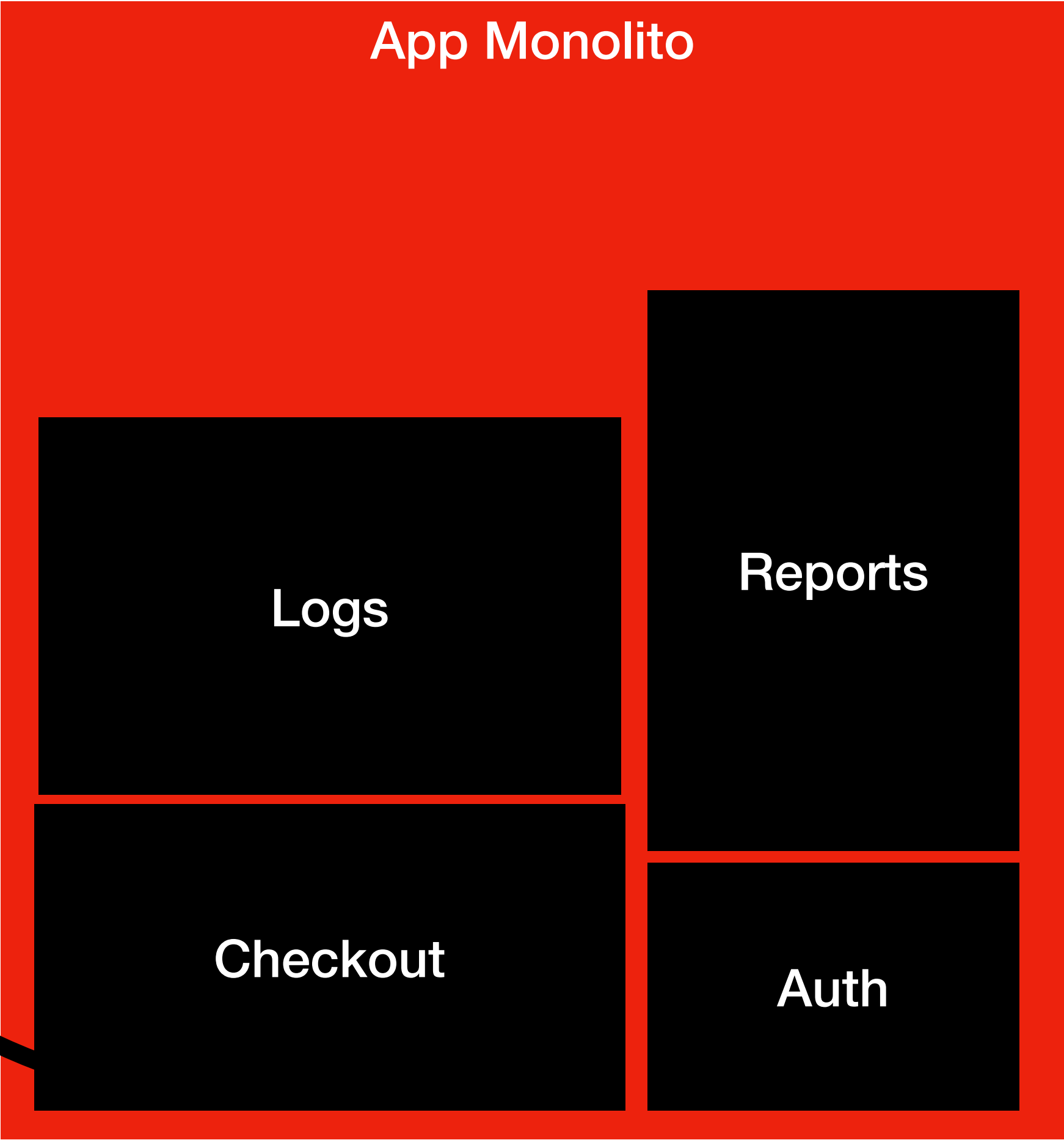
Como estar pronto
técnicamente e emocionalmente
para migrar?

Técnicamente

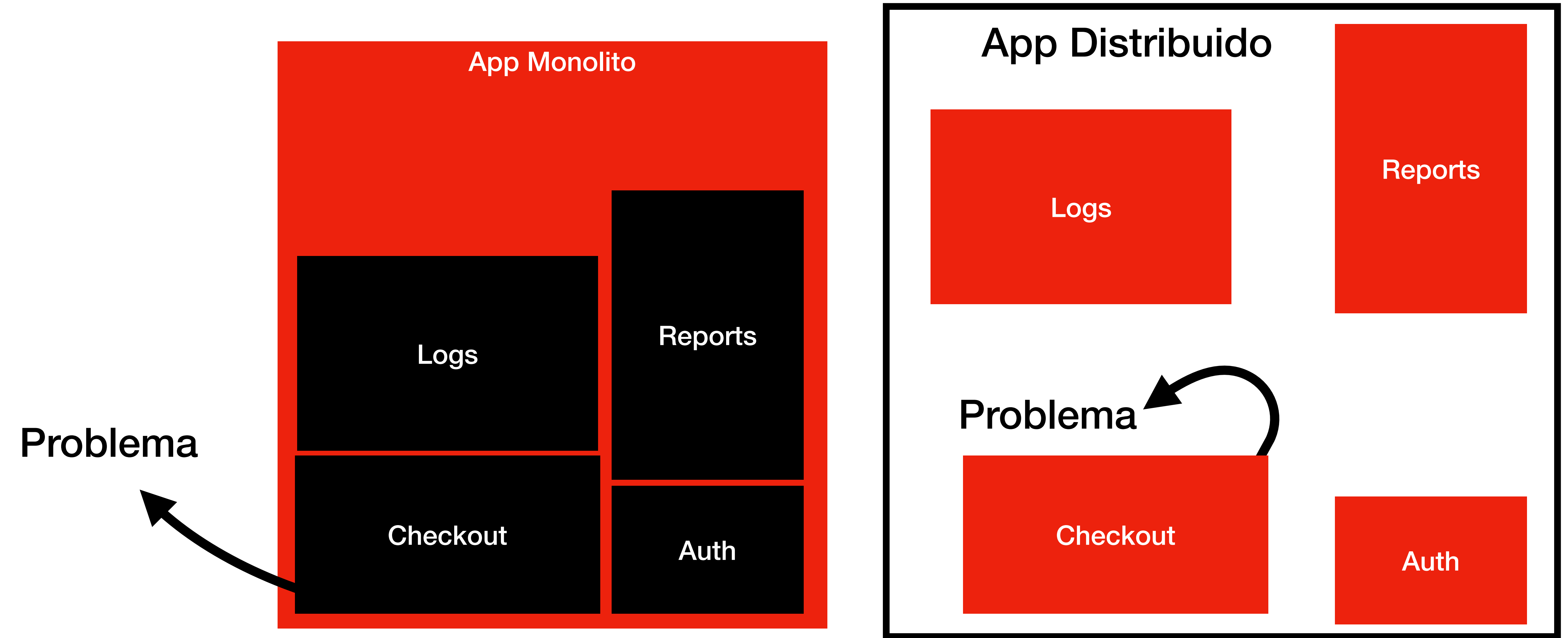
Refatorar antes de migrar



Problema

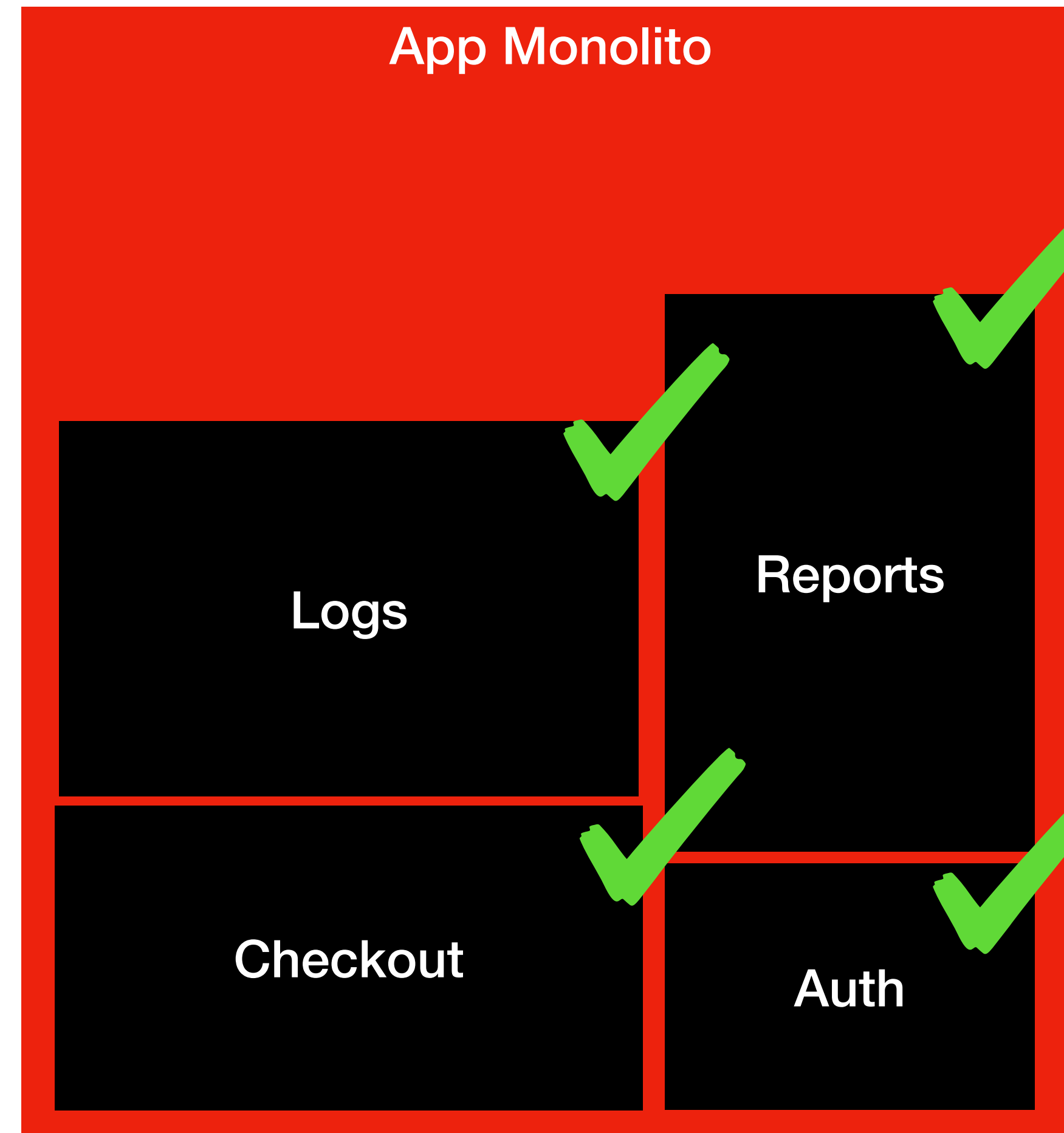


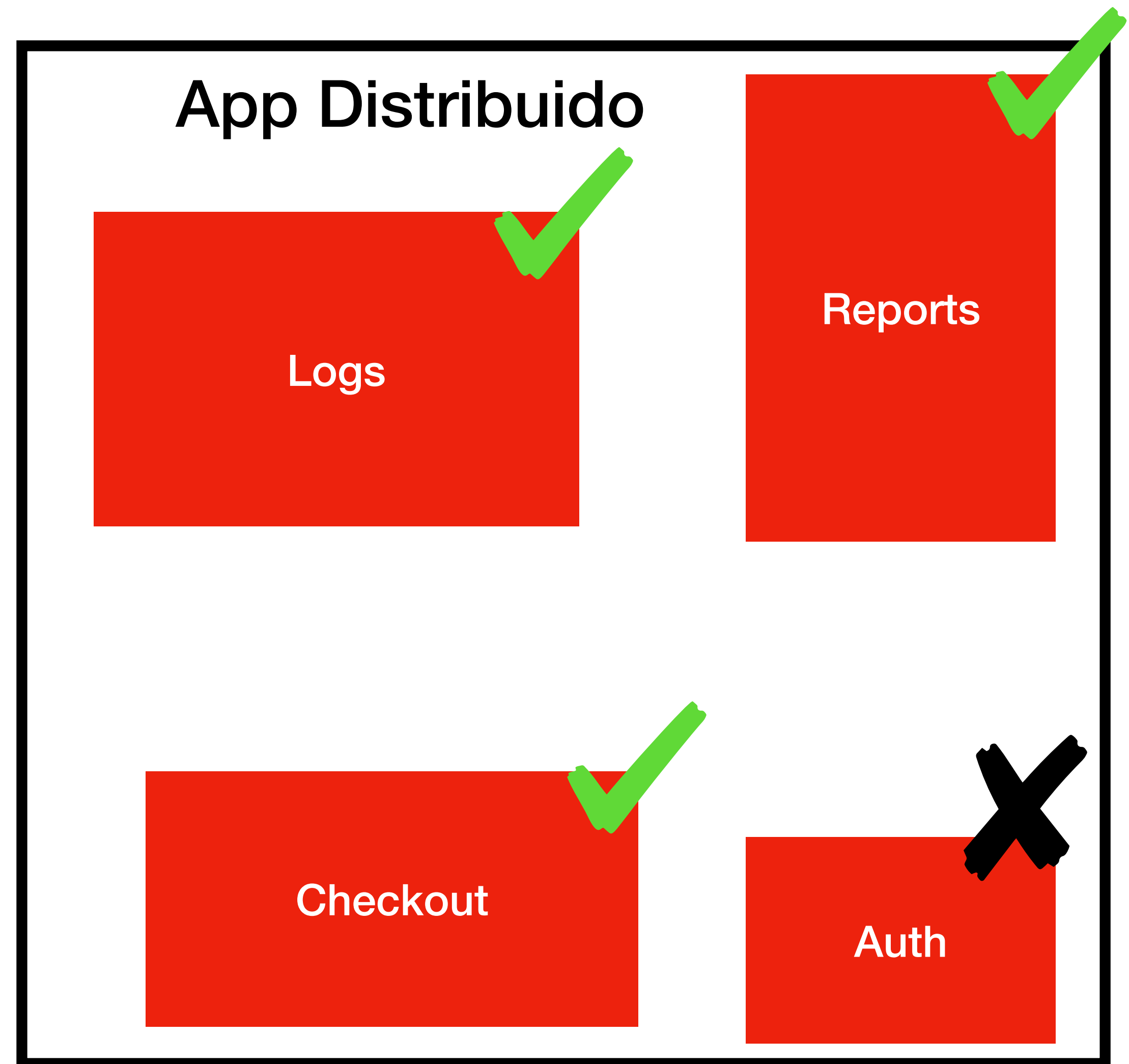
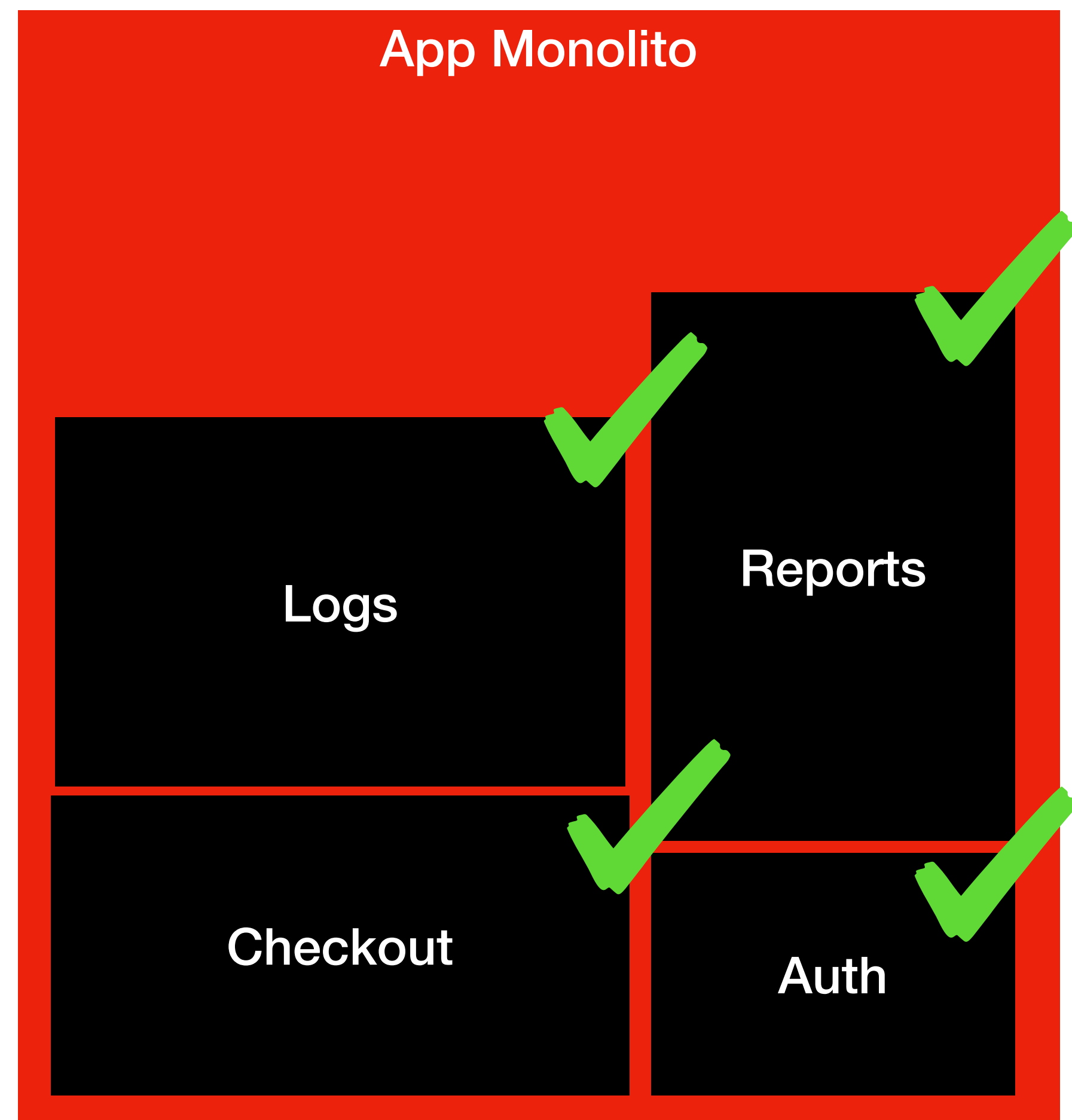
Agora seu problema só mudou de pod

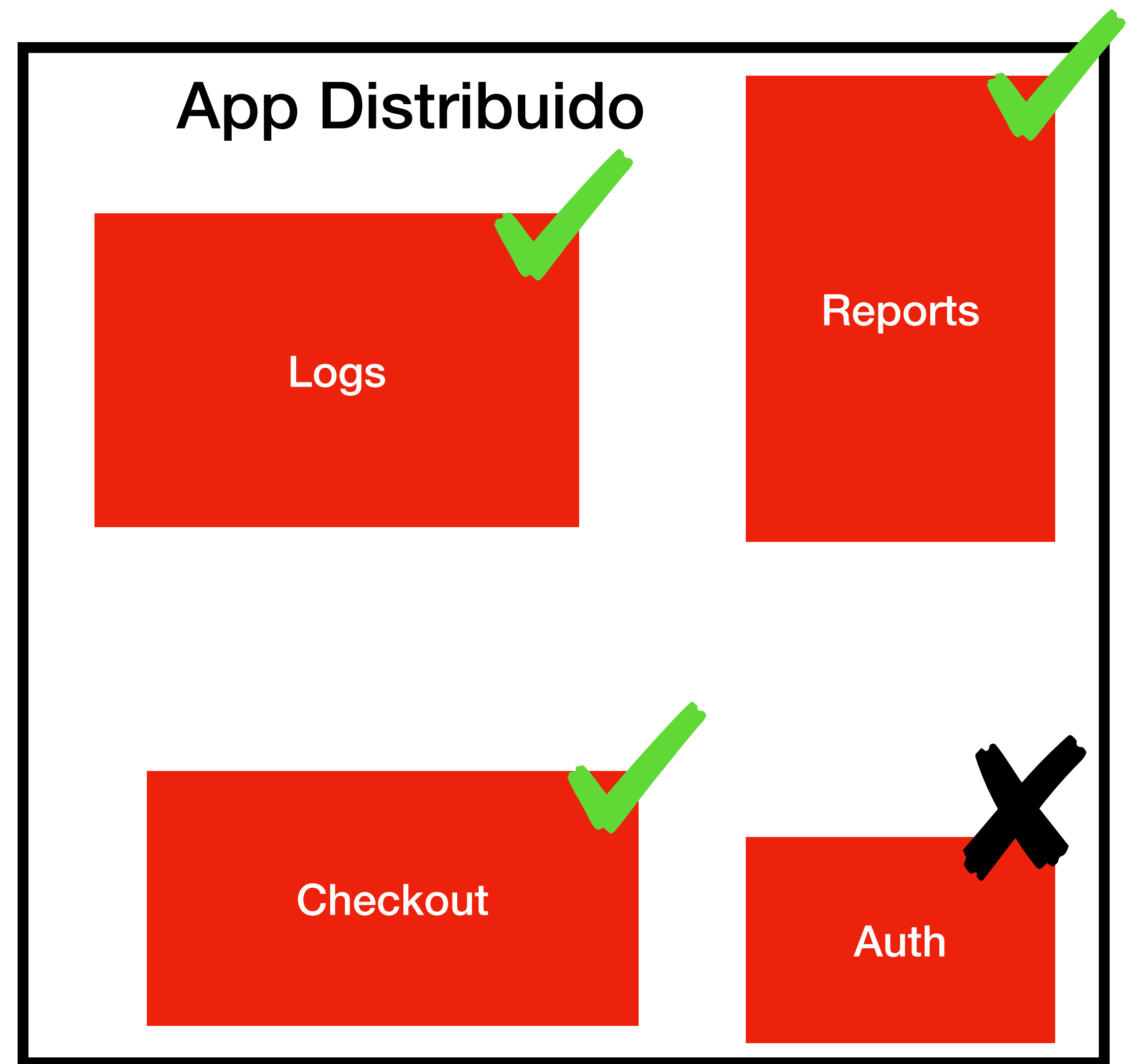
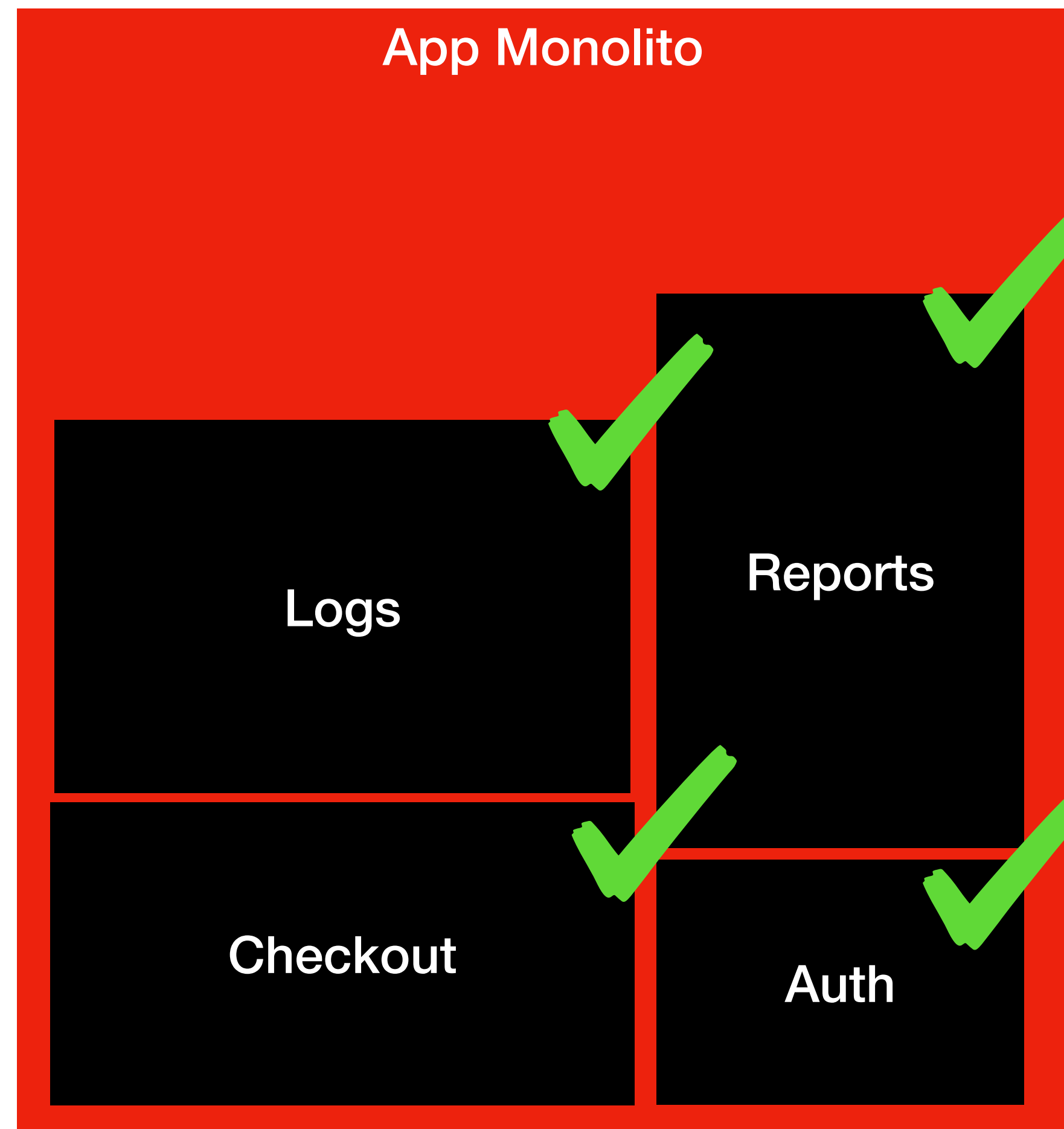


Refatorar antes de migrar!!!!

Testes







São os **testes** que garantem a **qualidade** da migração

```
<?php

use PhpPact\Consumer\PactBuilder;
use PhpPact\Consumer\Matcher\Like;

$pact = (new PactBuilder())
    ->setConsumer('FrontendApp')
    ->setProvider('AuthService')
    ->build();

$pact->addInteraction()
    ->given('Usuário válido existe')
    ->uponReceiving('requisição de login')
    ->with([
        'method' => 'POST',
        'path'    => '/auth/login',
        'body'    => [
            'email'    => 'user@example.com',
            'password' => 'secret'
        ],
        'headers' => [
            'Content-Type' => 'application/json'
        ]
    ])
    ->willRespondWith([
        'status' => 200,
        'body'   => [
            'token' => Like::like('eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...')
        ],
        'headers' => [
            'Content-Type' => 'application/json'
        ]
    ]);

$pact->run(fn() => {
    $response = Http::post('http://localhost:7200/auth/login', [
        'email' => 'user@example.com',
        'password' => 'secret',
    ]);

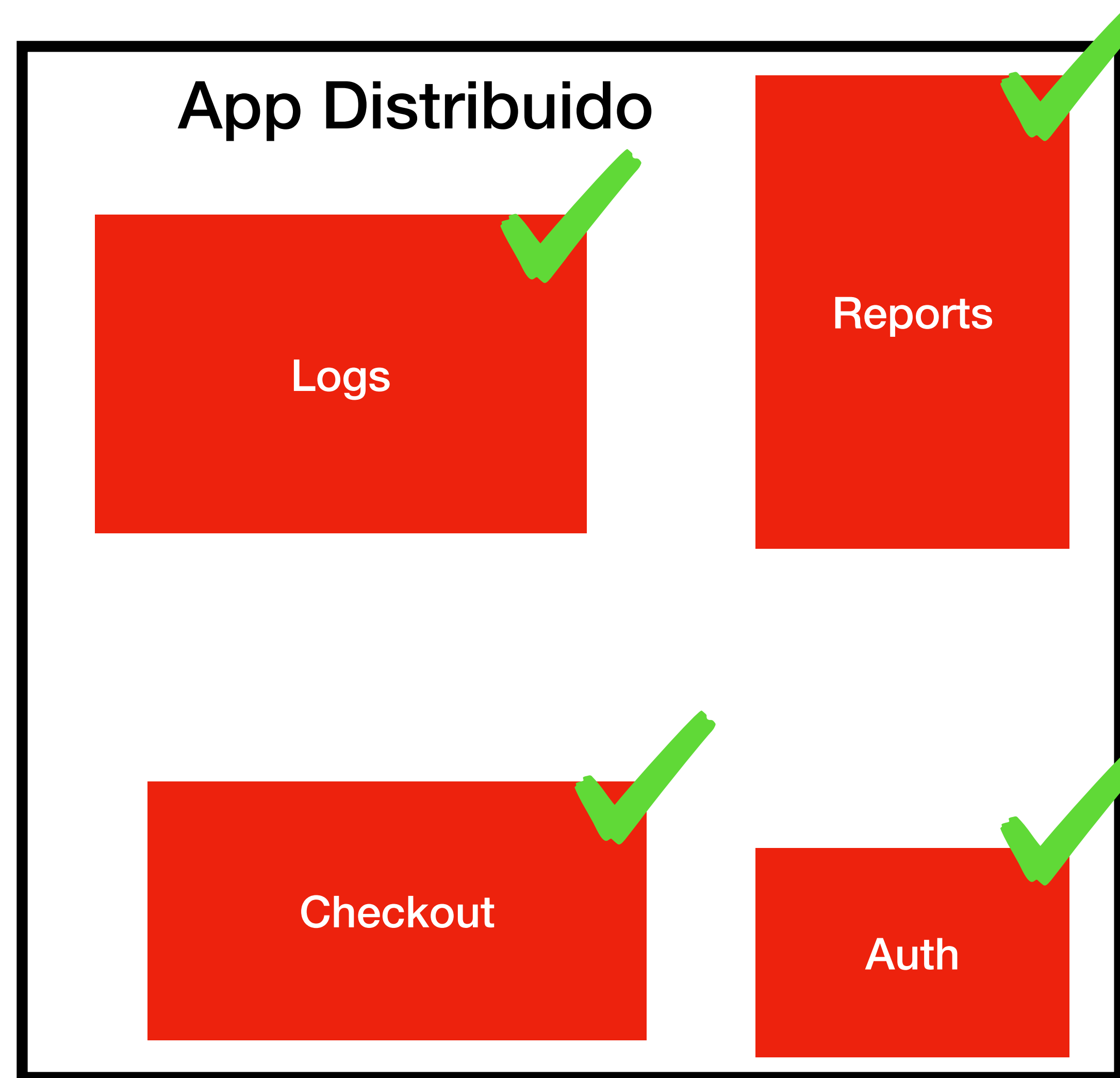
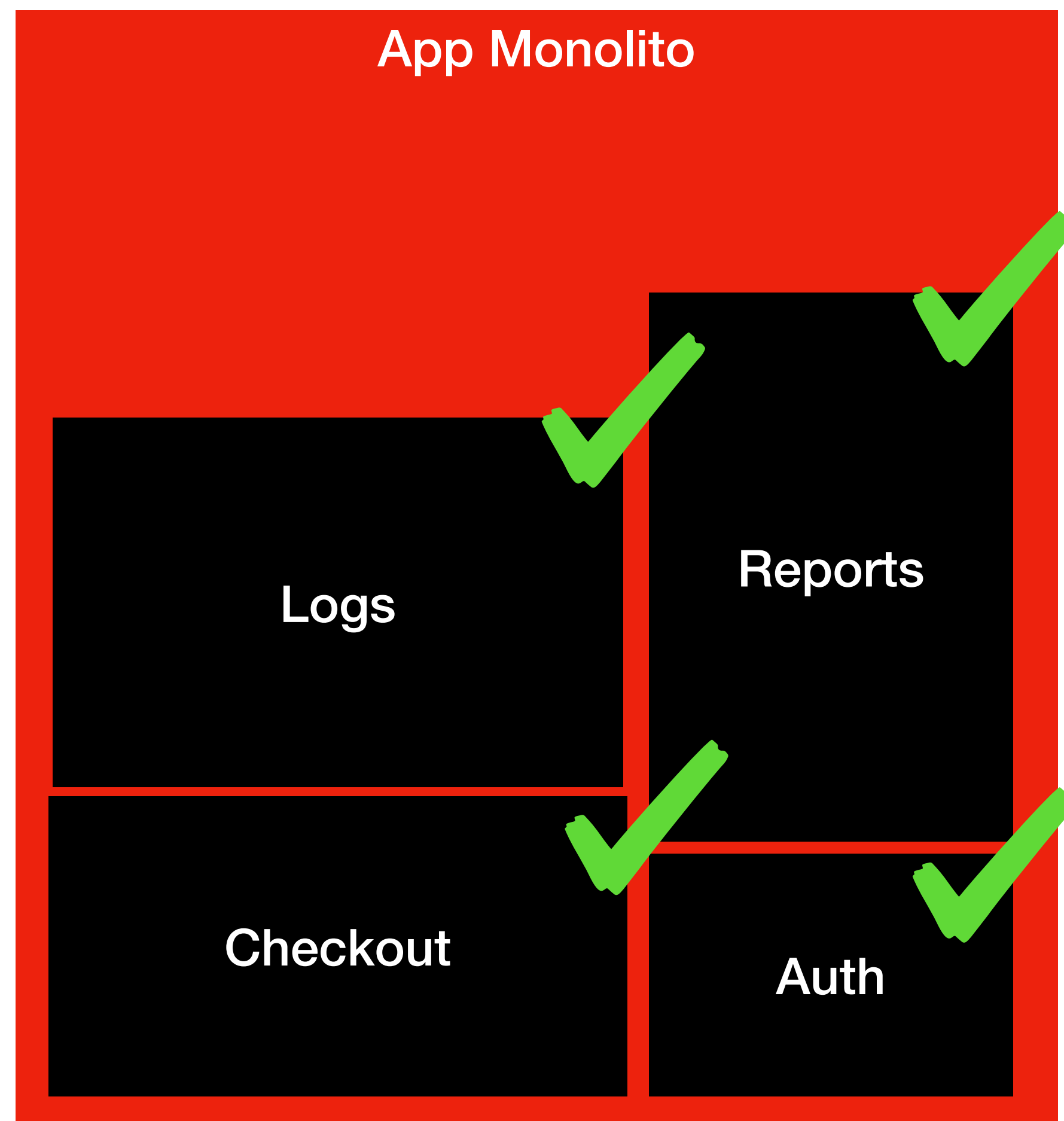
    if (!$response->ok()) {
        throw new Exception('Contrato violado: status diferente de 200');
    }

    if (!isset($response['token'])) {
        throw new Exception('Contrato violado: token ausente');
    }
});
```

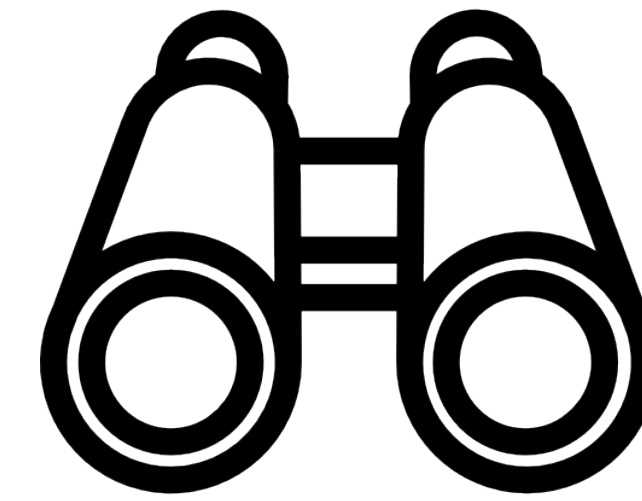
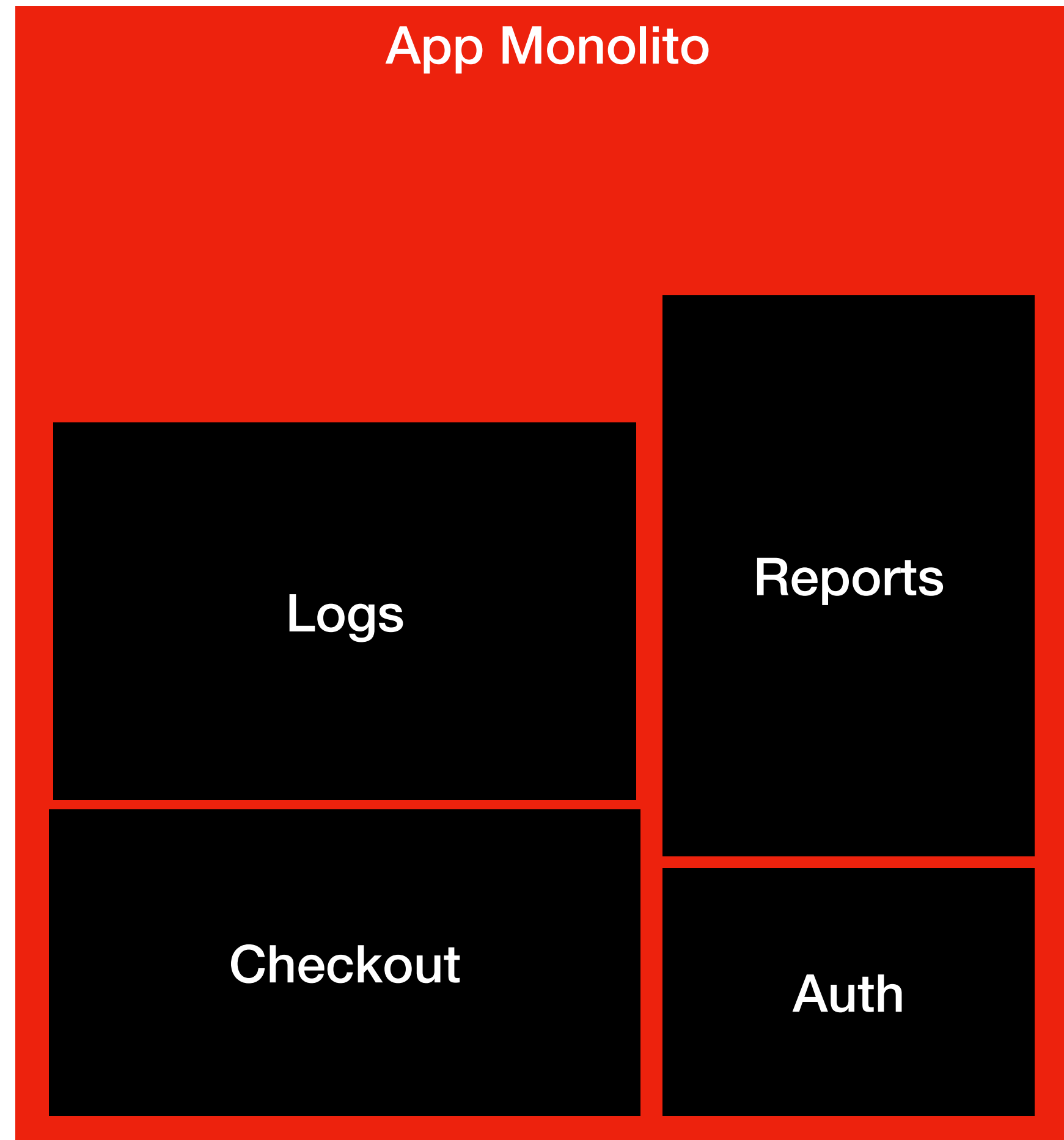
```
$pact->addInteraction()  
  ->given('Usuário válido existe')  
  ->uponReceiving('requisição de login')  
  ->with([  
    'method'    => 'POST',  
    'path'      => '/auth/login',  
    'body'      => [  
      'email'    => 'user@example.com',  
      'password' => 'secret'  
    ],  
    'headers' => [  
      'Content-Type' => 'application/json'  
    ]  
  ])  
  ->willRespondWith([  
    'status'    => 200,  
    'body'      => [  
      'token' => Like::like('eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...')  
    ],  
    'headers' => [  
      'Content-Type' => 'application/json'  
    ]  
  ])  
];
```



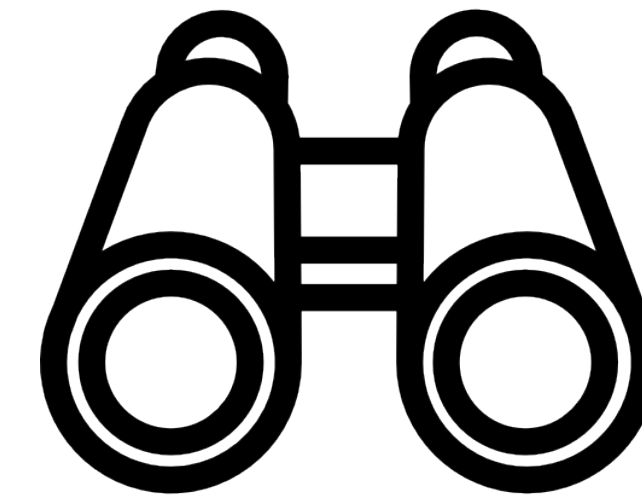
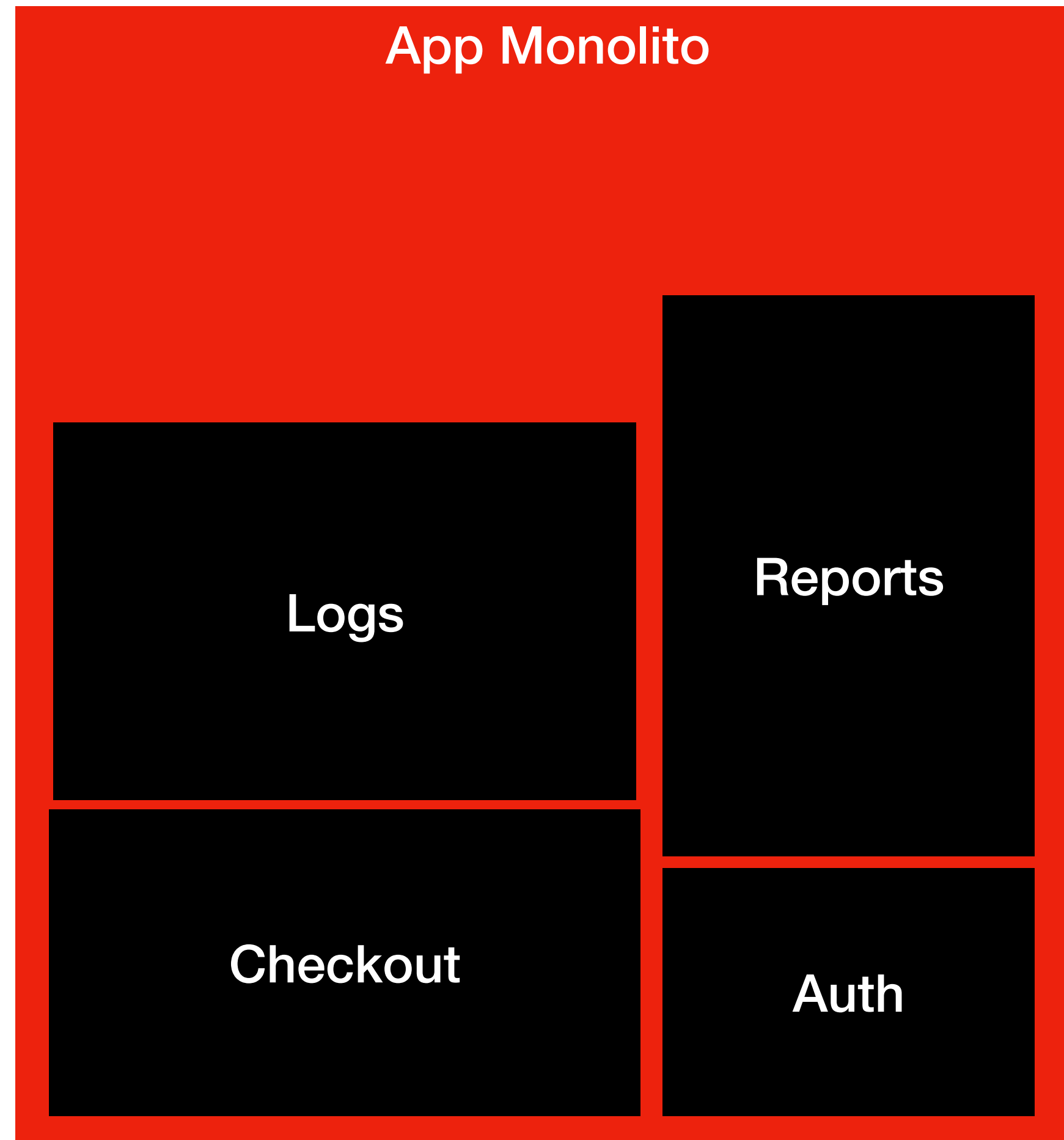
```
$pact->run(fn( ) => {  
    $response = Http::post('http://localhost:7200/auth/login', [  
        'email' => 'user@example.com',  
        'password' => 'secret',  
    ]);  
    if (!$response->ok()) {  
        throw new Exception('Contrato violado: status diferente de 200');  
    }  
  
    if (!isset($response['token'])) {  
        throw new Exception('Contrato violado: token ausente');  
    }  
});
```



Observabilidade

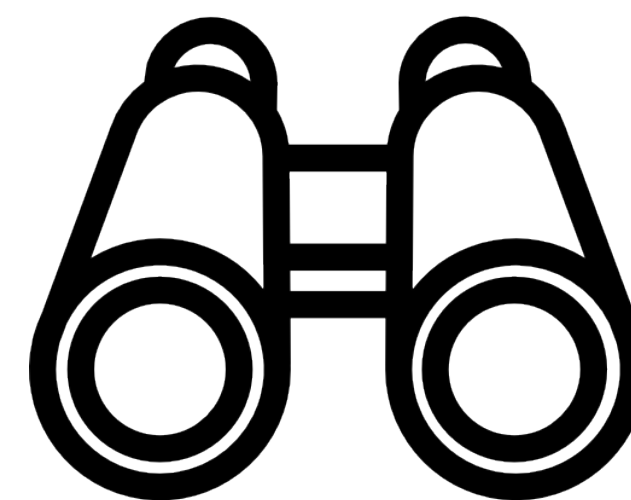
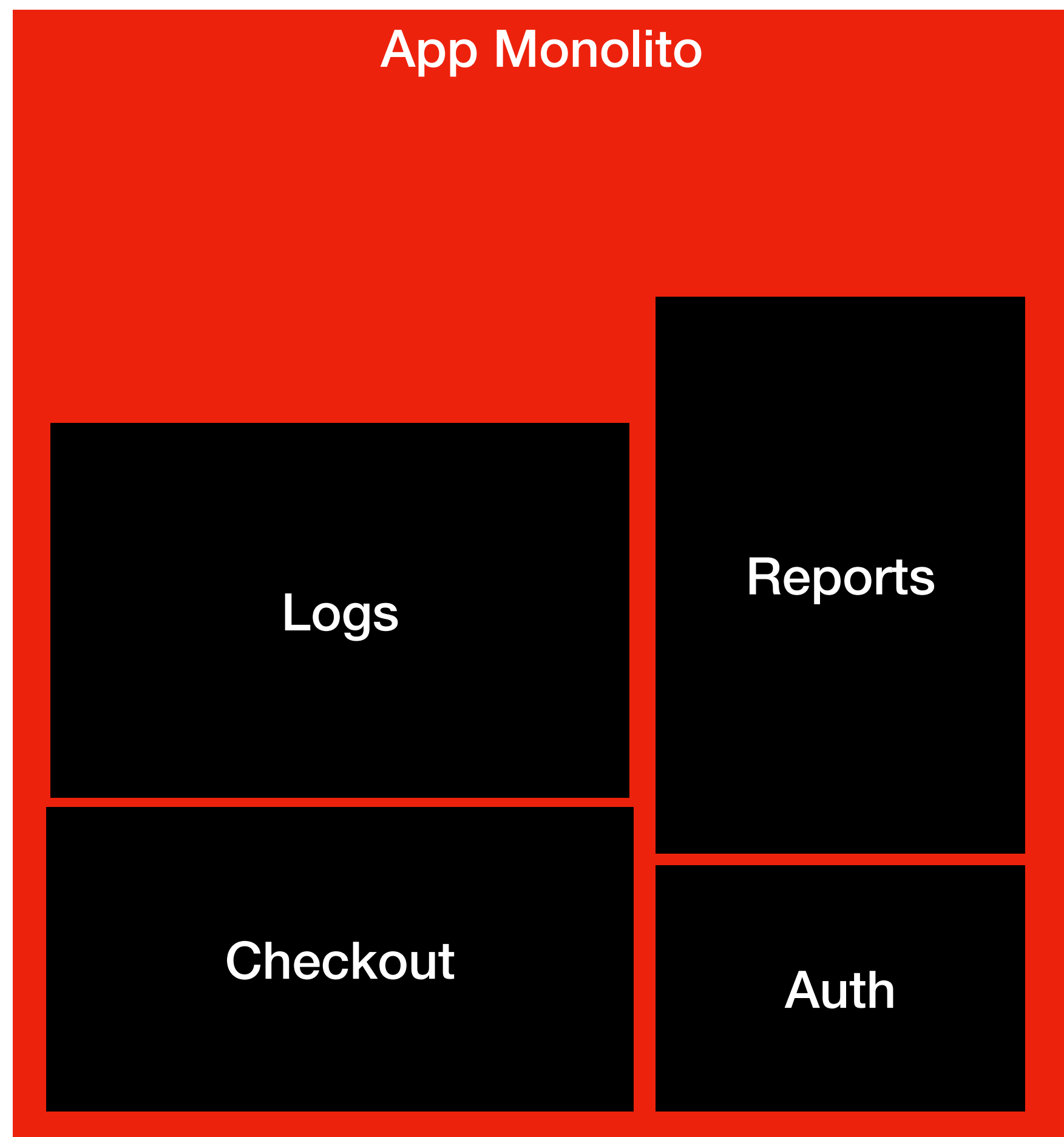


3 Pilares



3 Pilares

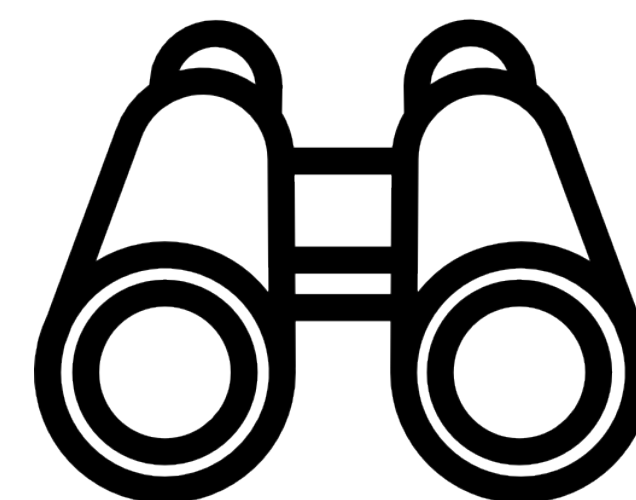
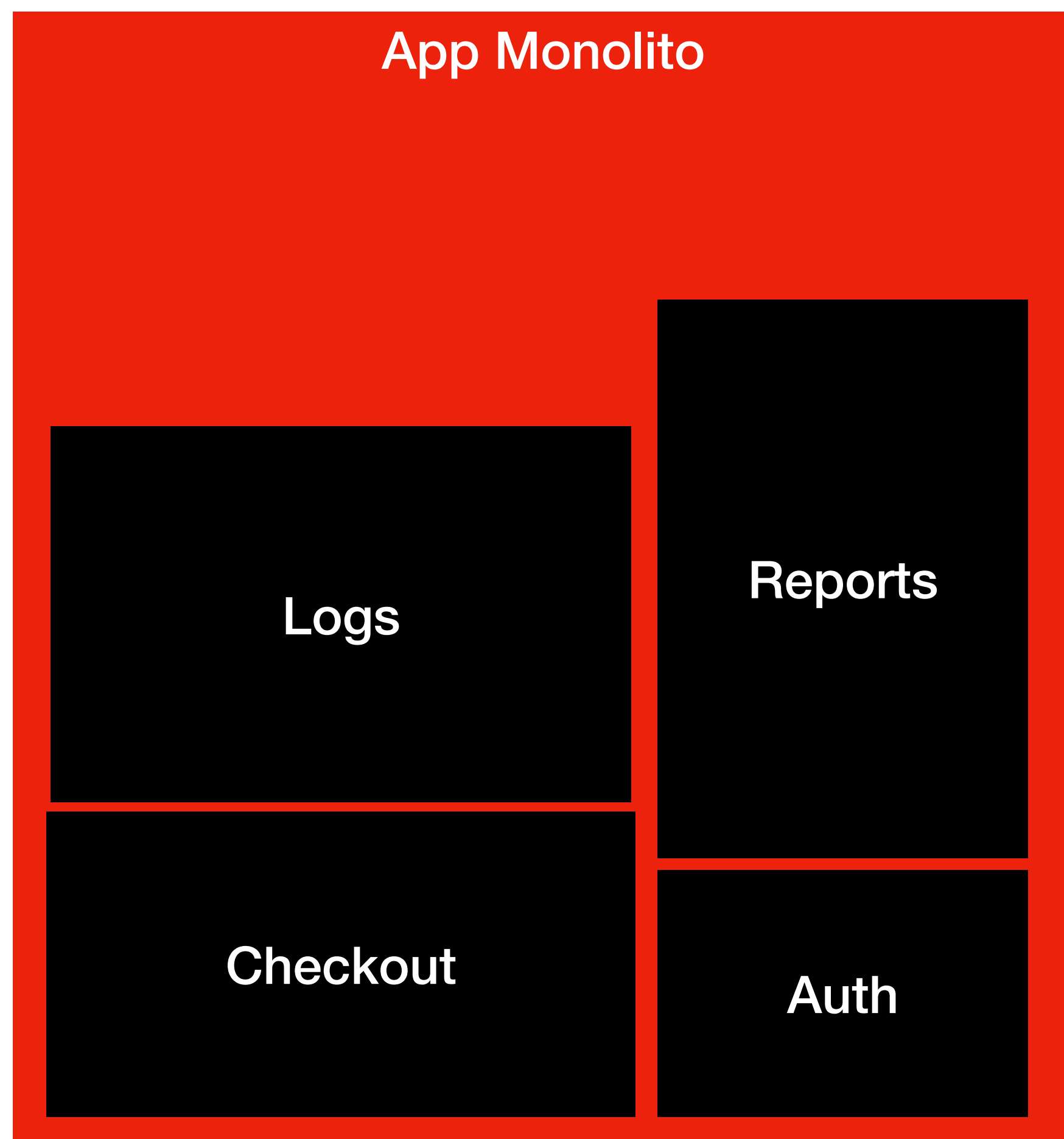
Logs



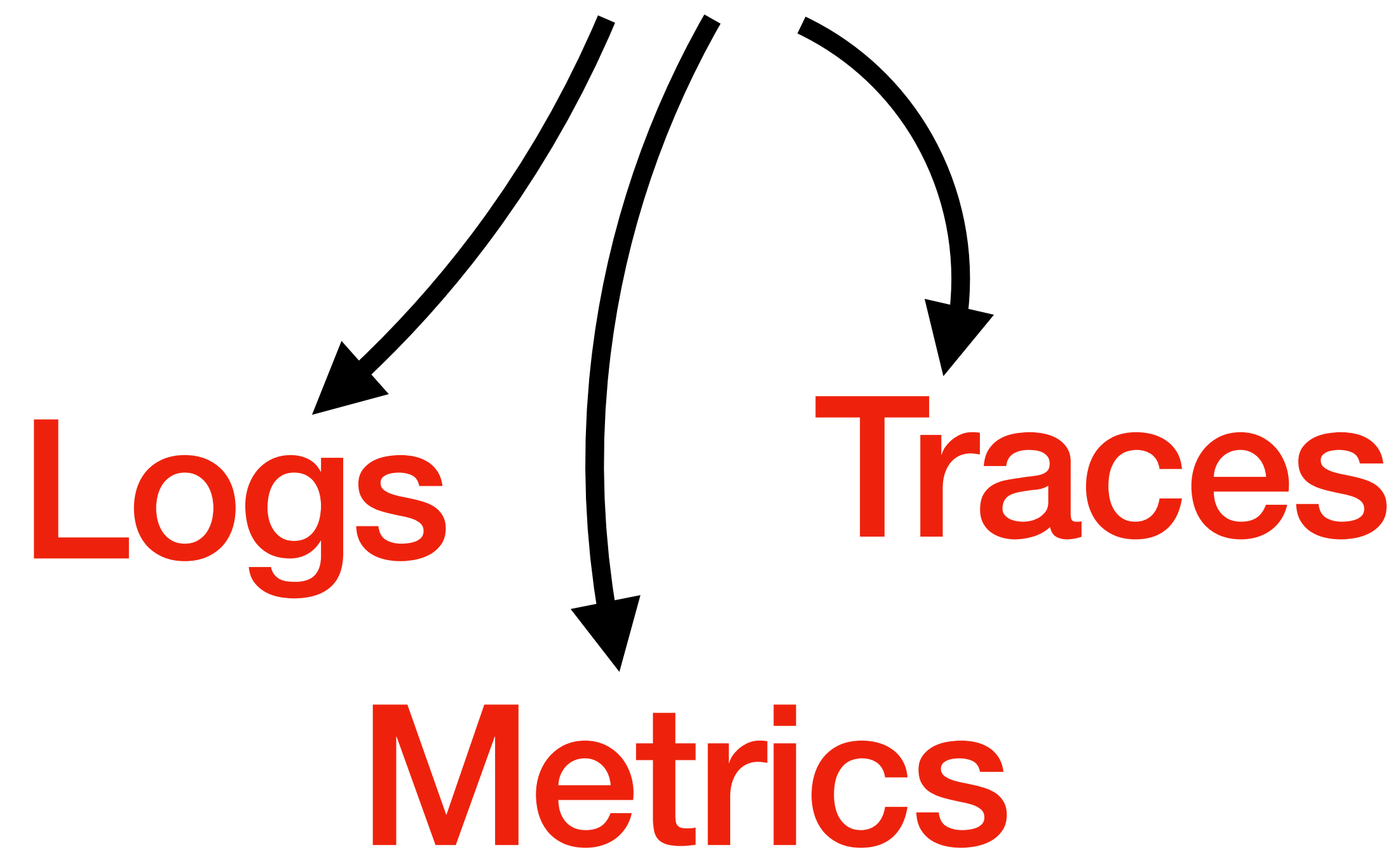
3 Pilares

Logs

Metrics



3 Pilares



Logs

Logs são registros cronológicos de eventos gerados por um sistema.

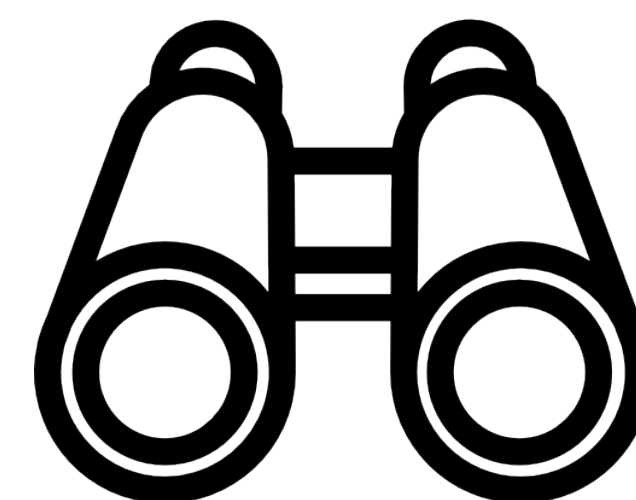
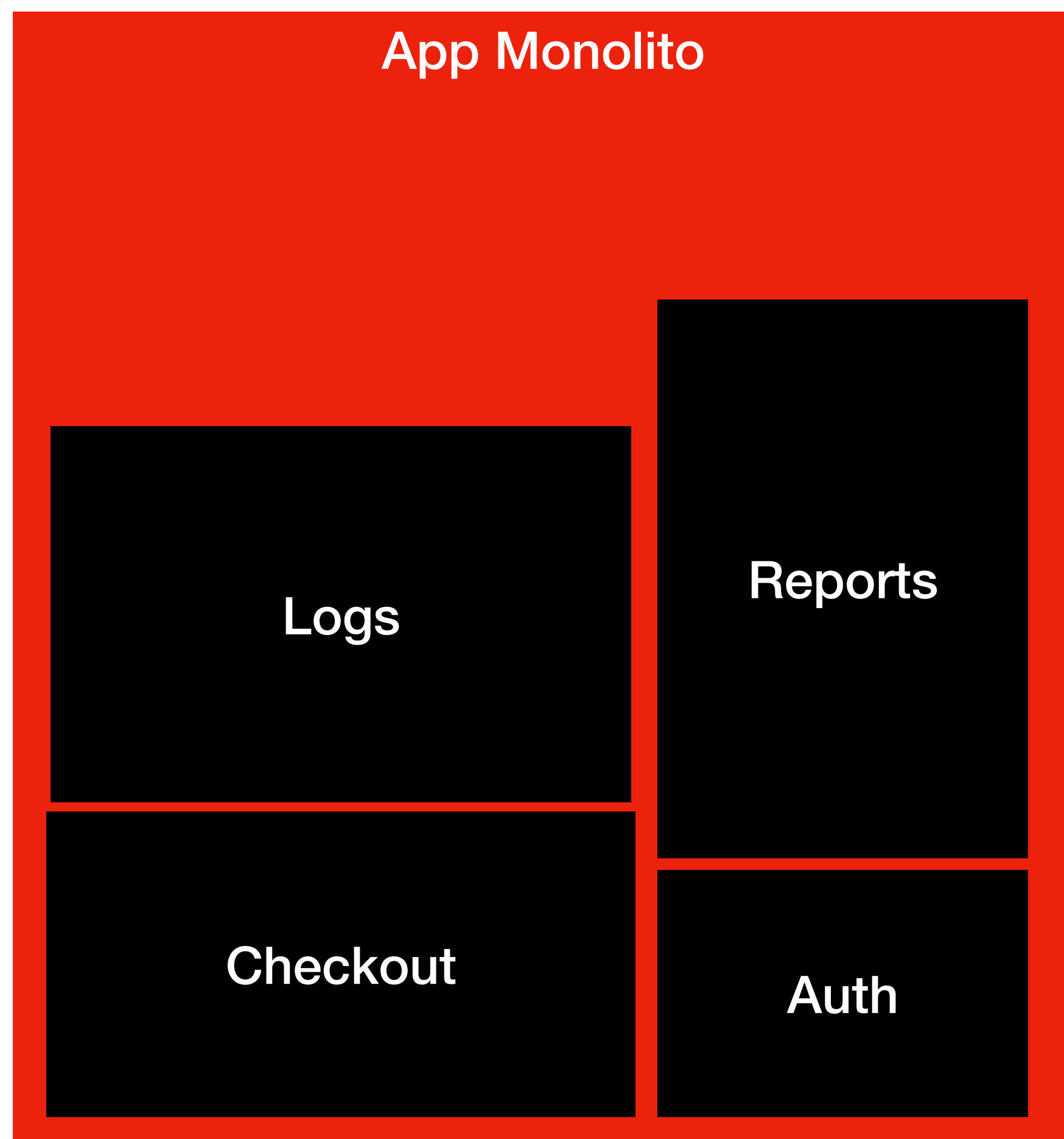
```
2025-11-22 10:14:36 INFO retry-worker Retrying operation attempt=1 task=update_order_status
```

```
2025-11-22 10:14:37 INFO retry-worker Operation successful task=update_order_status
```

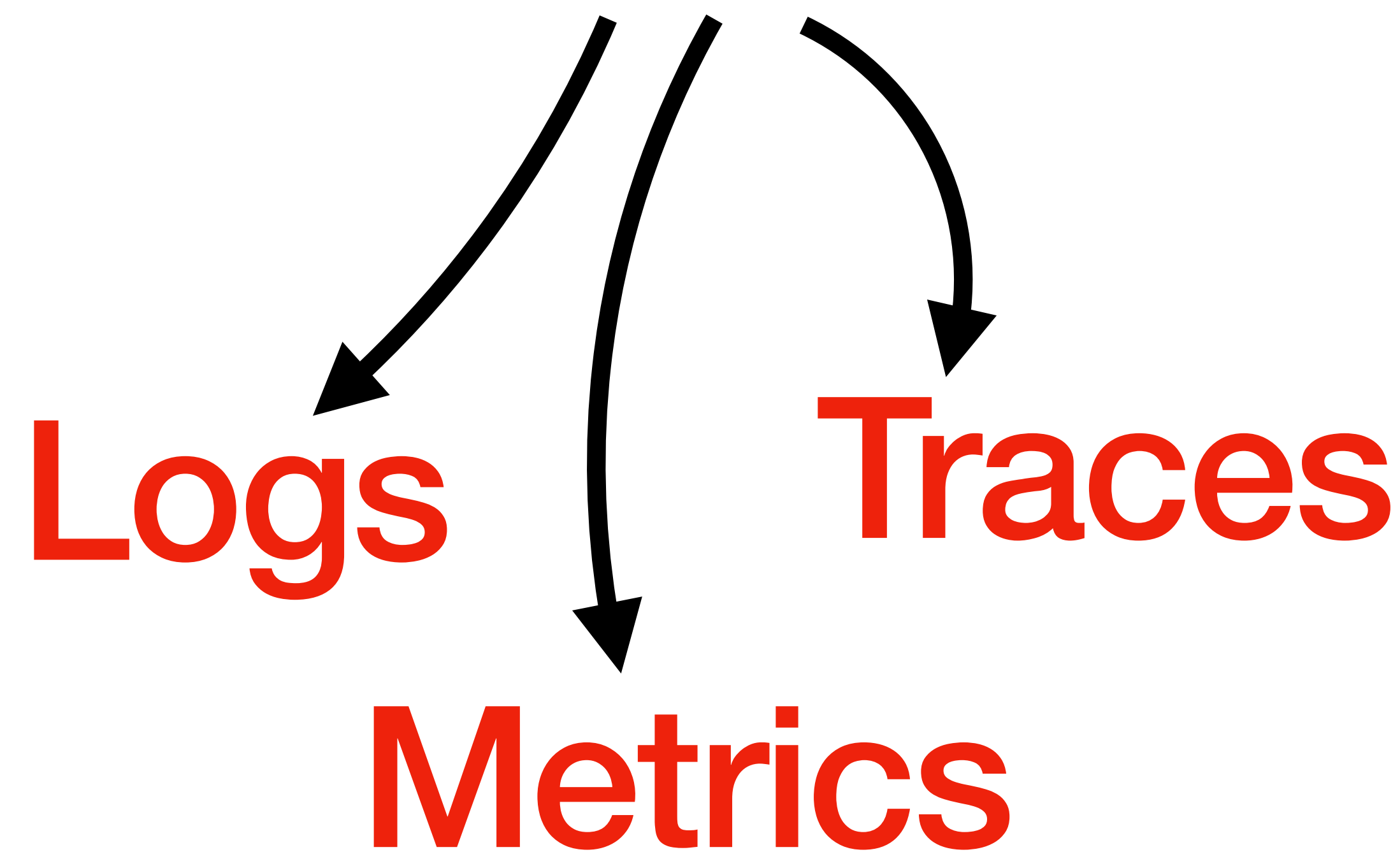
```
2025-11-22 10:14:32 INFO order-service Request received user_id=182 product_id=77
```

```
2025-11-22 10:14:32 DEBUG order-service Validating stock product_id=77 qty=1
```

```
2025-11-22 10:14:33 WARN stock-service Low stock detected product_id=77 remaining=3
```

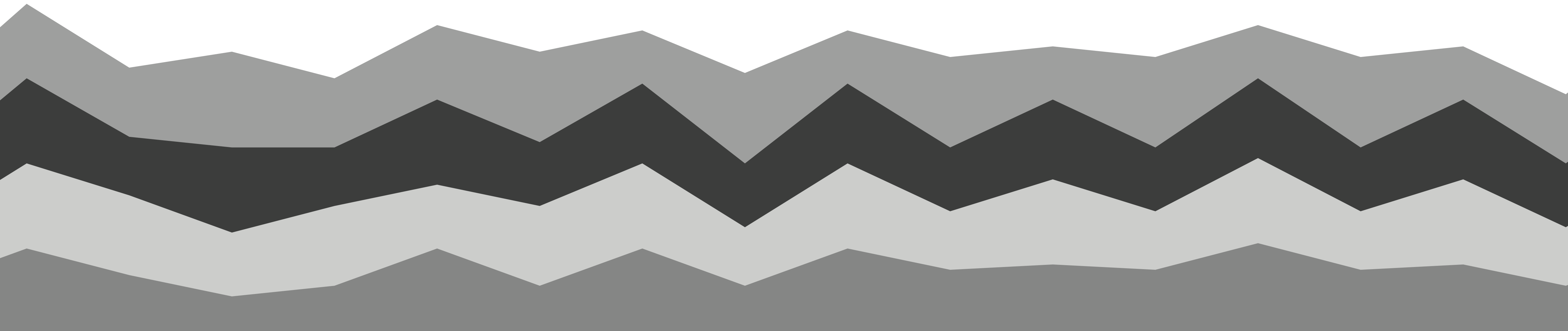



3 Pilares

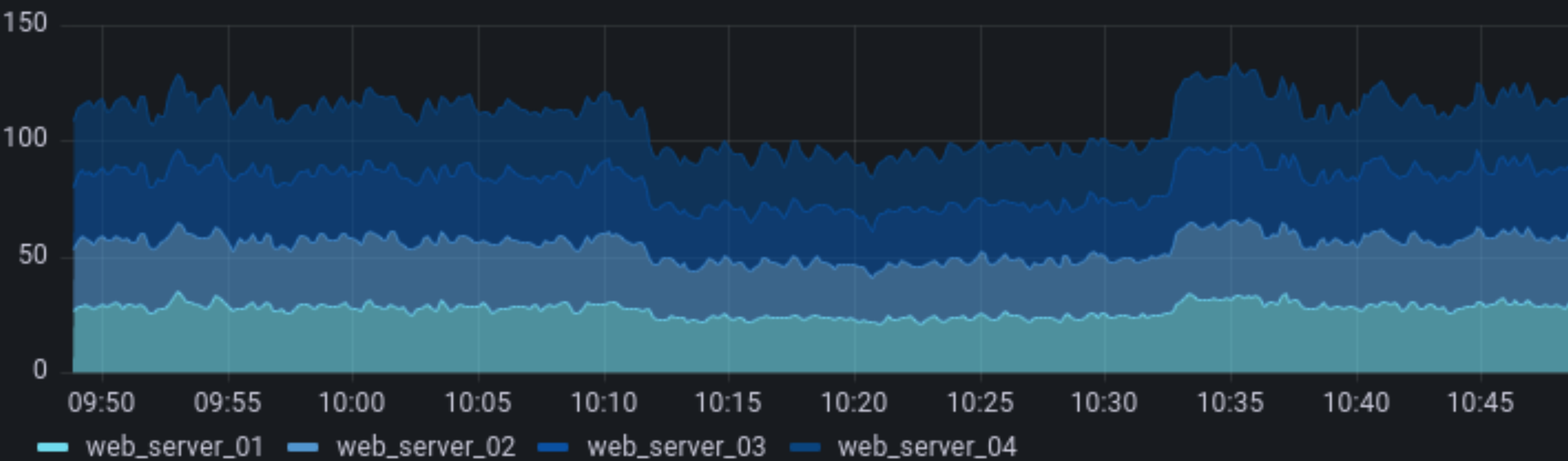


Metrics

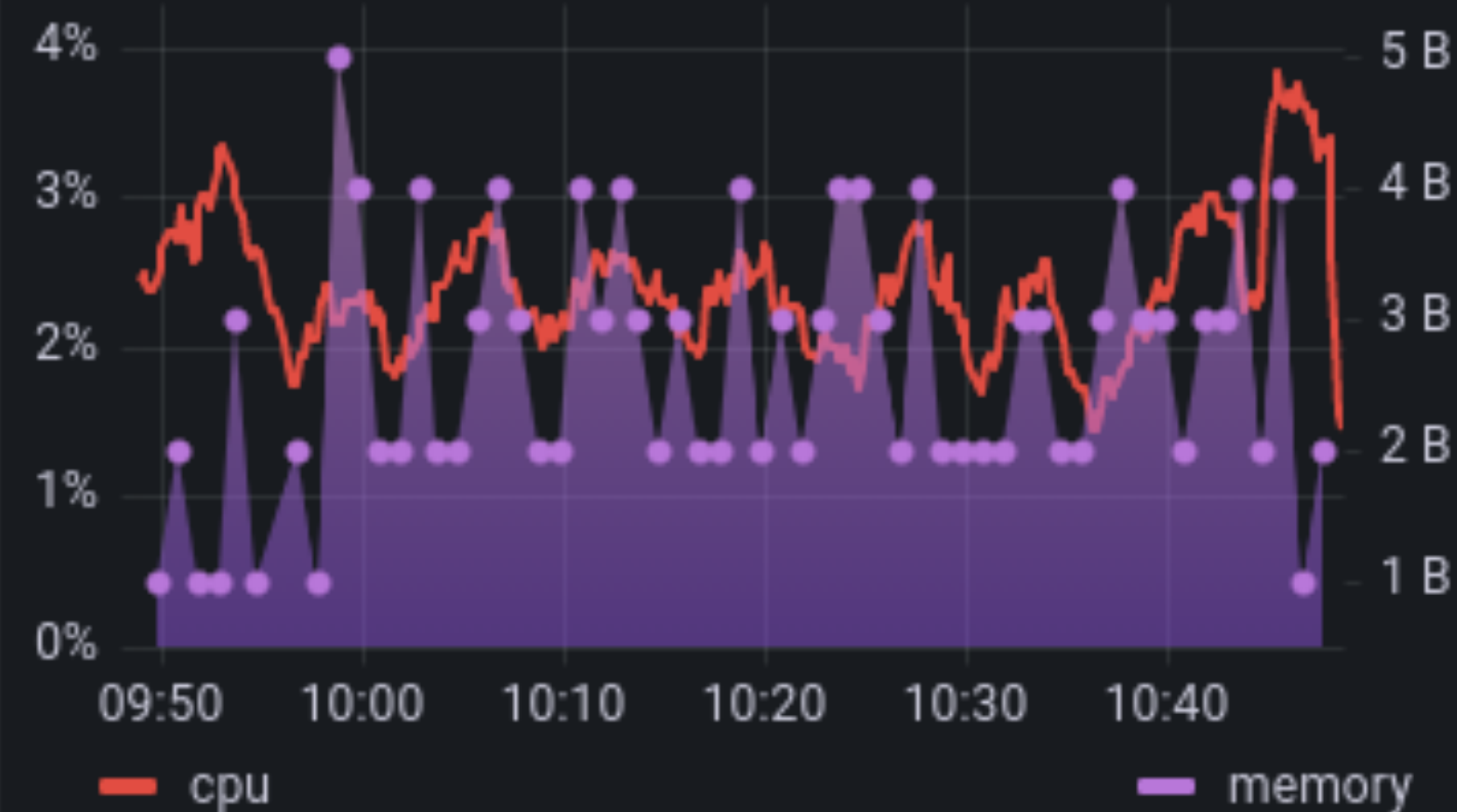
Metrics são valores numéricos coletados ao longo do tempo que quantificam o desempenho e o estado de um sistema



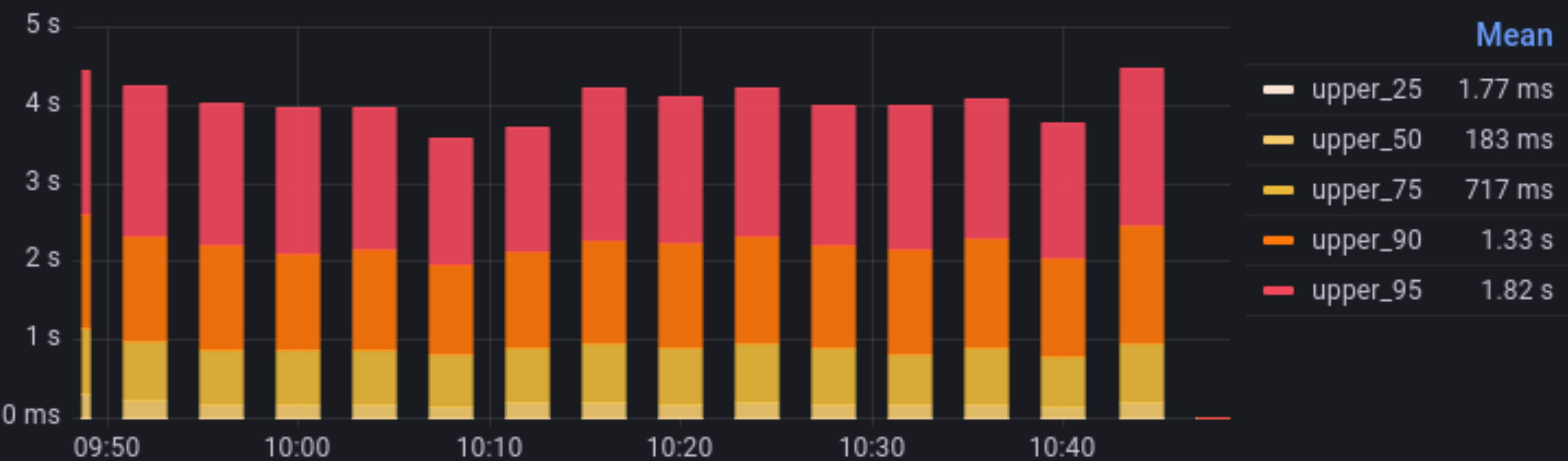
server requests



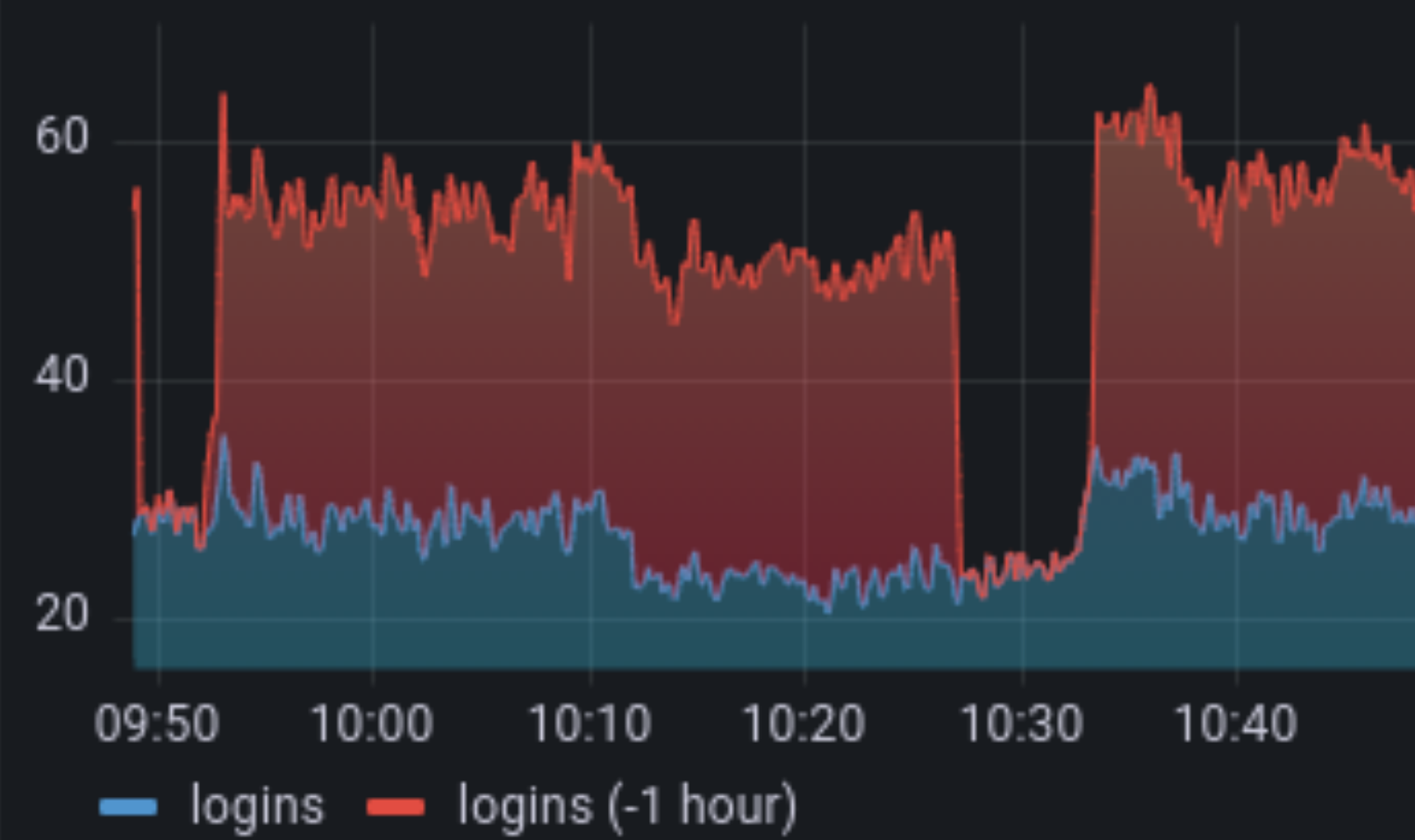
Memory / CPU



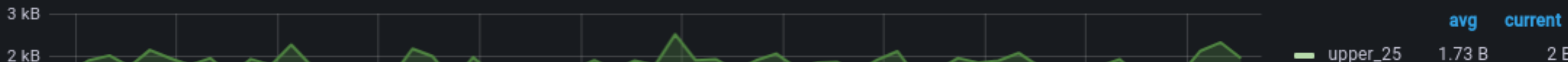
client side full page load

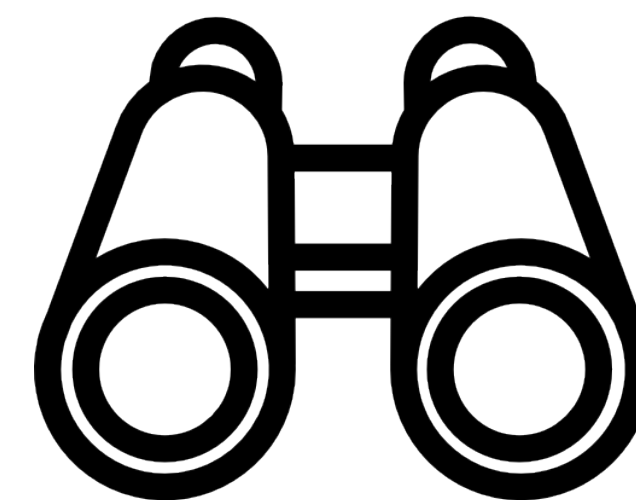
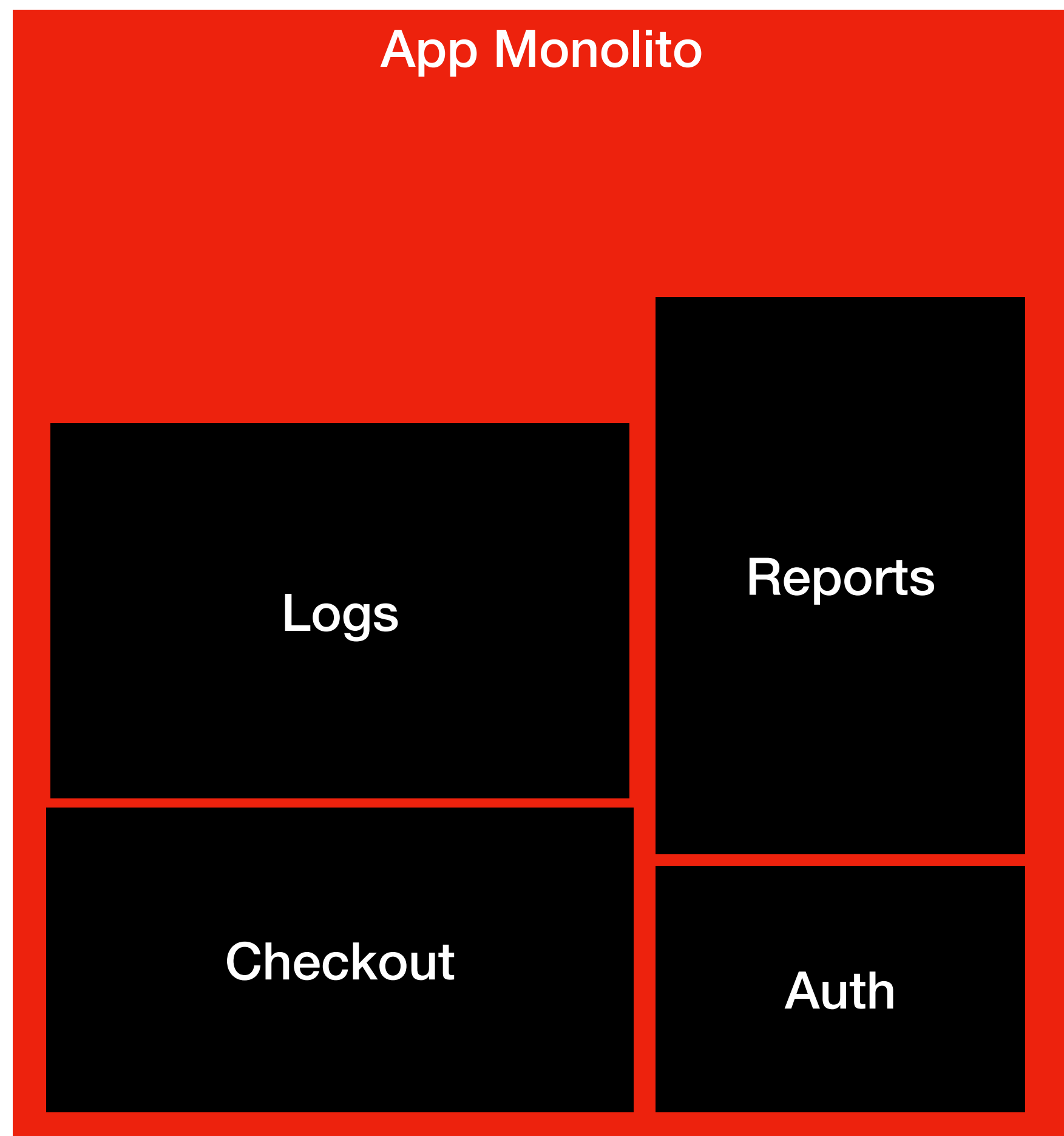


logins

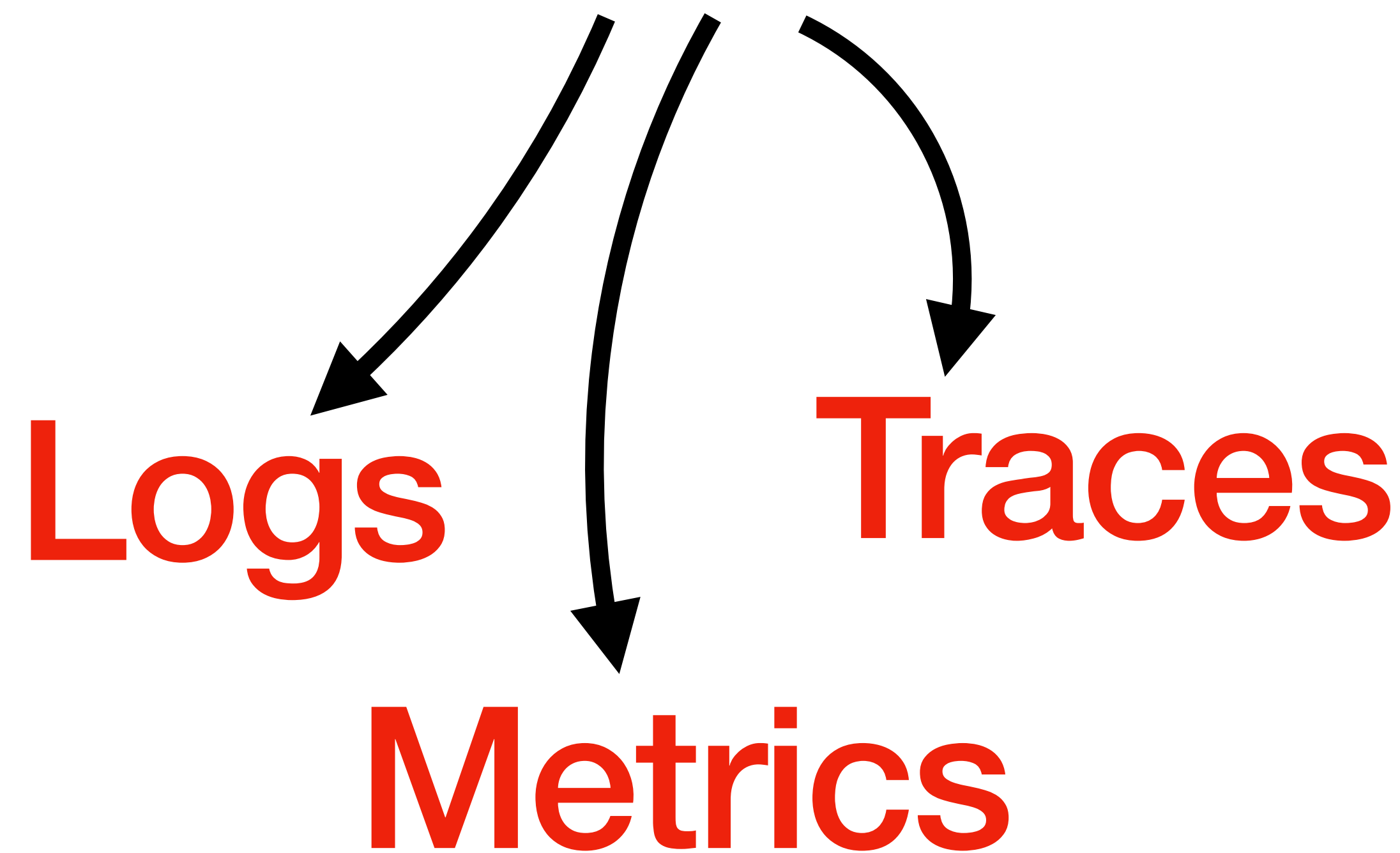


Traffic In/Out



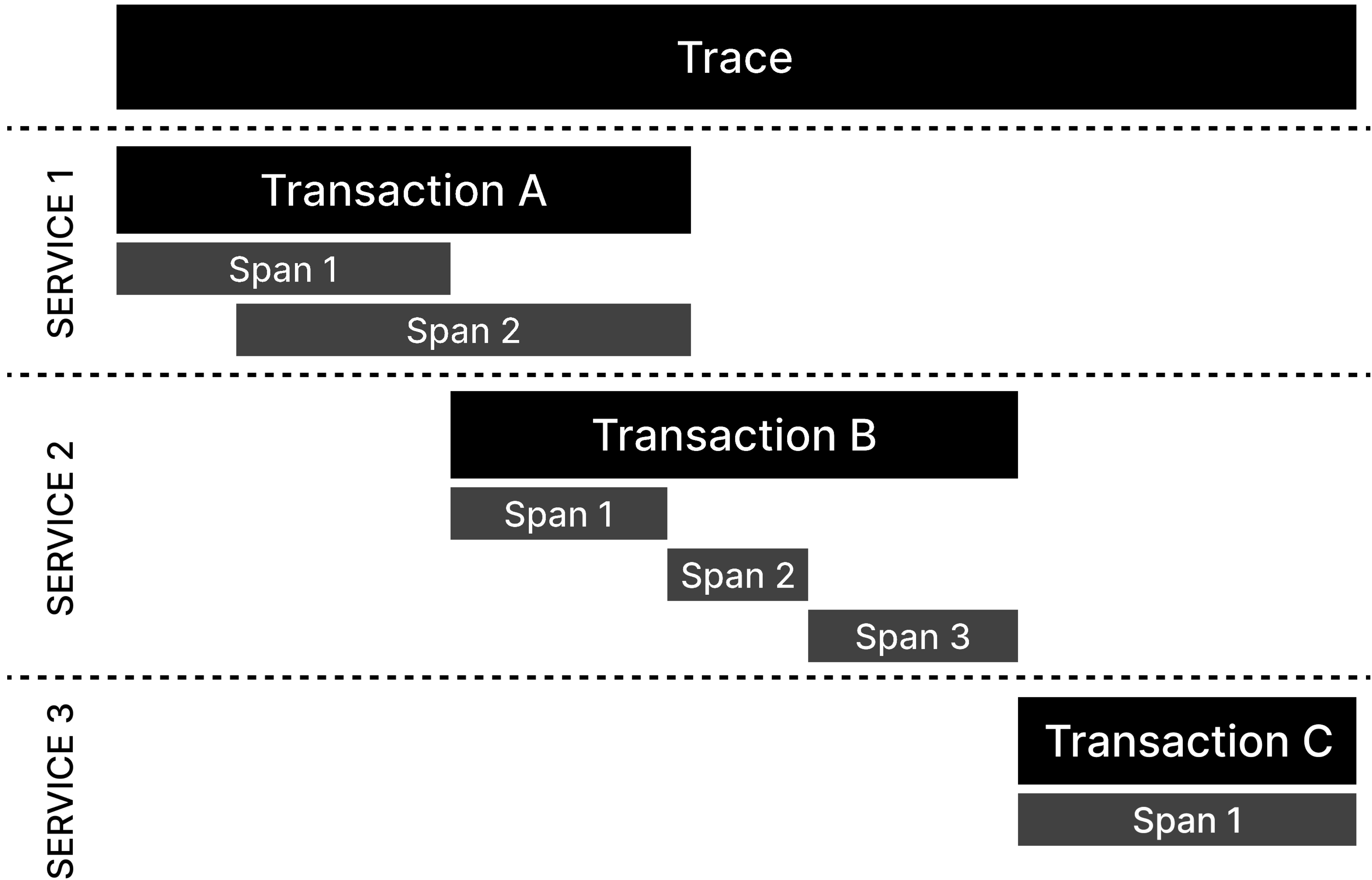


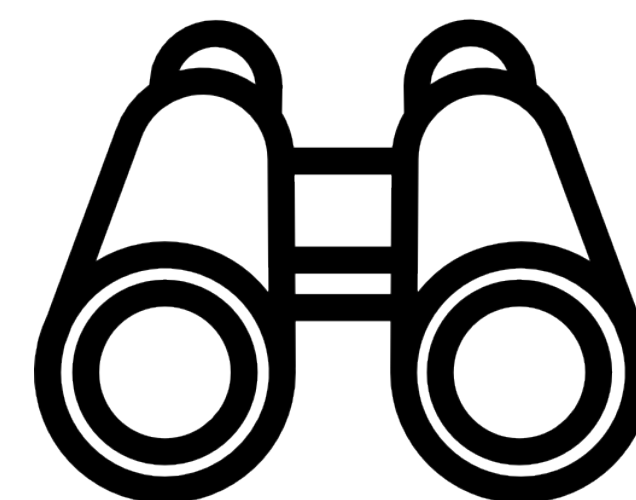
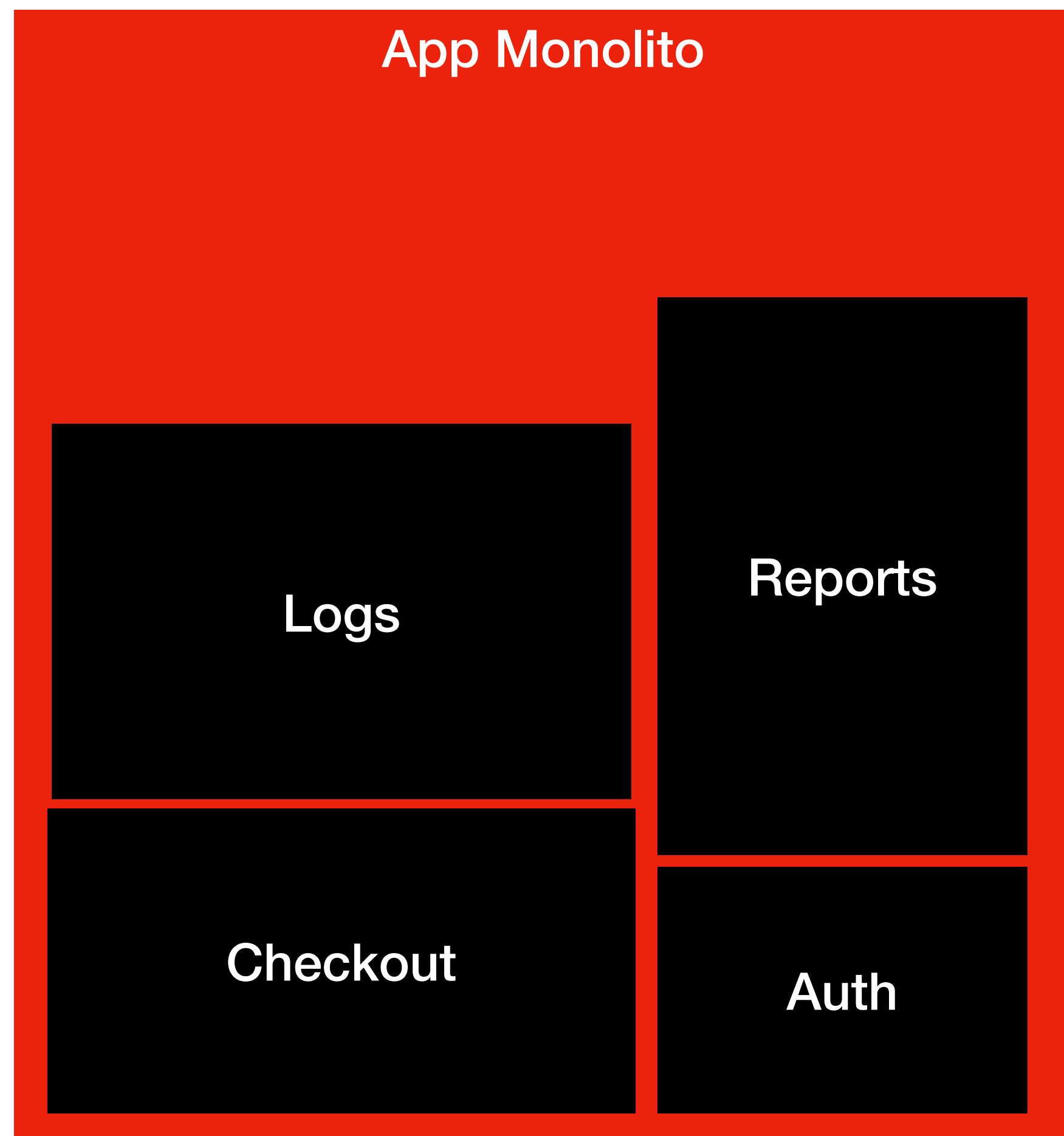
3 Pilares



Traces

Traces são registros do caminho completo que uma requisição percorre através de sistemas e serviços





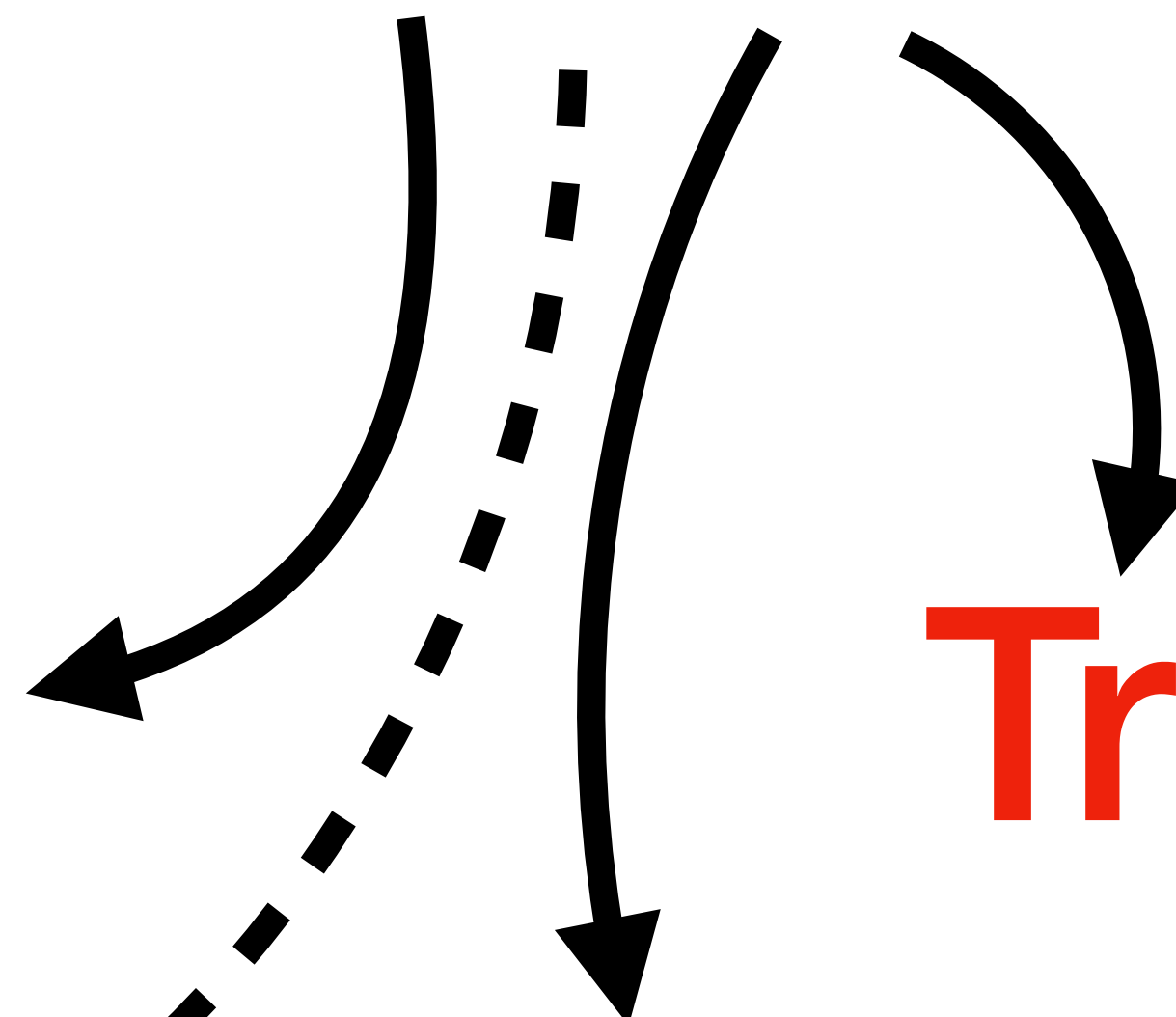
4 Pilares

Logs

Traces

Analytics

Metrics



Logs vs. Analytics

Logs explicam como o sistema se comporta

Analytics explicam como o usuário e o negócio se comportam

Log

```
INFO 2025-02-10T14:33:07 PaymentService gateway_response status=200  
request_id=ac83f01 latency=214ms provider=Stripe  
payload_size=482B
```

Analytics

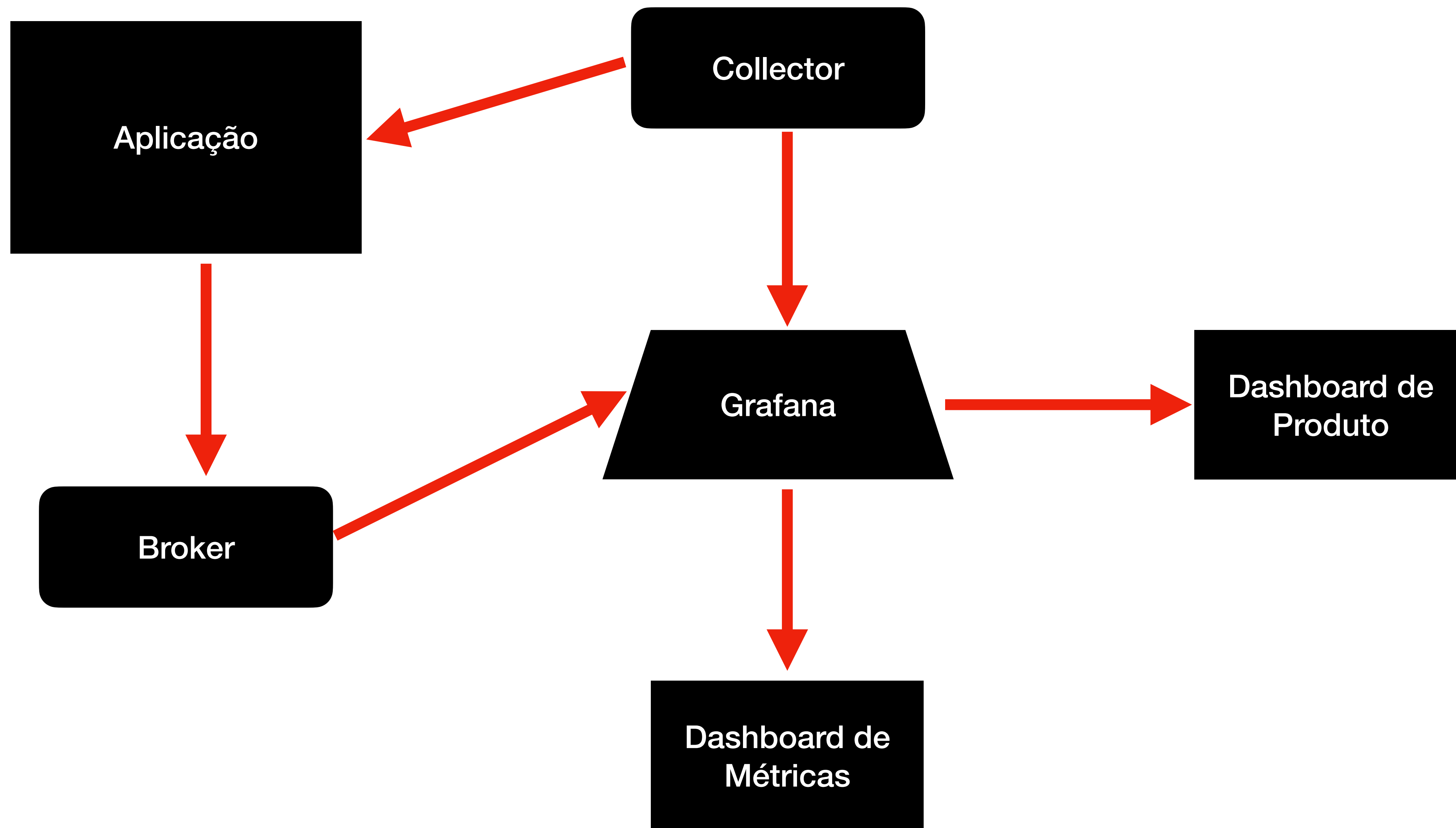
```
event: "payment_authorized"  
order_id: 555012  
user_id: 1029  
transaction_id: "tk_9182af1"  
value: 149.90  
payment_method: "credit_card"  
success: true
```


O log diz:

“A comunicação com o gateway funcionou e levou 214ms.”

O analytics diz:

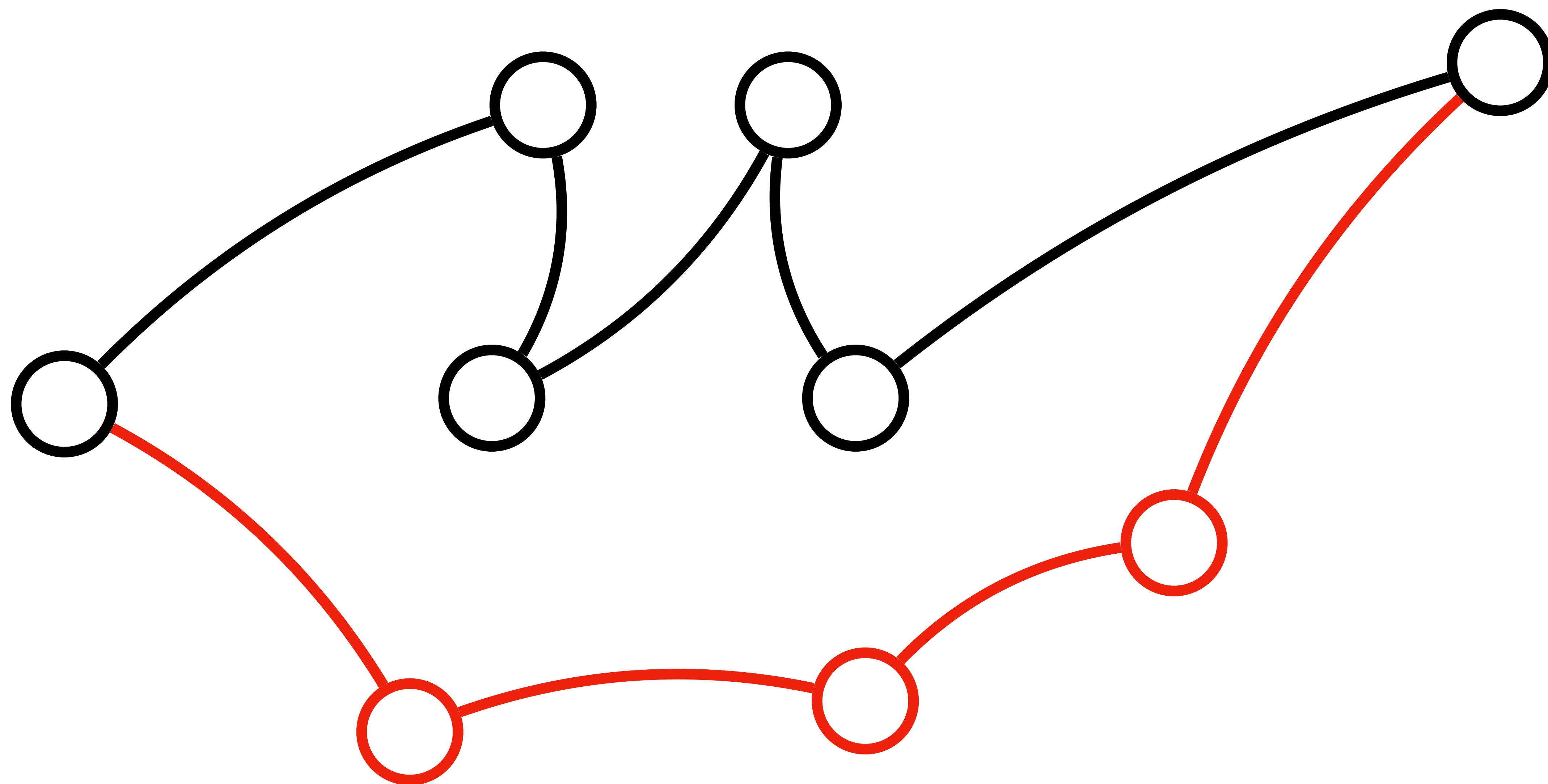
“O pagamento do pedido 555012 foi aprovado e gerou receita de R\$149,90.”

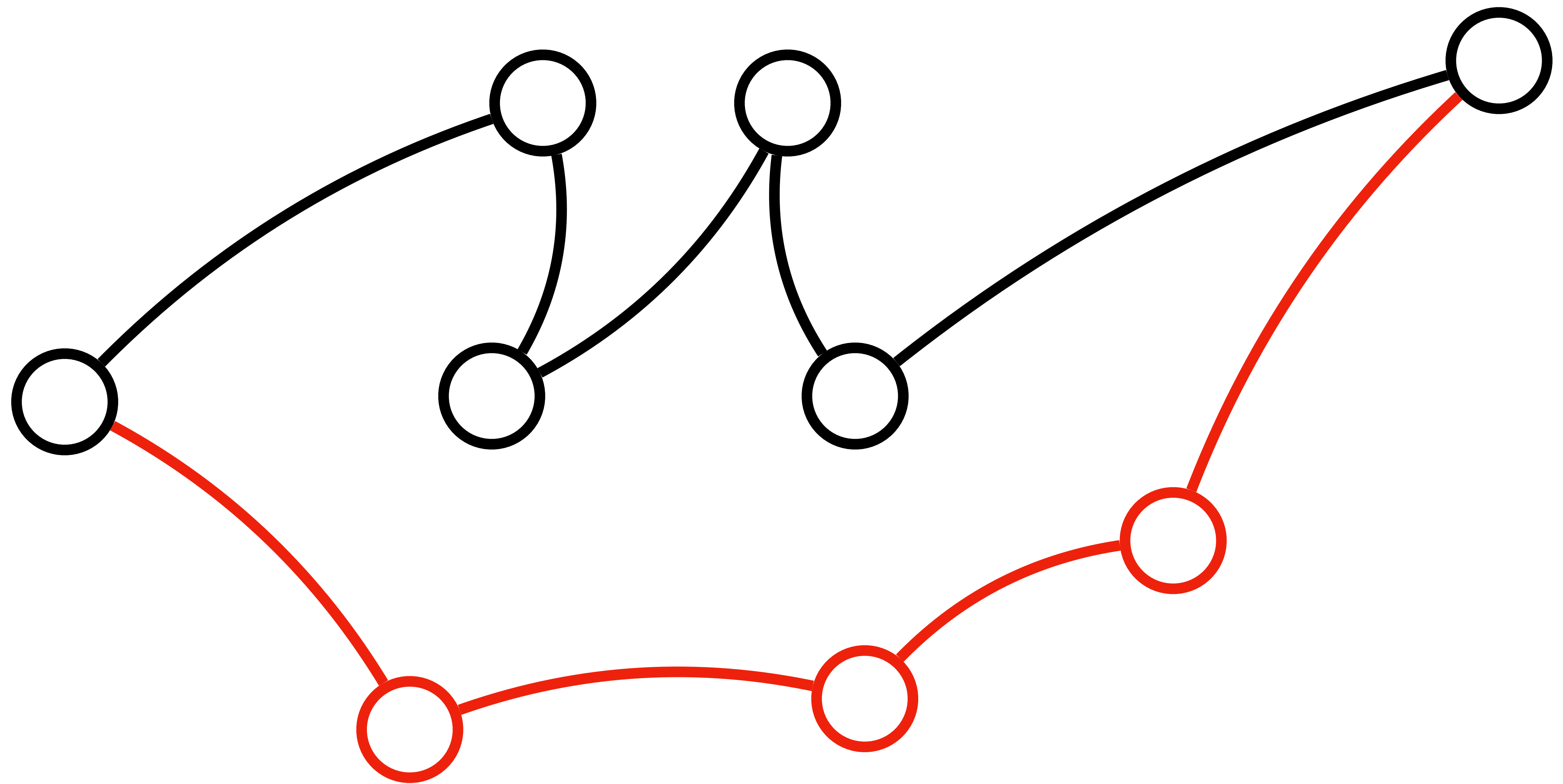


Plano de rollback

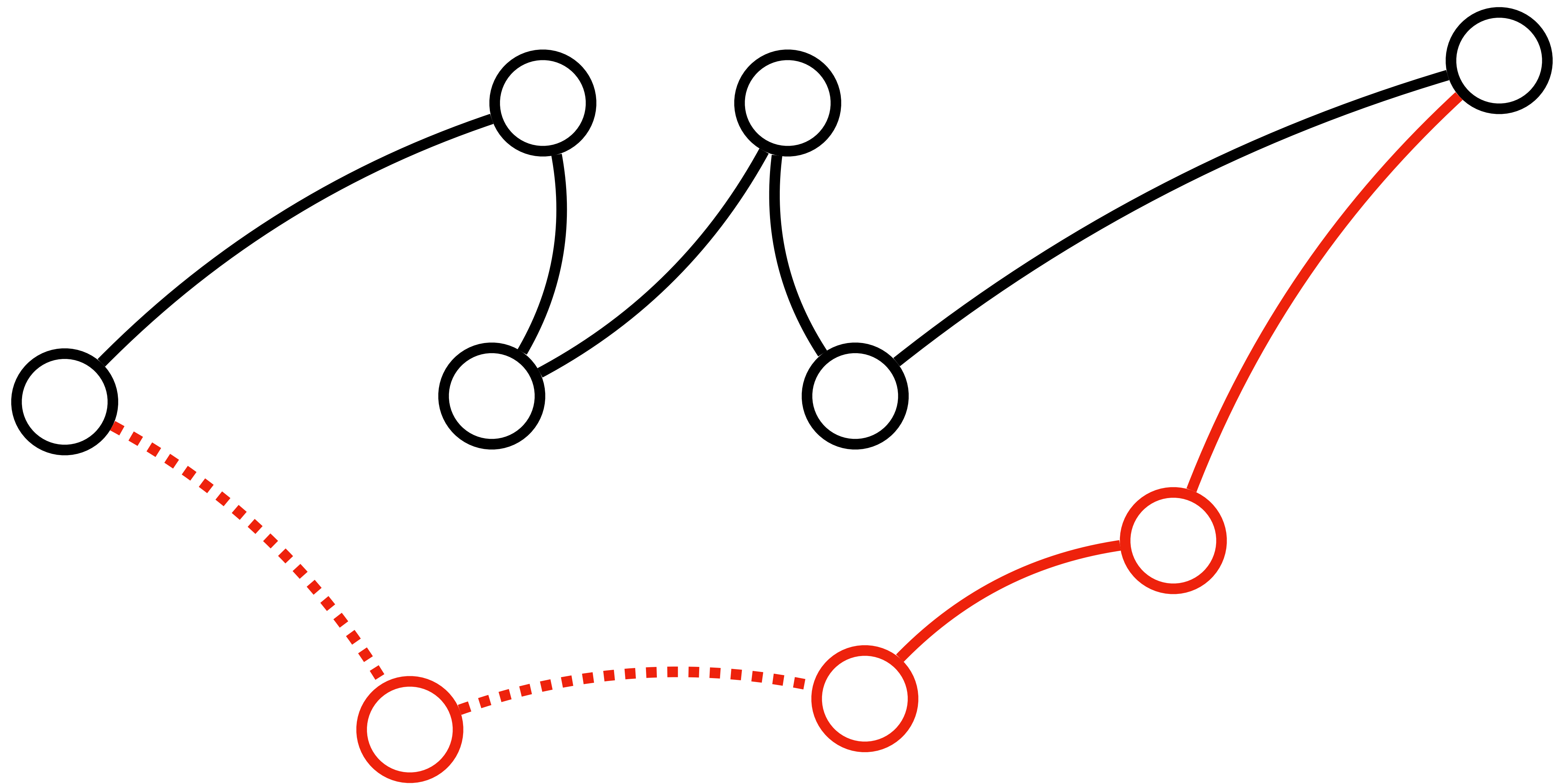
~~Plano de rollback~~

O que fazer caso de errado





Você precisa conseguir voltar ao estado inicial

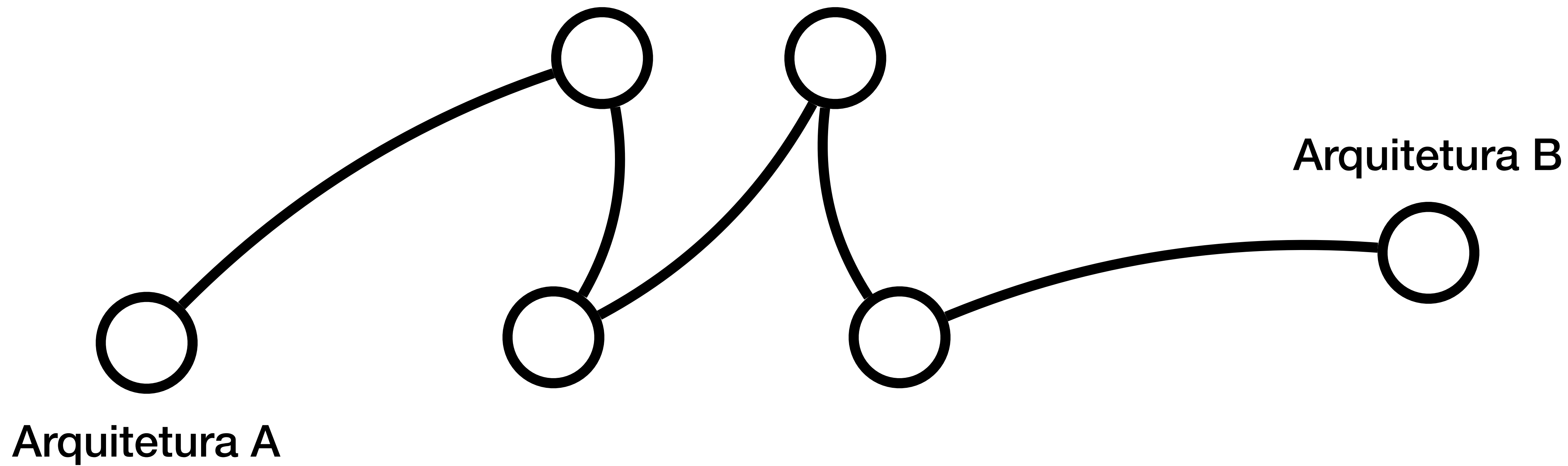


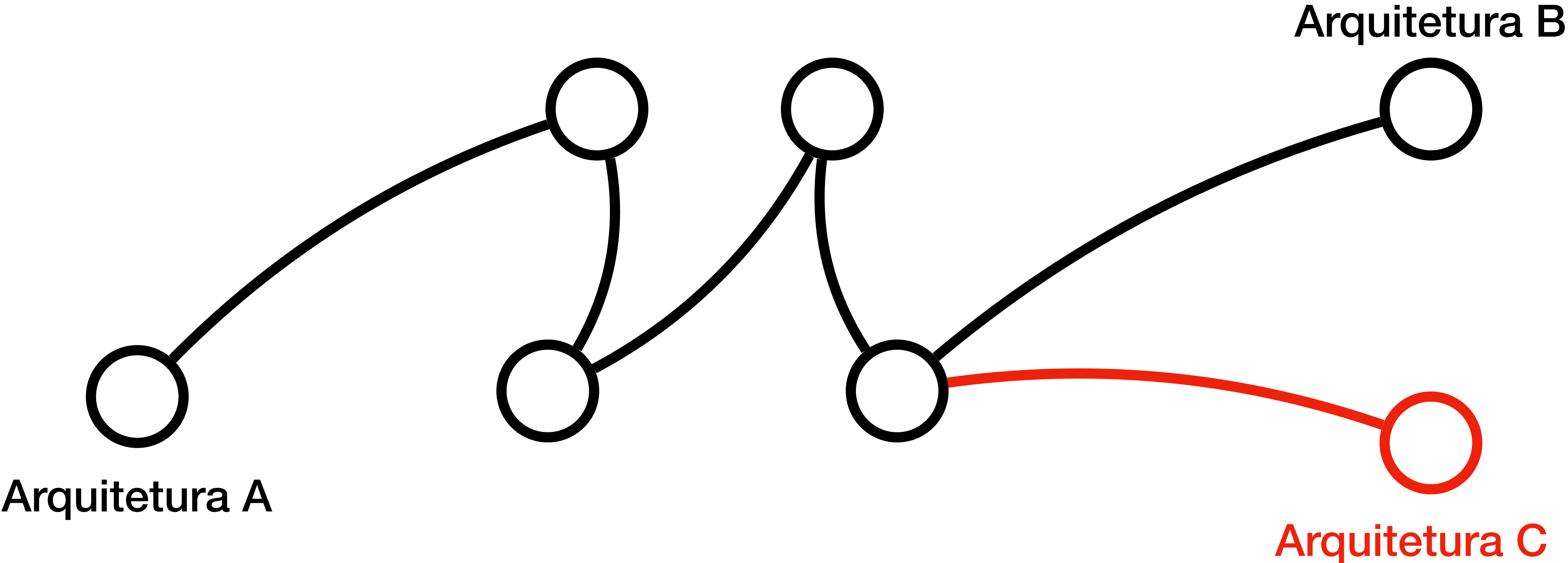
Uma migração sem rollback é aposta arriscada

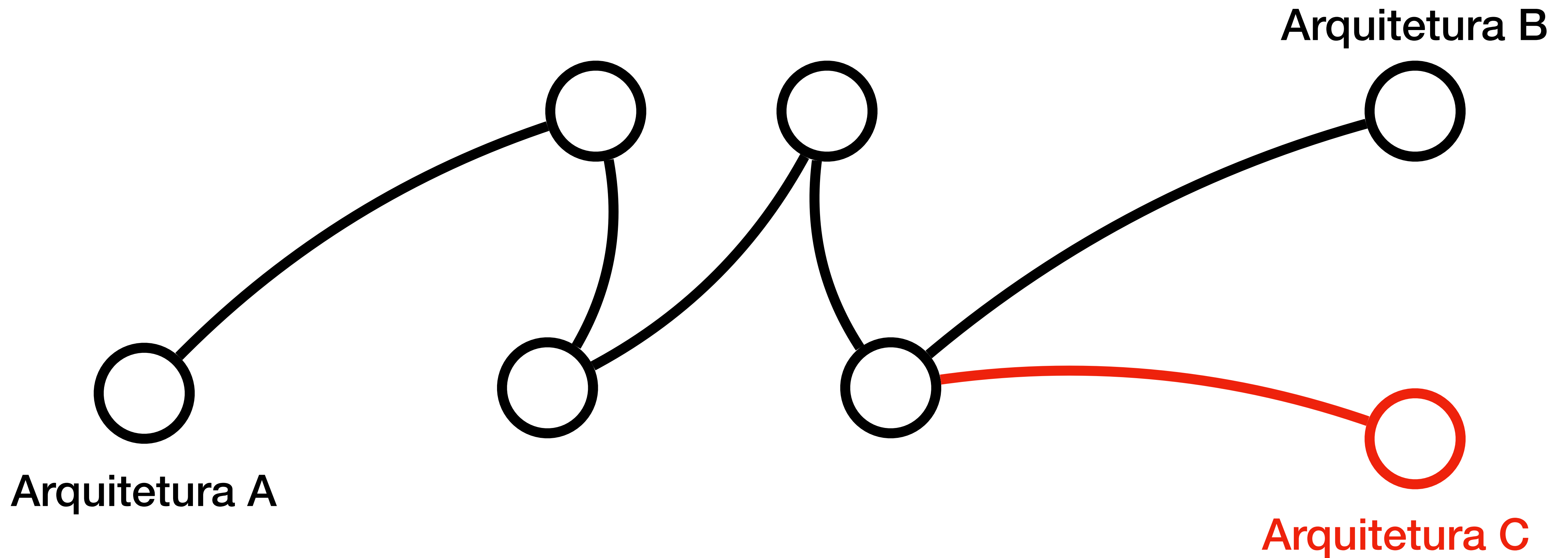
Como estar pronto
~~técnicamente~~ e emocionalmente
para migrar?

Emocionalmente

Saiba exatamente onde você quer chegar







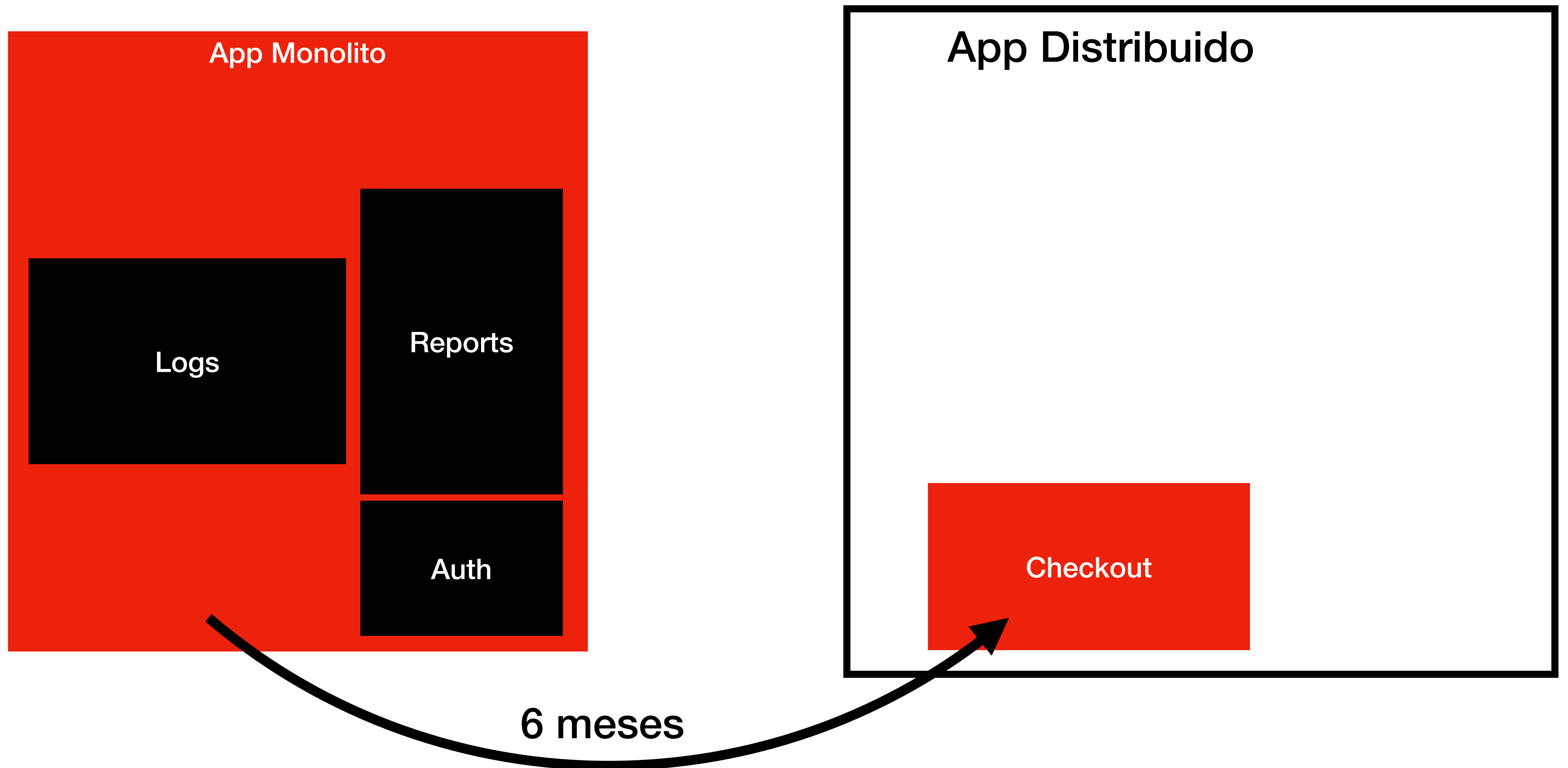
Isso aqui é exemplo de decisão **não planejada**

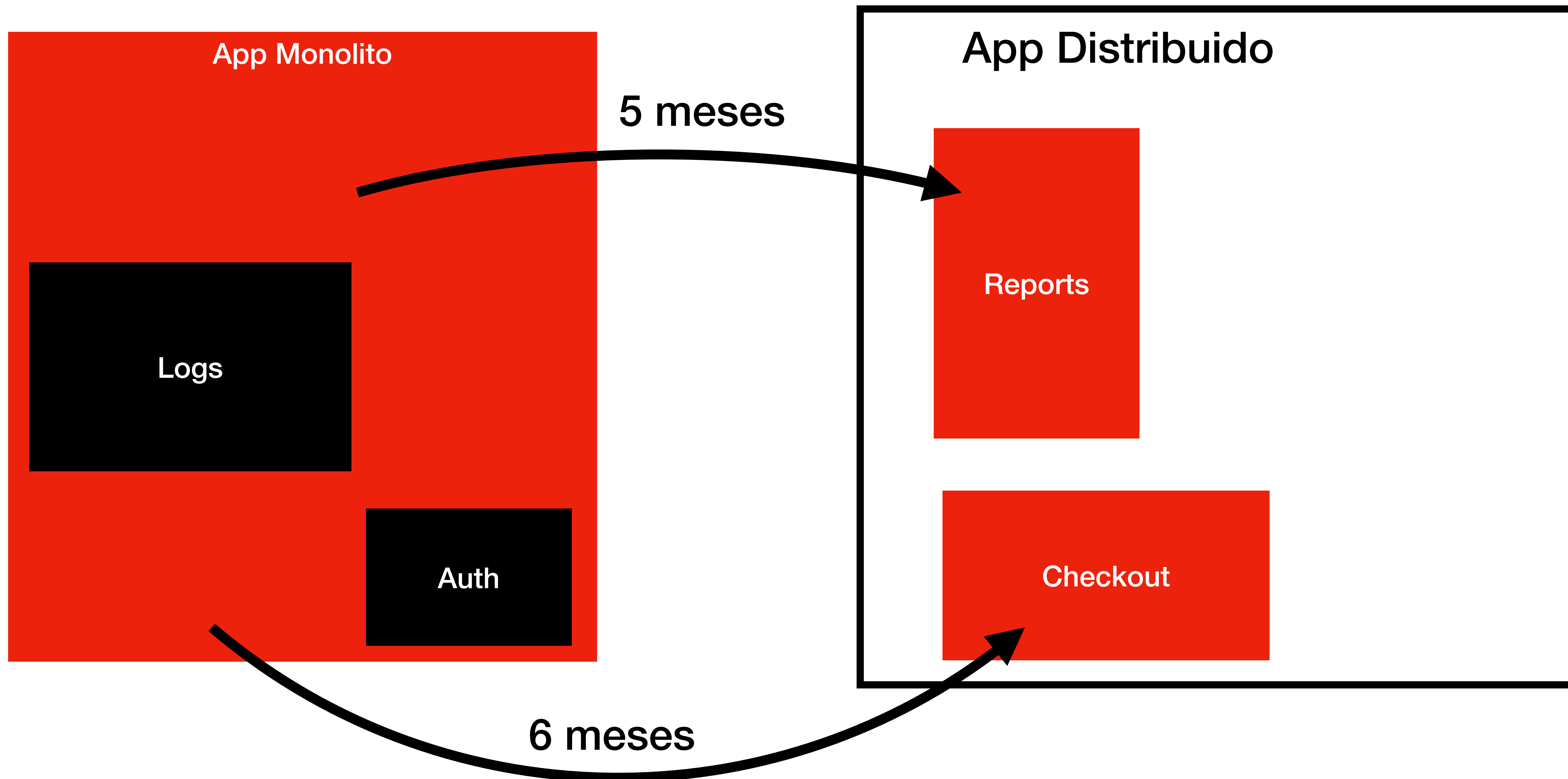
Relembre sempre os motivos da migração

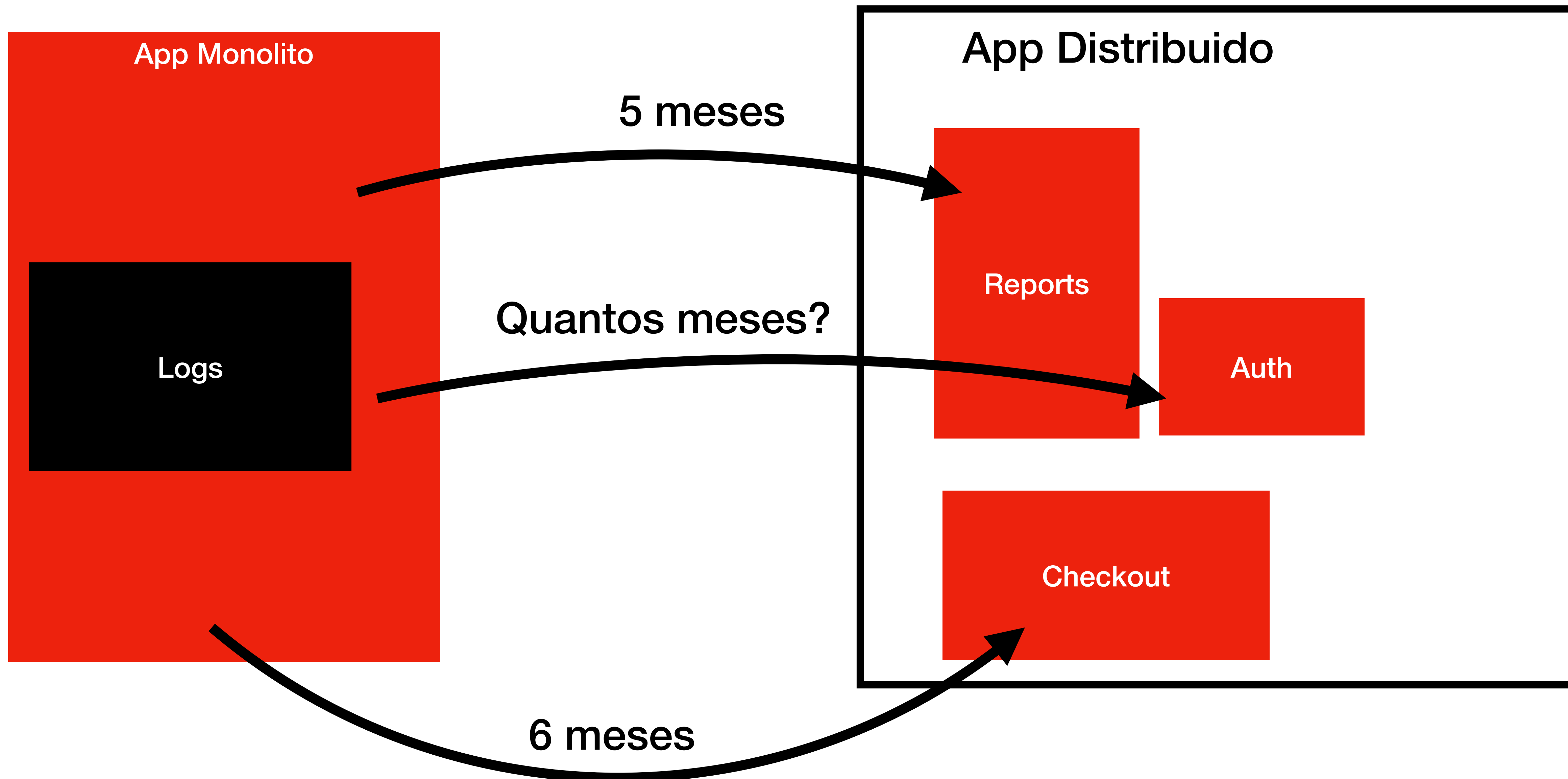
Problemas para escalar
Dificuldades com manutenção
Decisões visando o projeto
Resolver travas técnicas

Paciência!!!

Quando as coisas começam a dar certo
tudo vira urgente







**Se os dois primeiros módulos
demoraram 5 e 6 meses**

Se os dois primeiros módulos
demoraram 5 e 6 meses

Esse também vai levar essa média...

Se sua gestão está pensando assim
volte duas casas

Se medir prazo de uma task já um mistério
imagina de uma migração

Alinhe sempre as expectativas

Bugs vão acontecer!

Features vão demorar mais

As coisas vão ficar instáveis

Ignorar isso só vai desgastar o seu time

Esses são os trade-offs

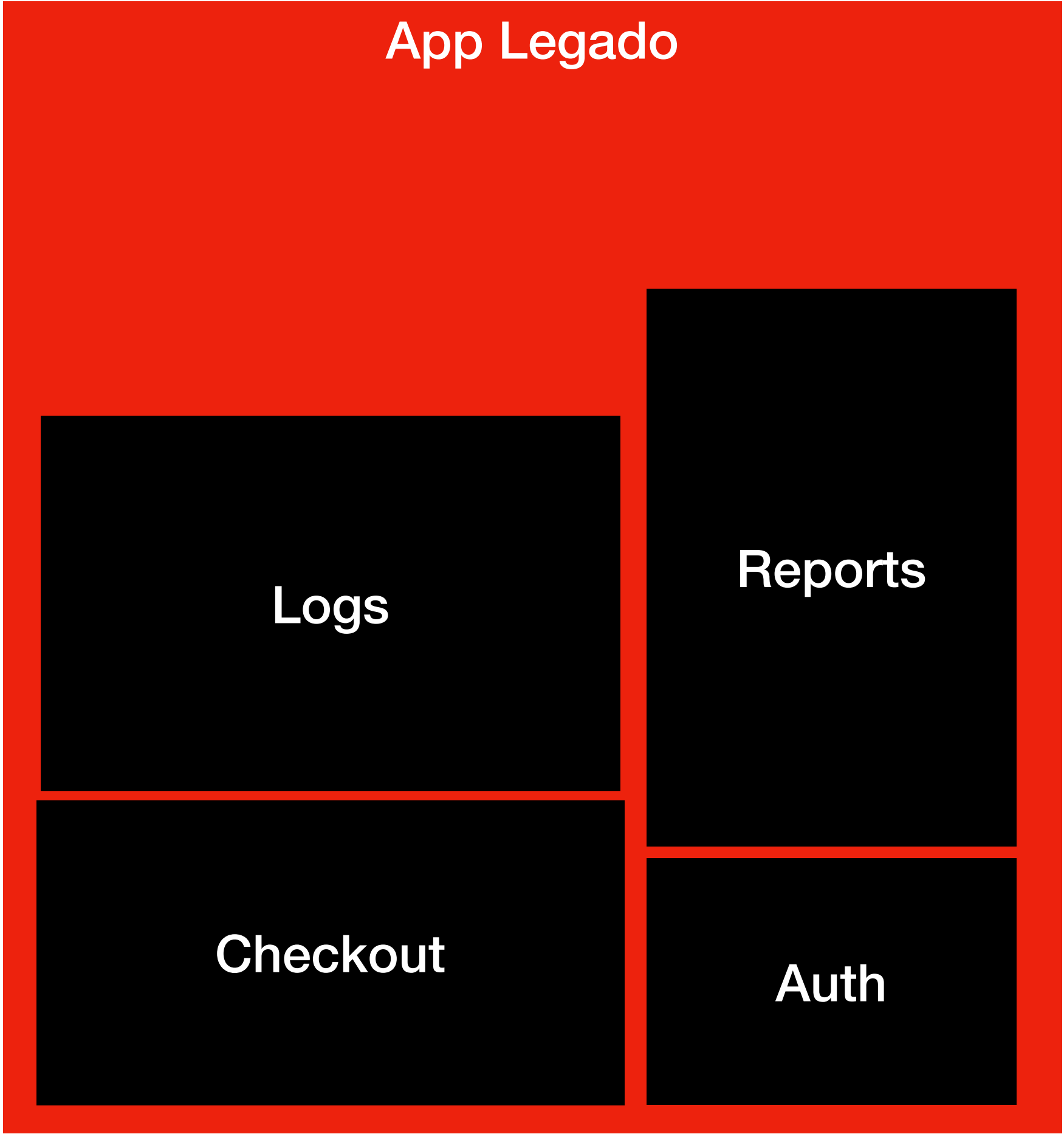
Técnicas de migração

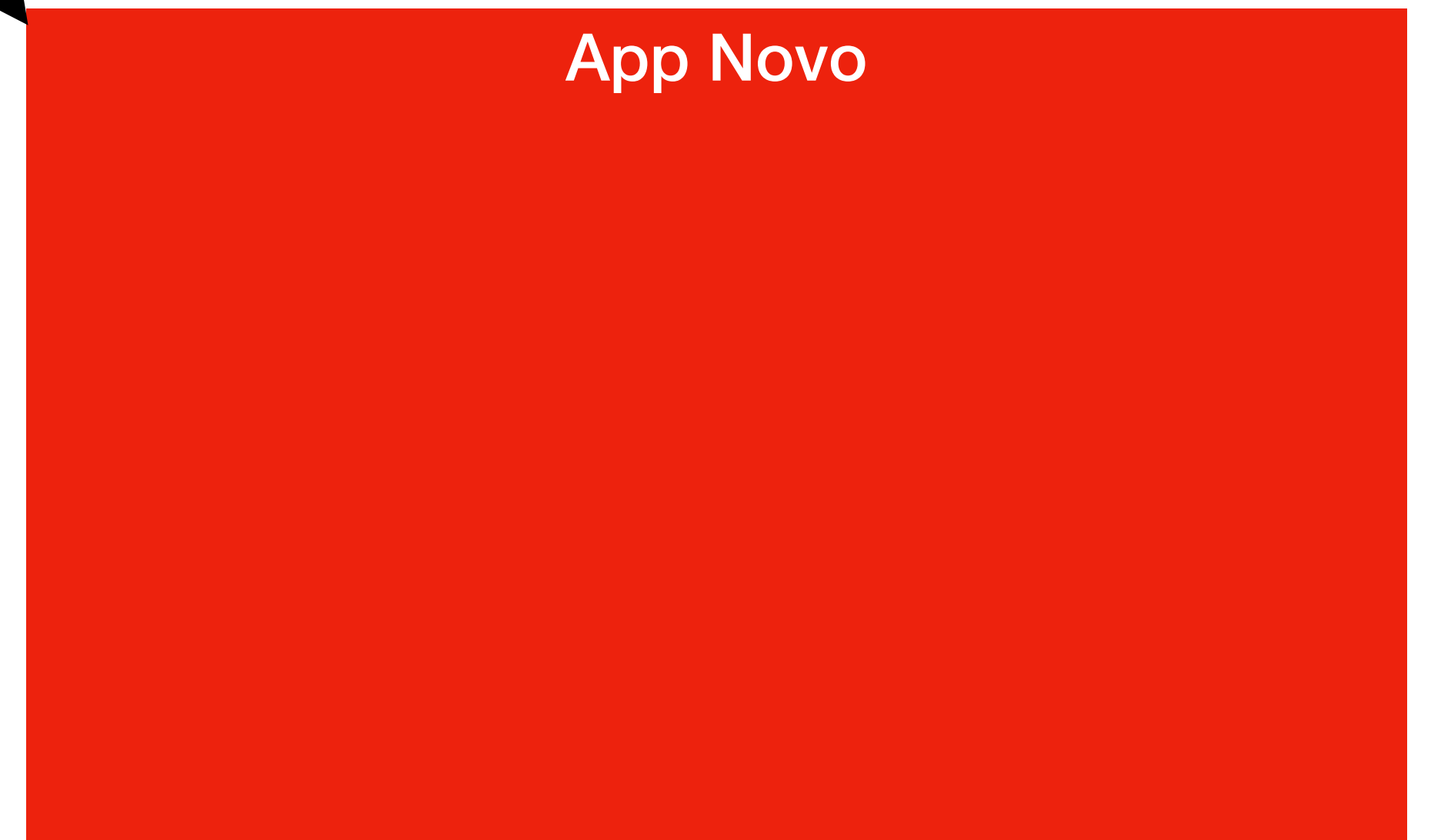
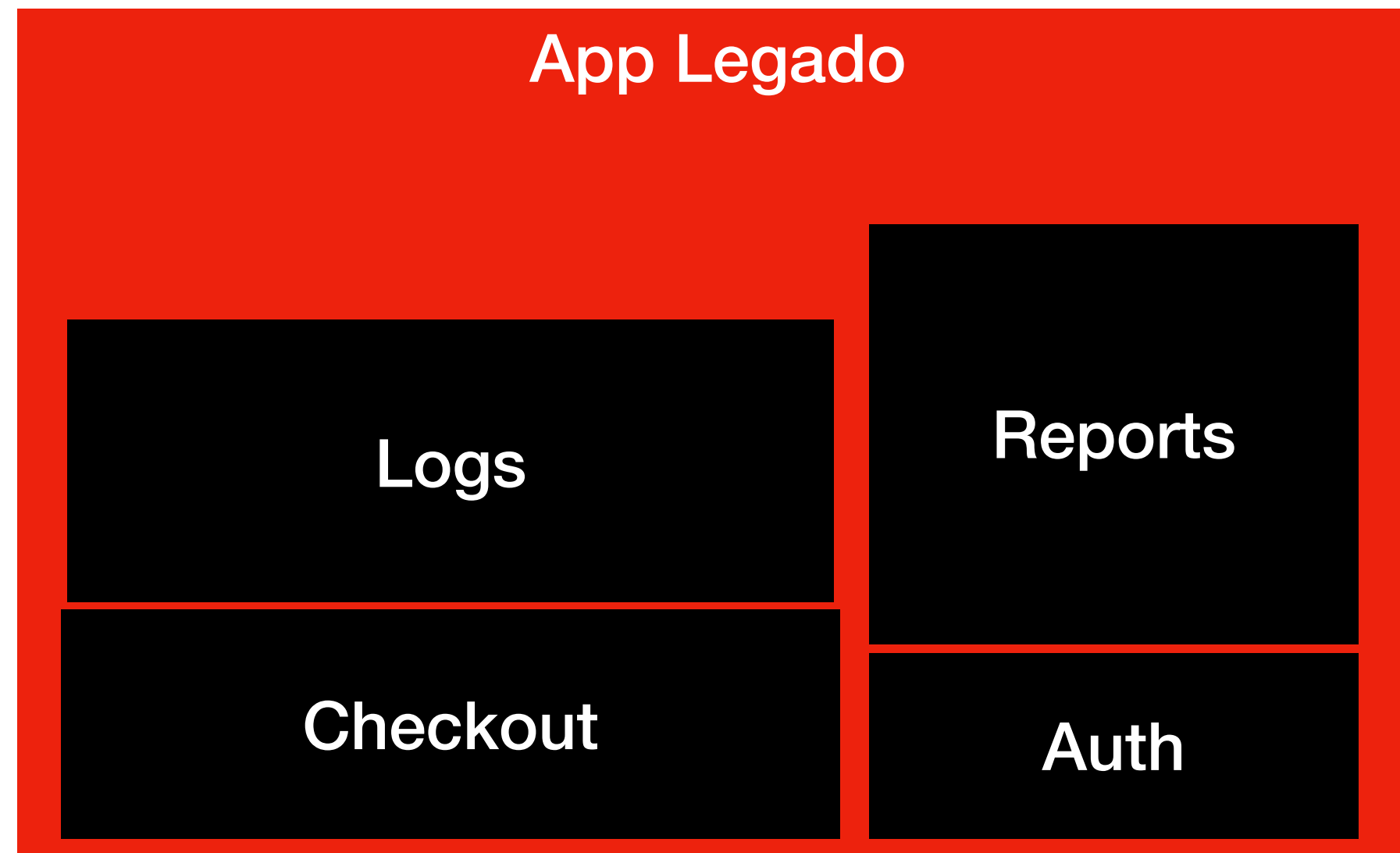
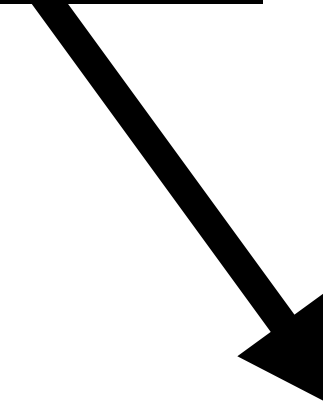
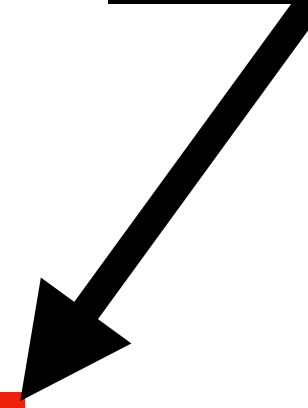
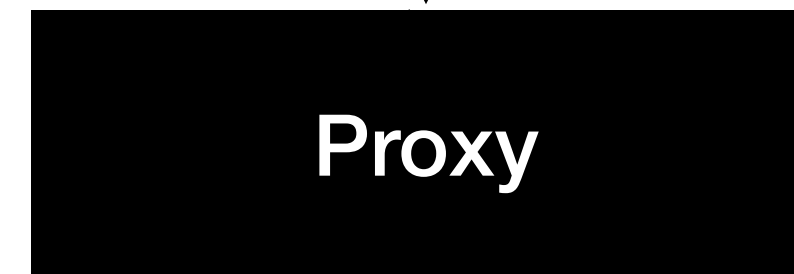
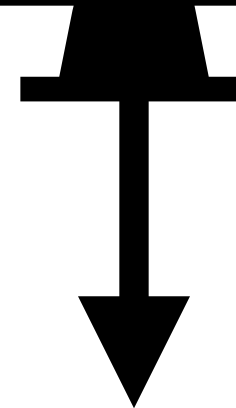
Strangler Pattern

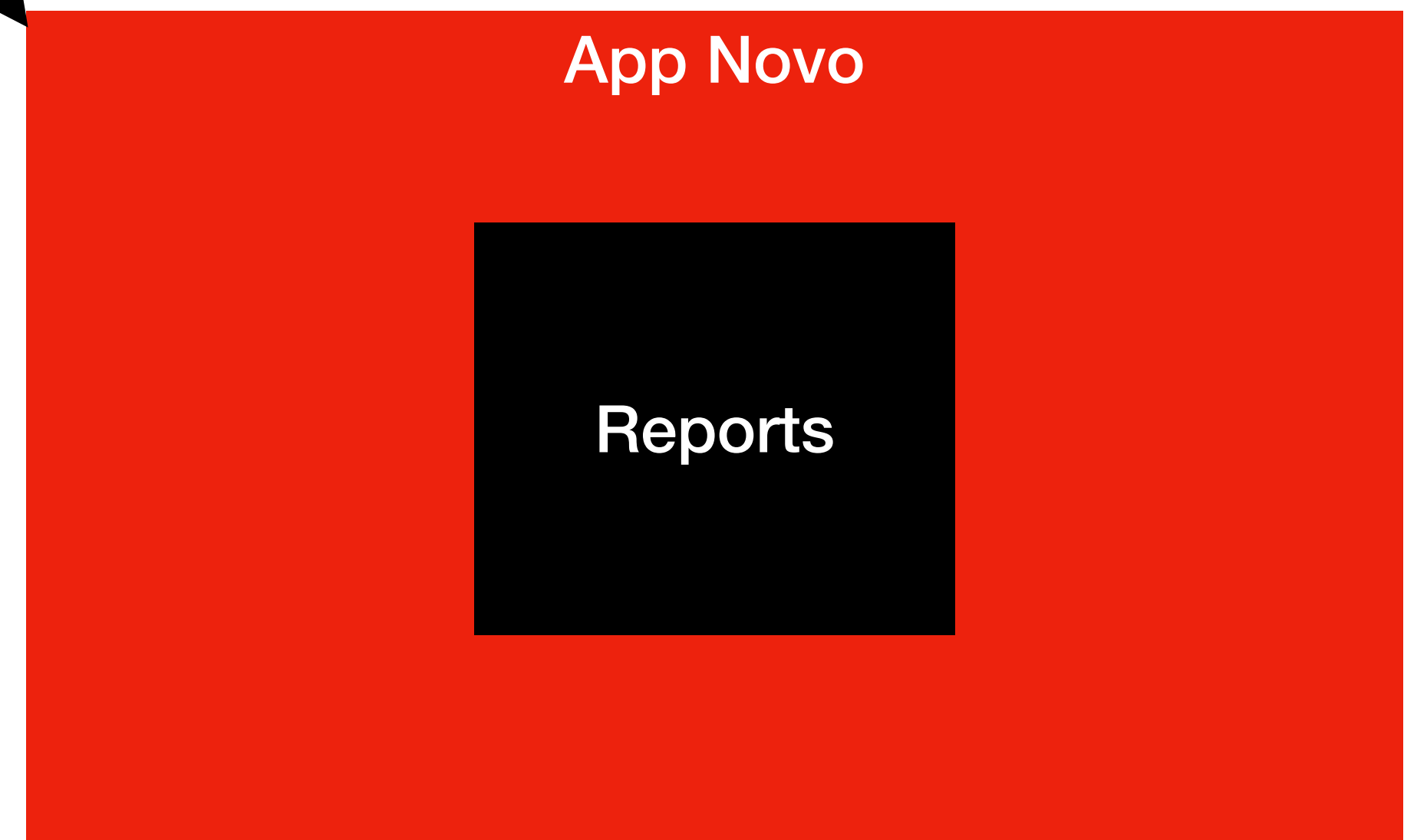
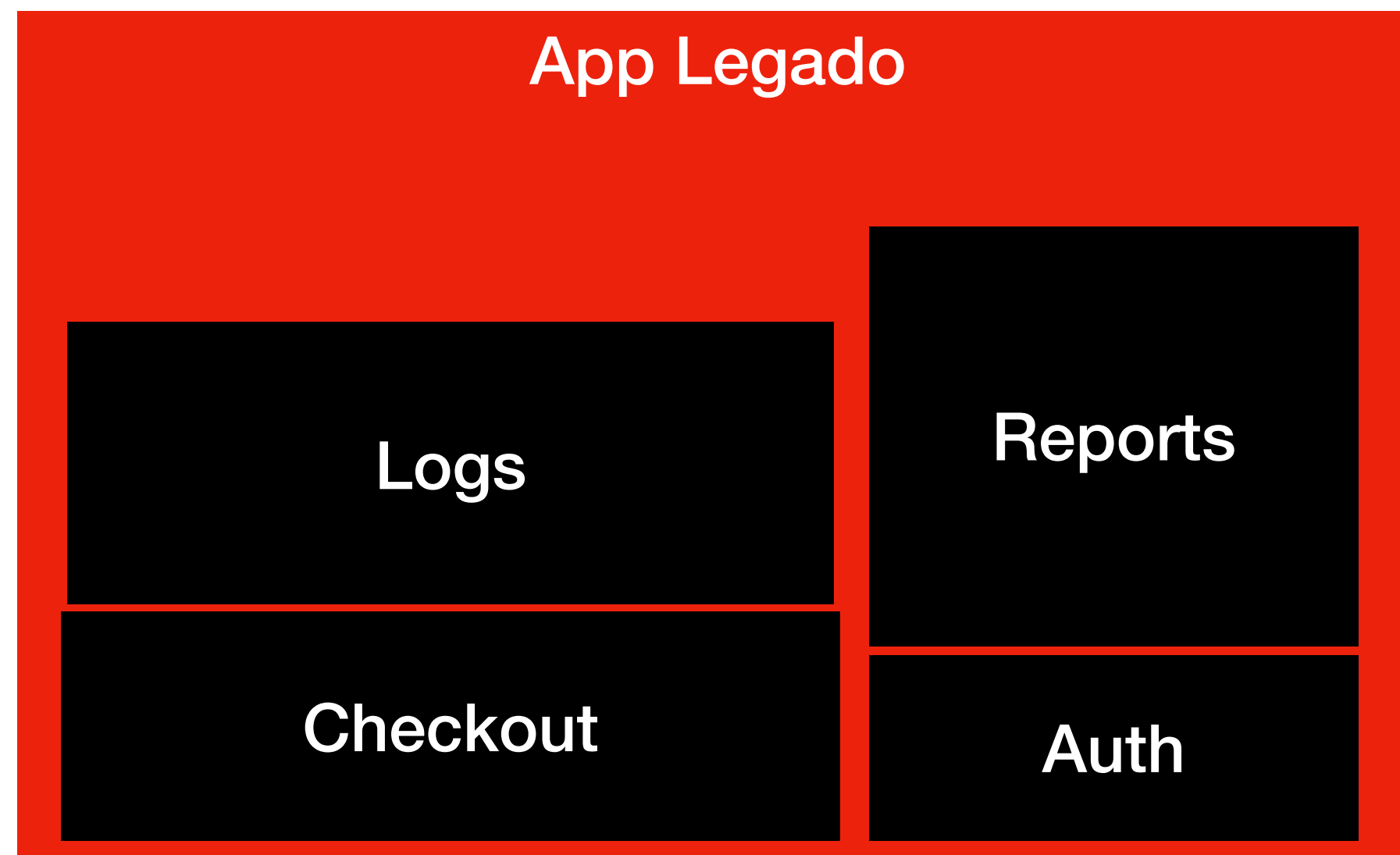
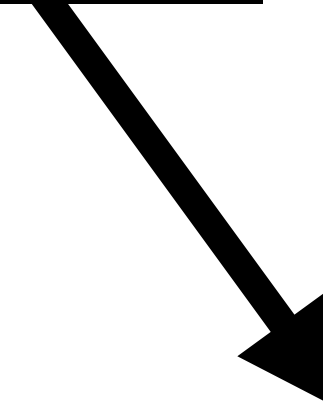
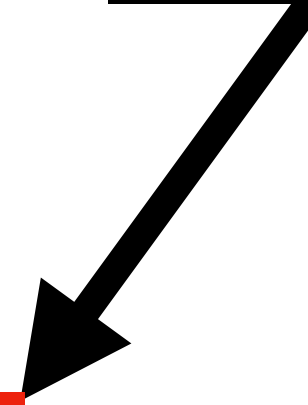
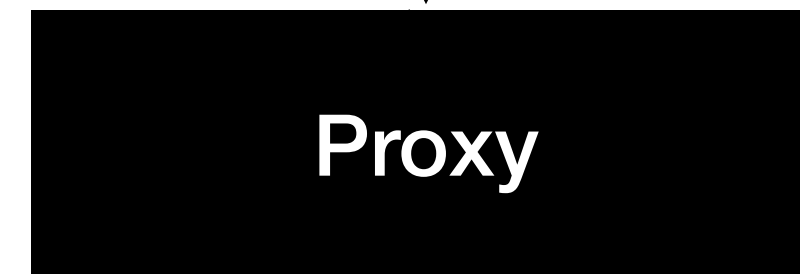
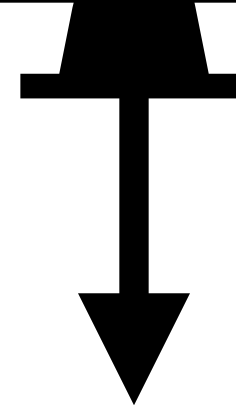
Padrão do Estrangulamento

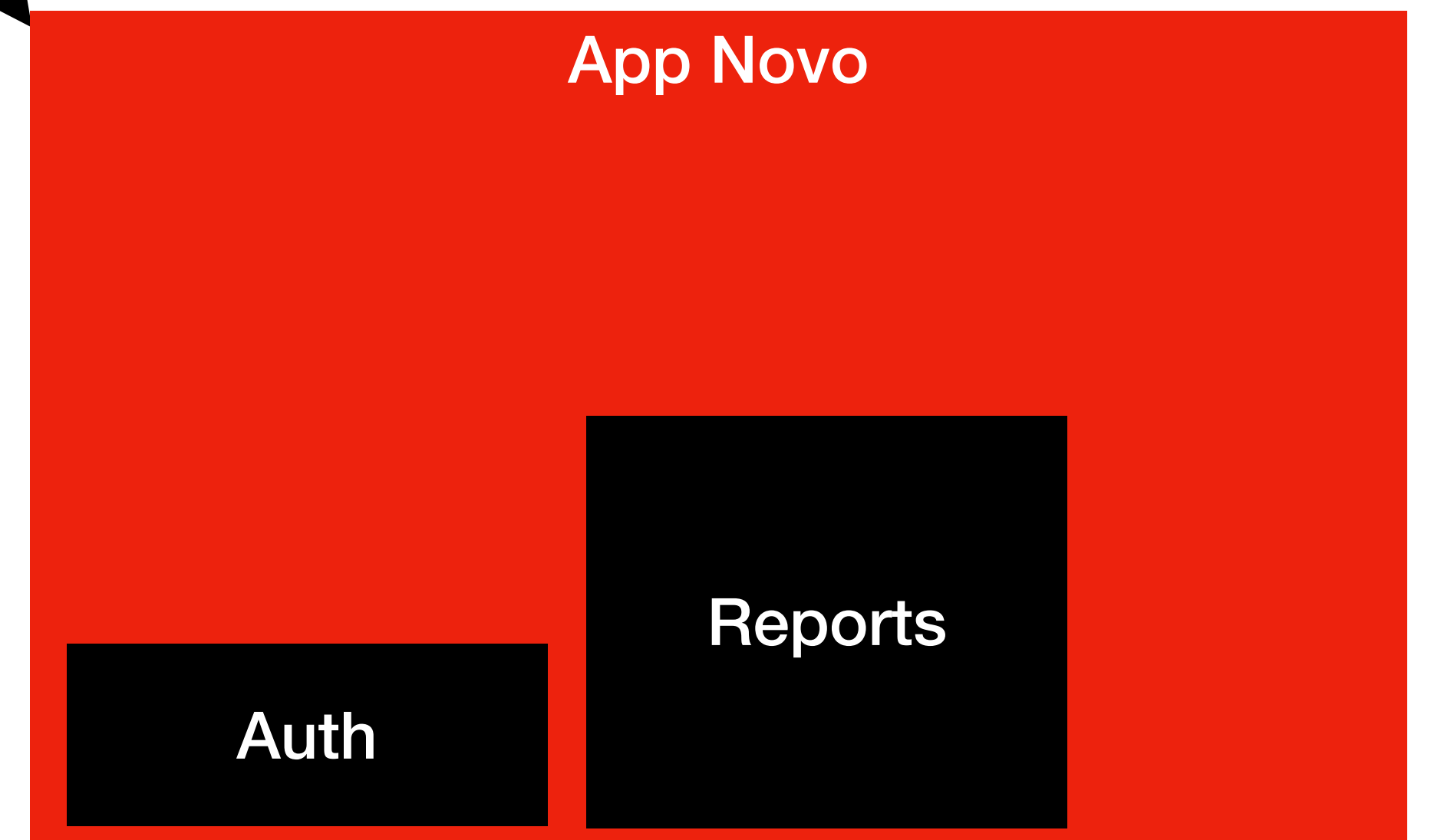
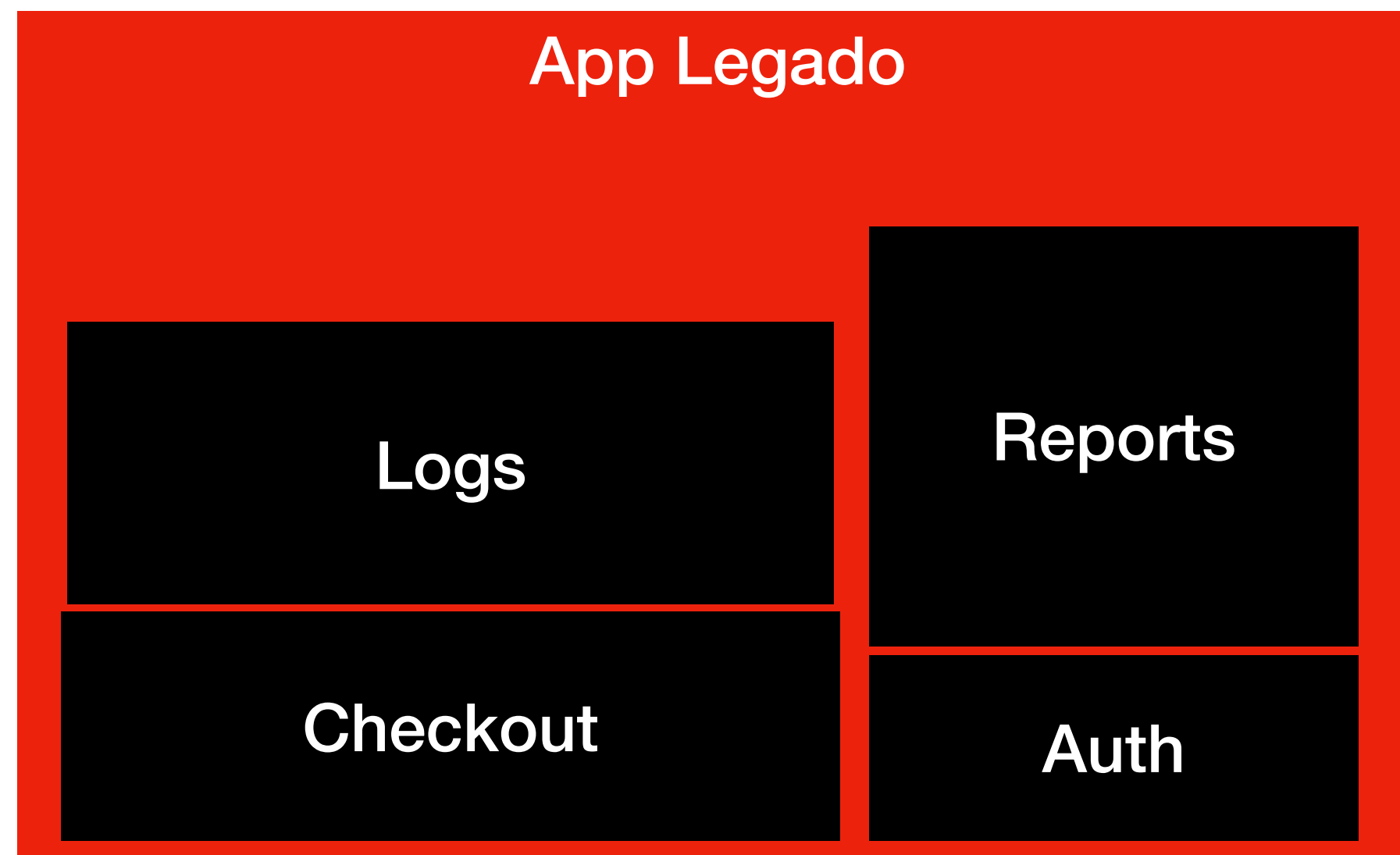
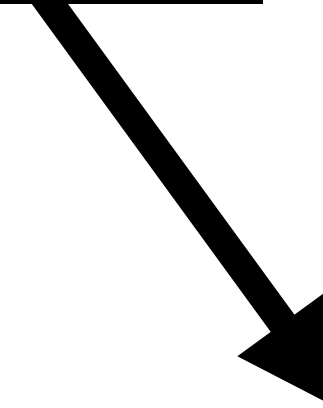
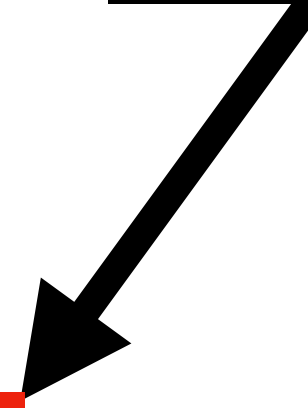
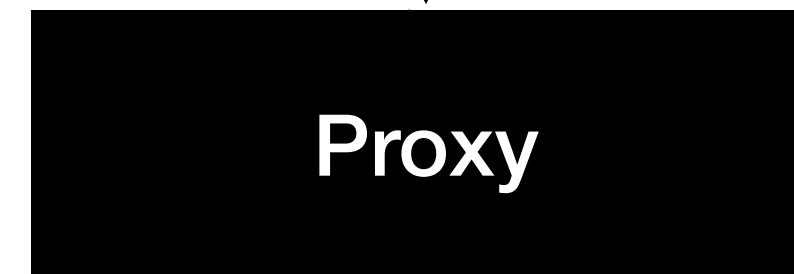
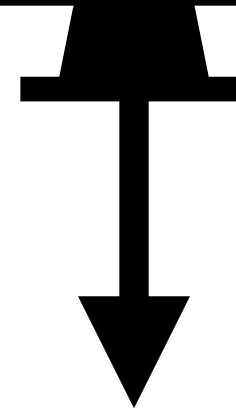
O **Strangler Pattern** é uma
estratégia de **migração incremental**

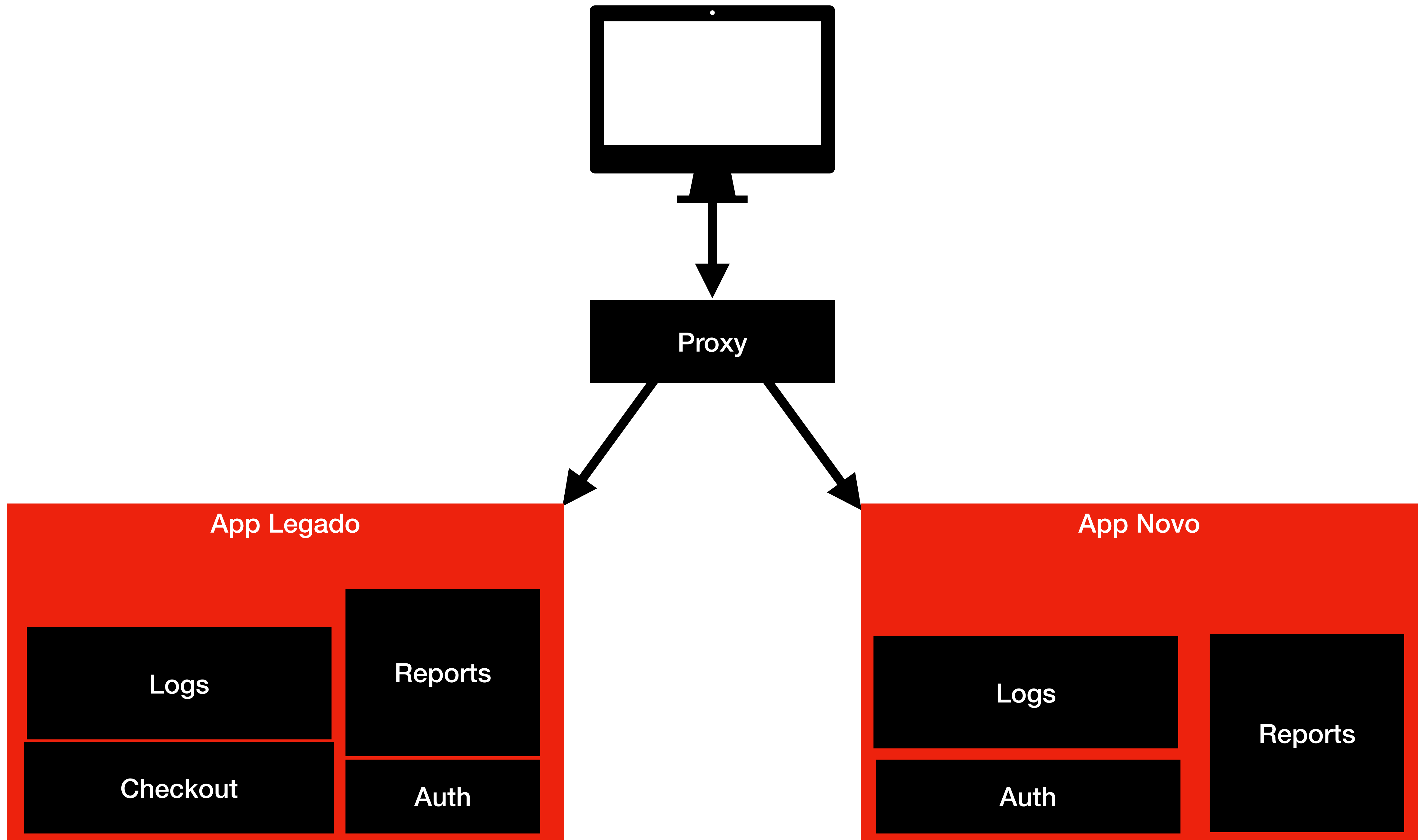
**Isso significa que a migração vai
ocorrendo aos poucos**

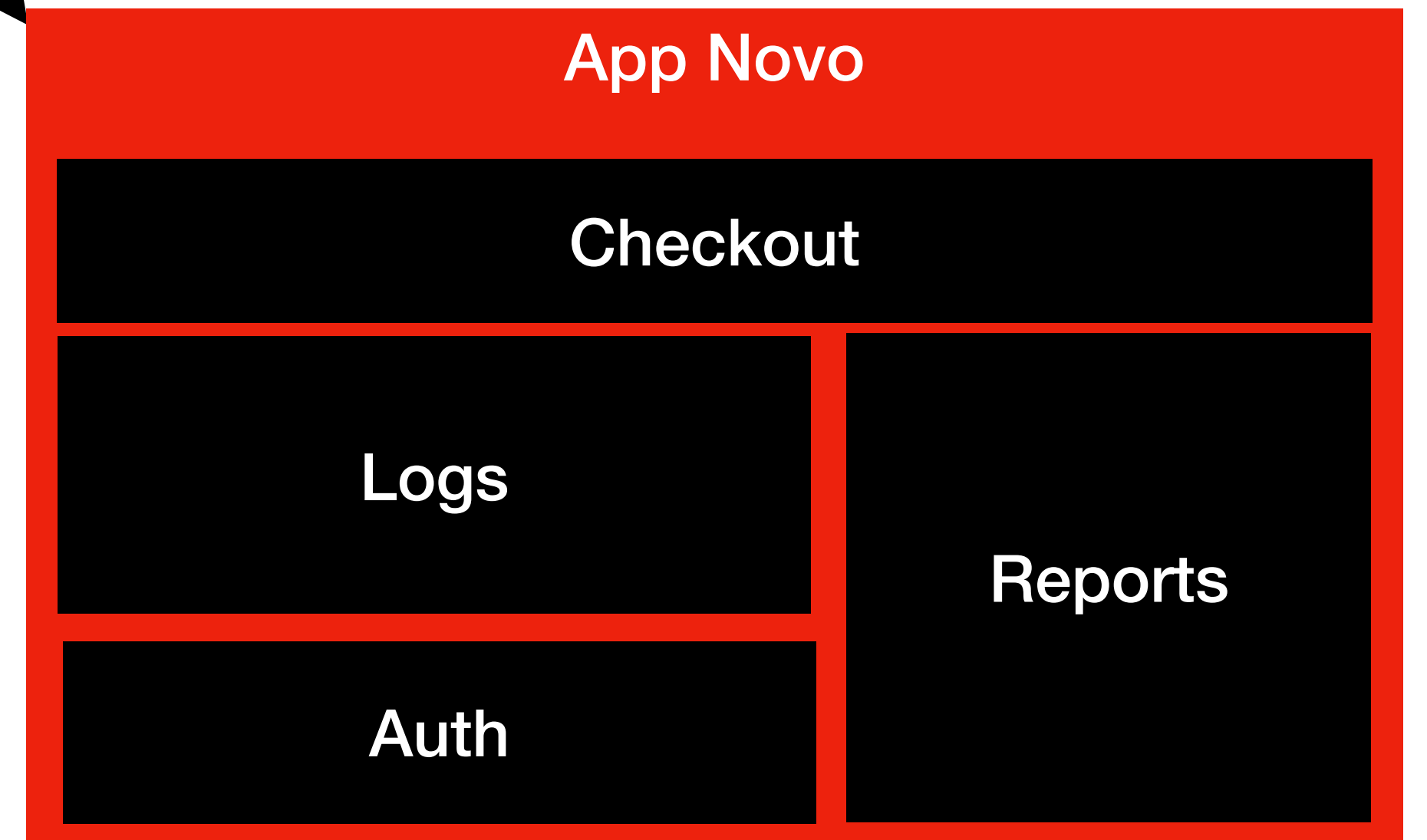
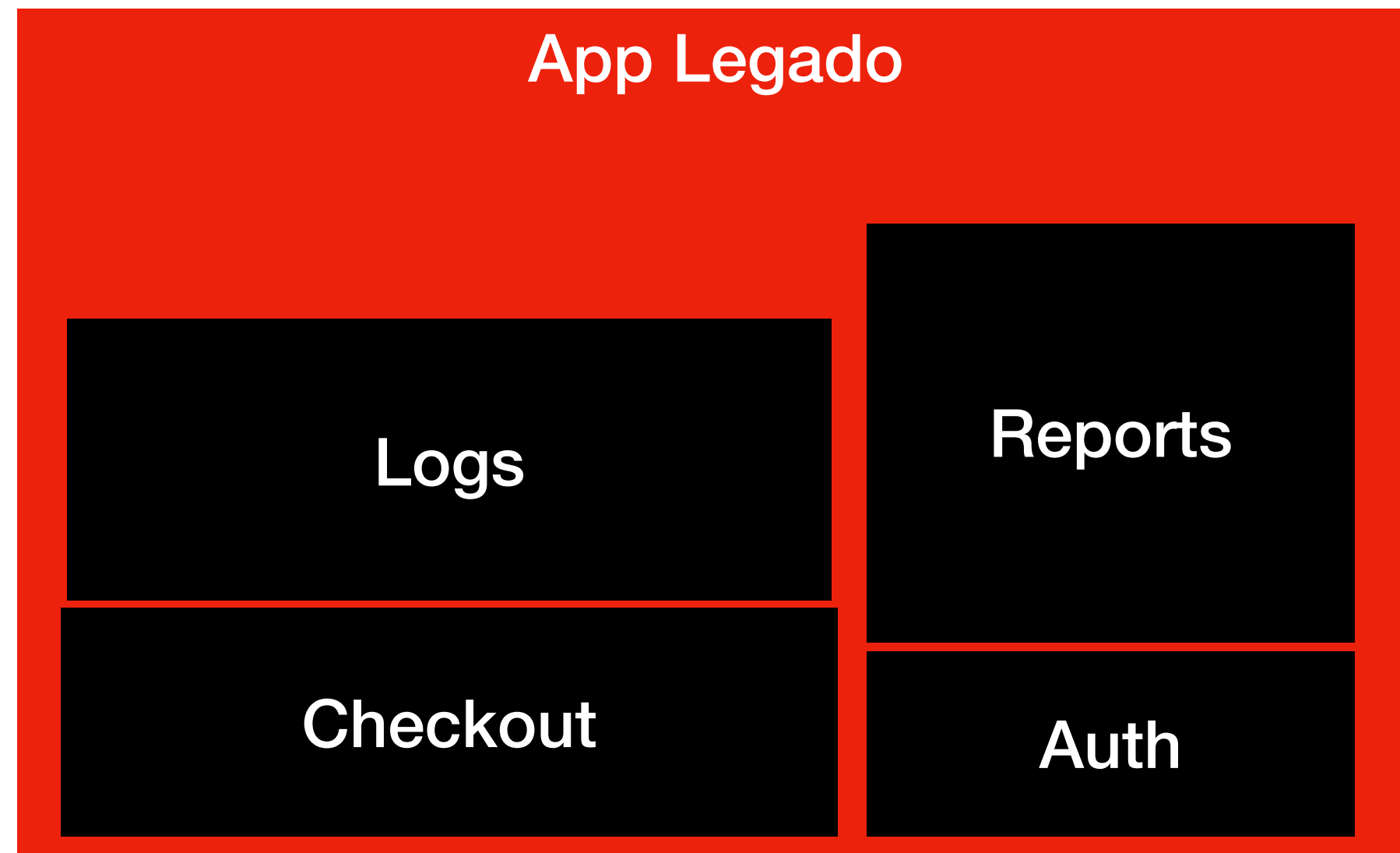
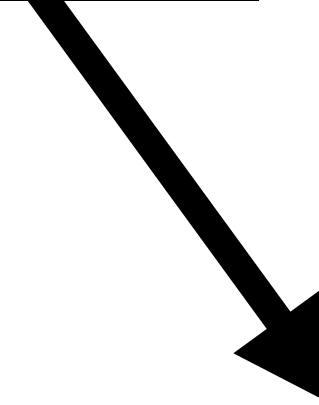
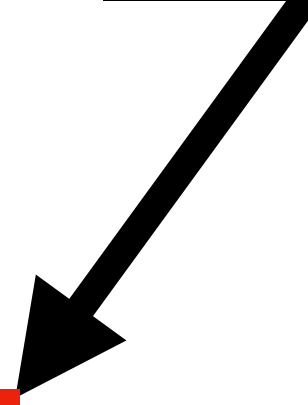
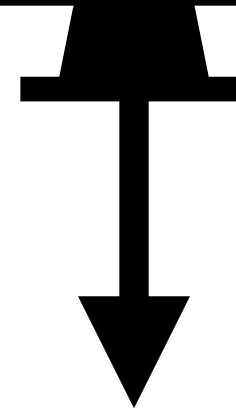


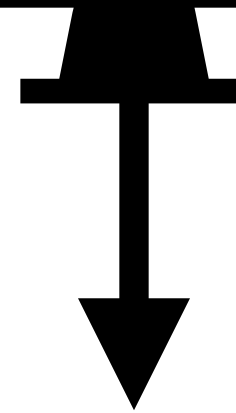




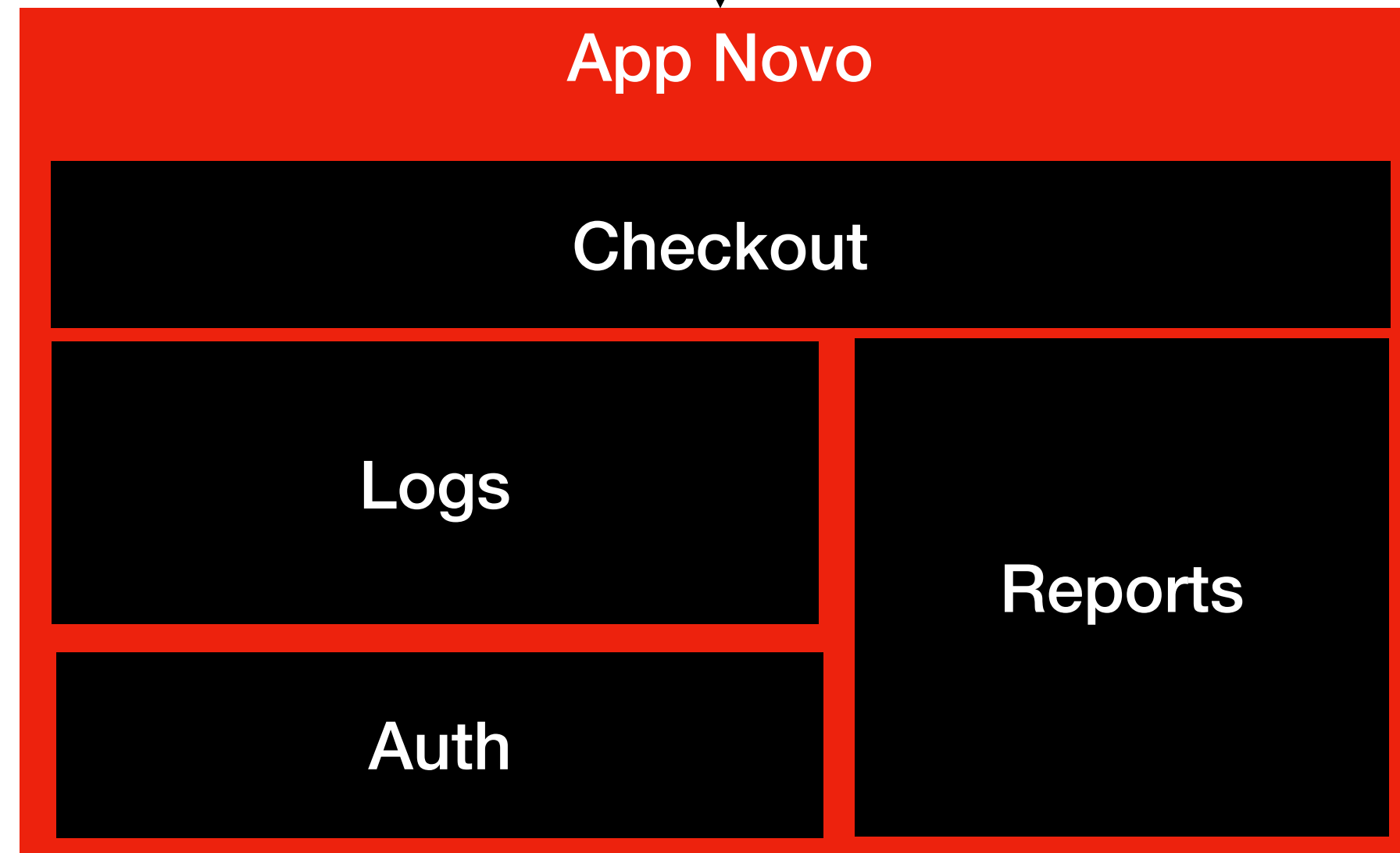


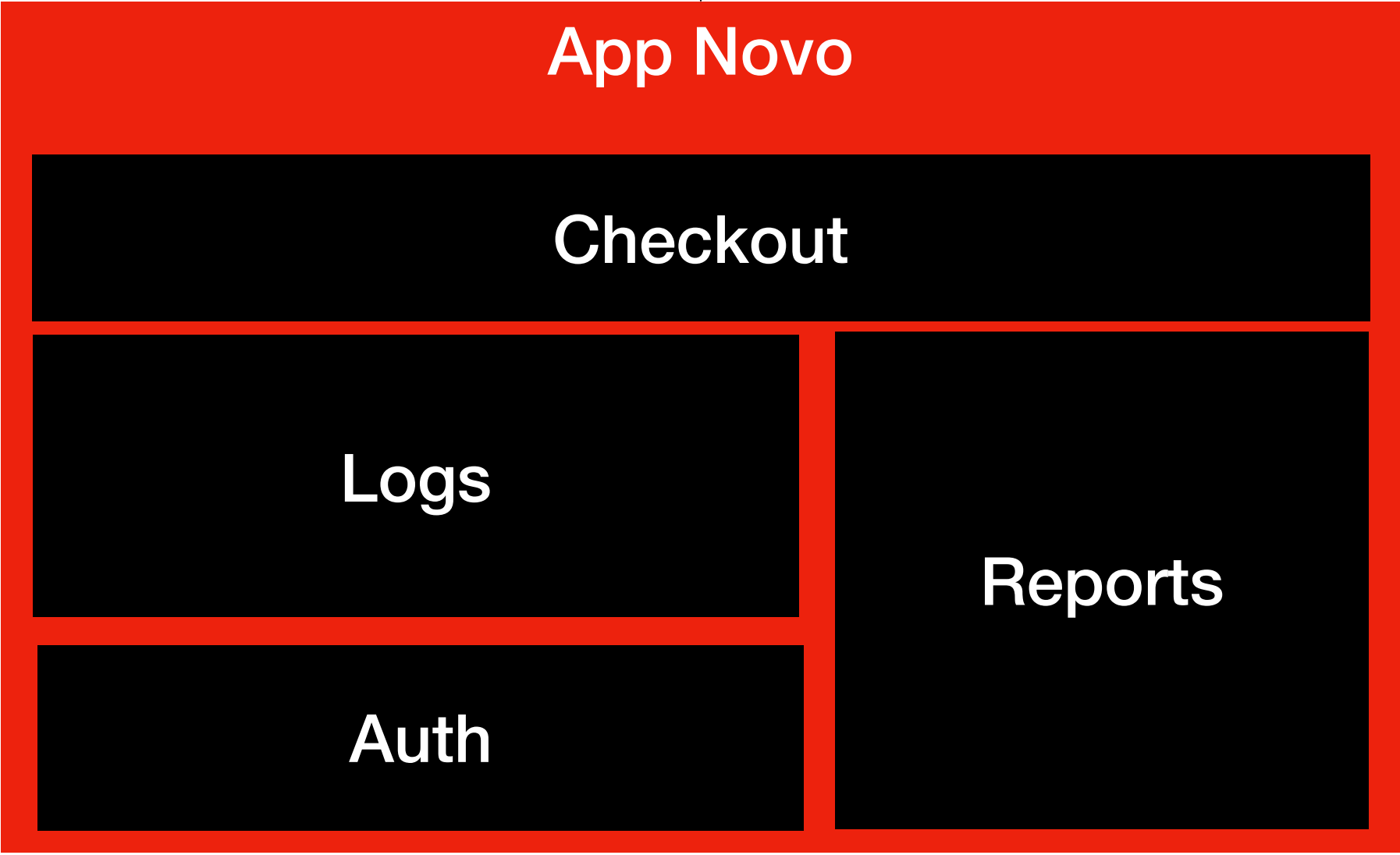
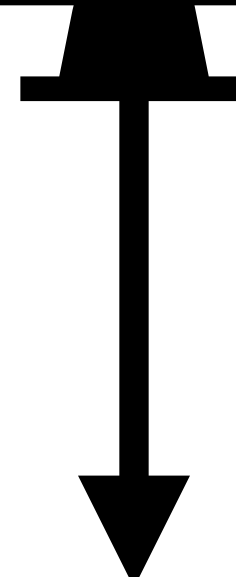






Proxy





Pontos de Atenção

Pode levar meses ou anos

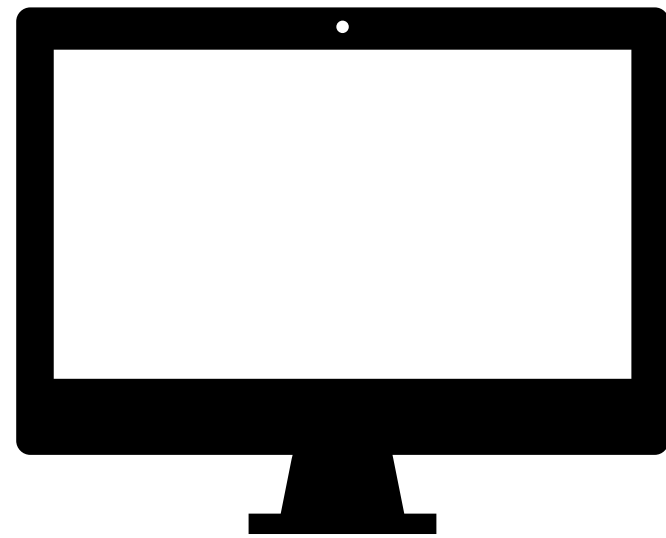
É comum criar “**dobra**” de lógica

O **proxy** precisa estar muito bem projetado

**Se você migrar sem estratégia
vira um sistema Frankenstein**

Mas o meu legado é impossível de mexer!!!

Black Box Pattern

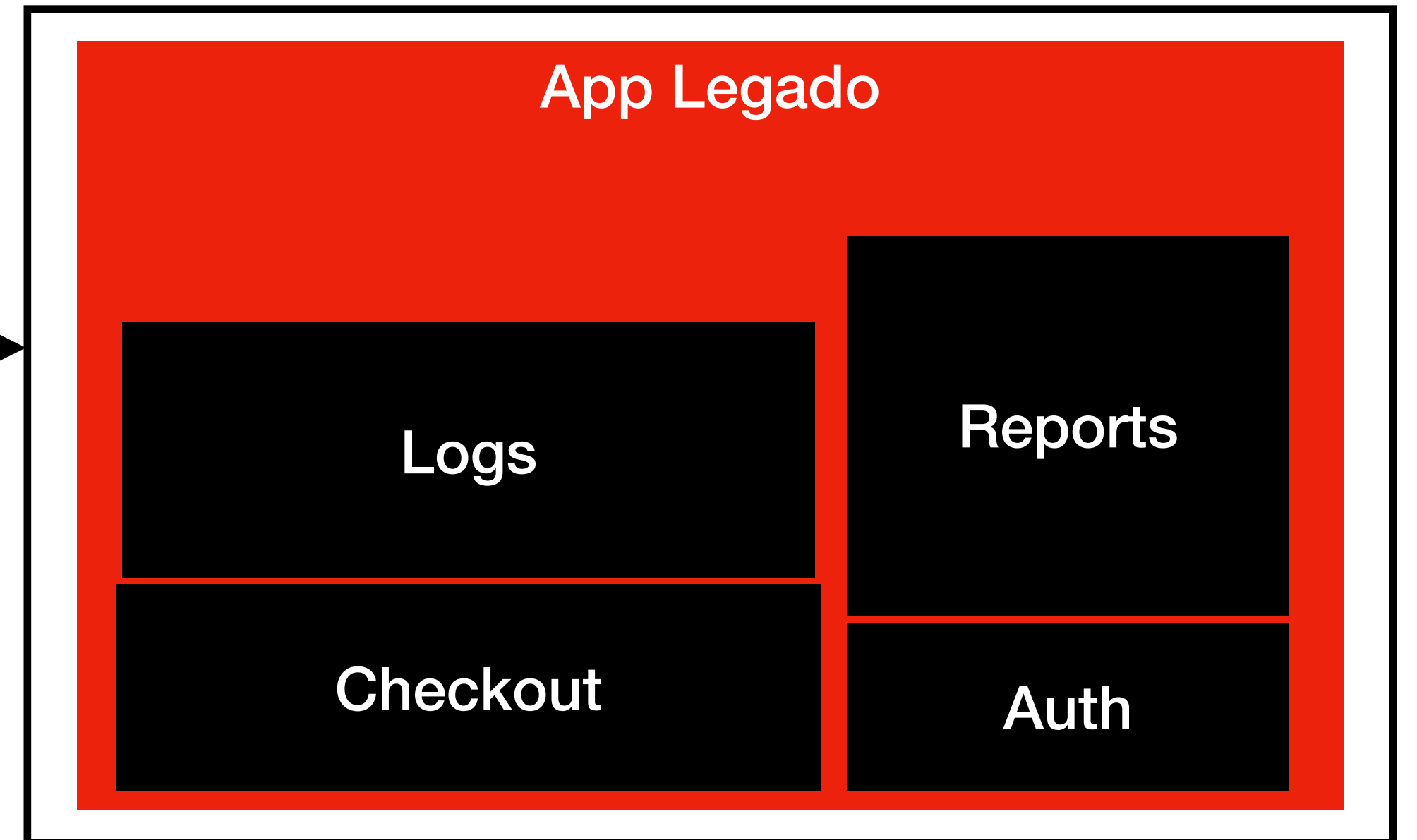


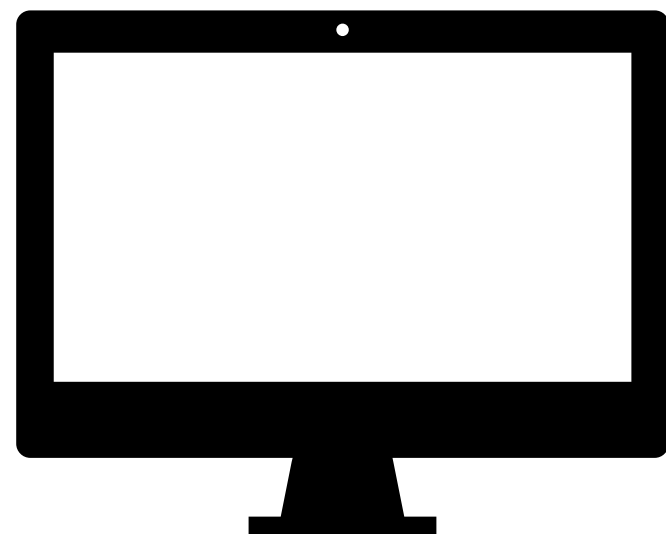
/logs

/checkout

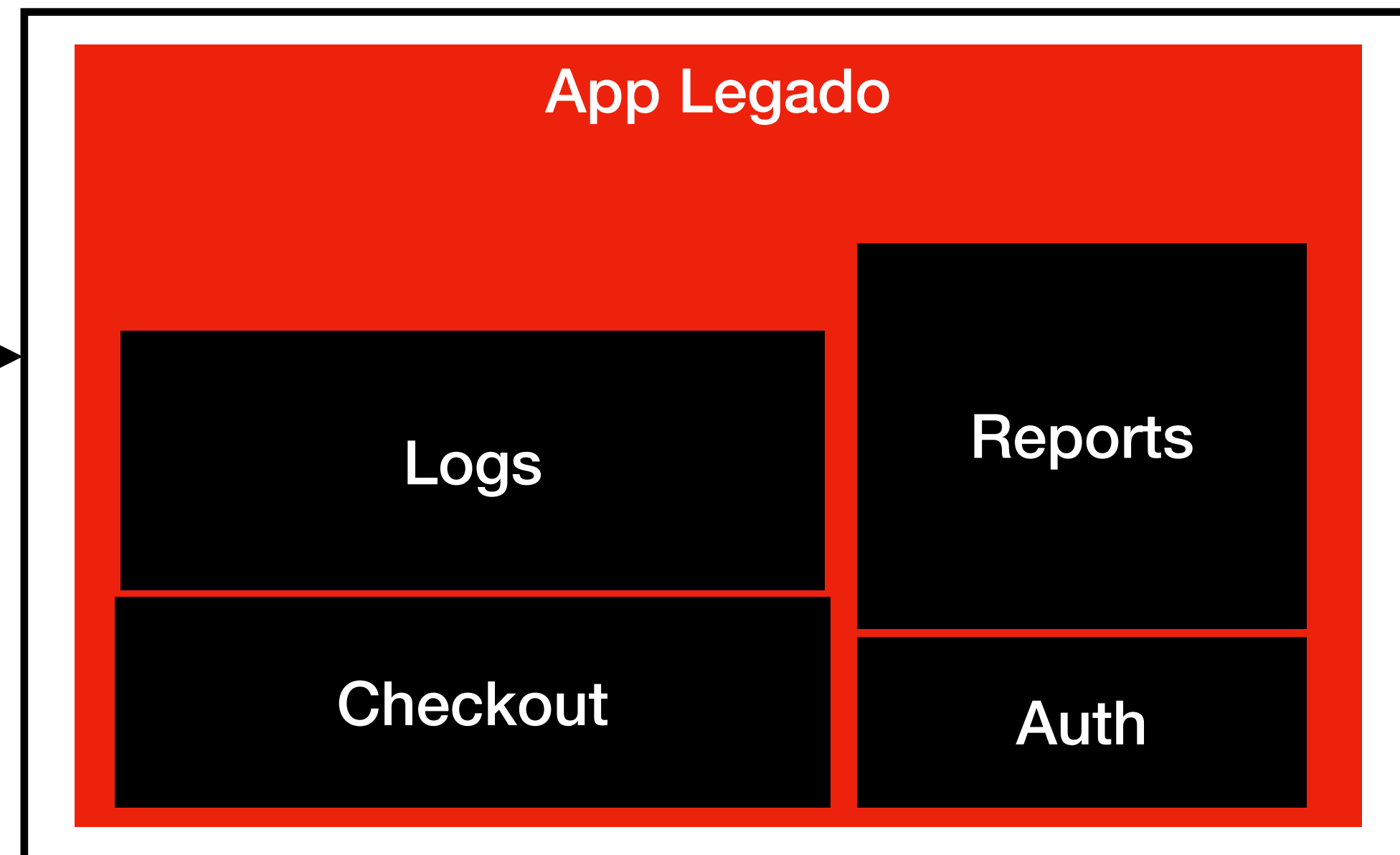
/reports

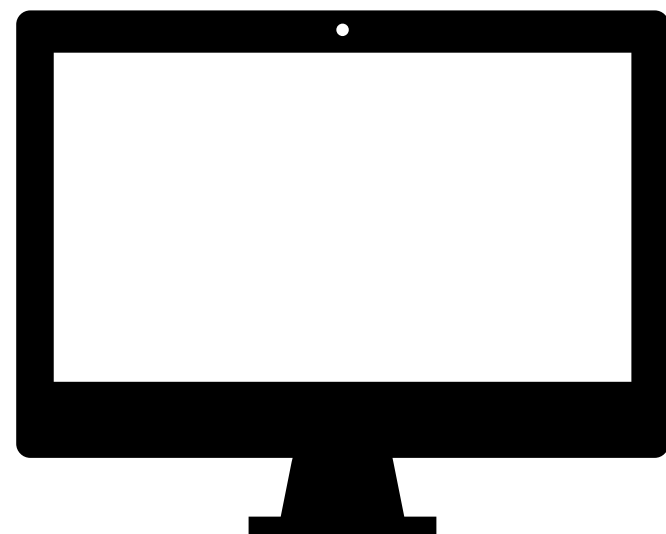
/auth



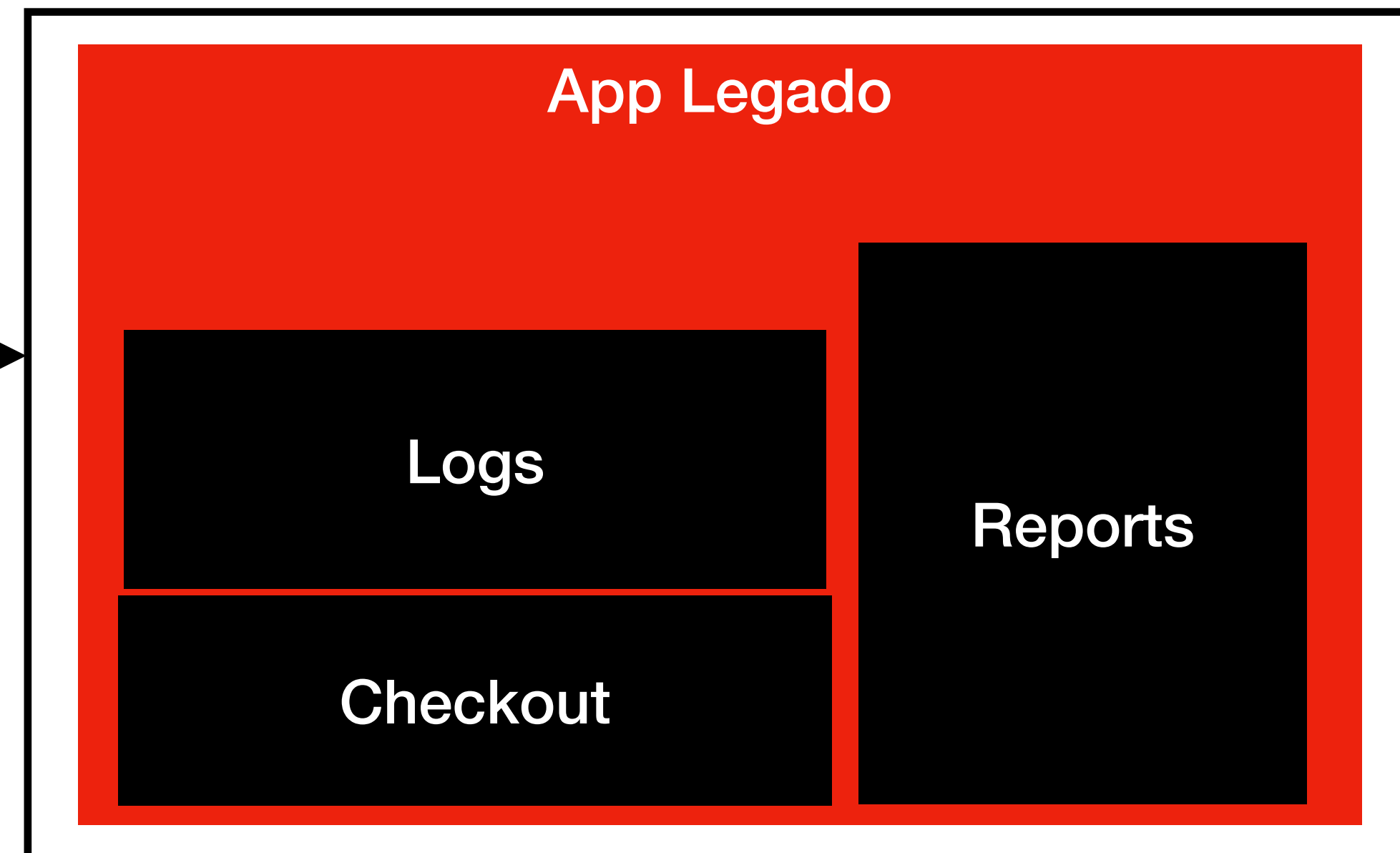


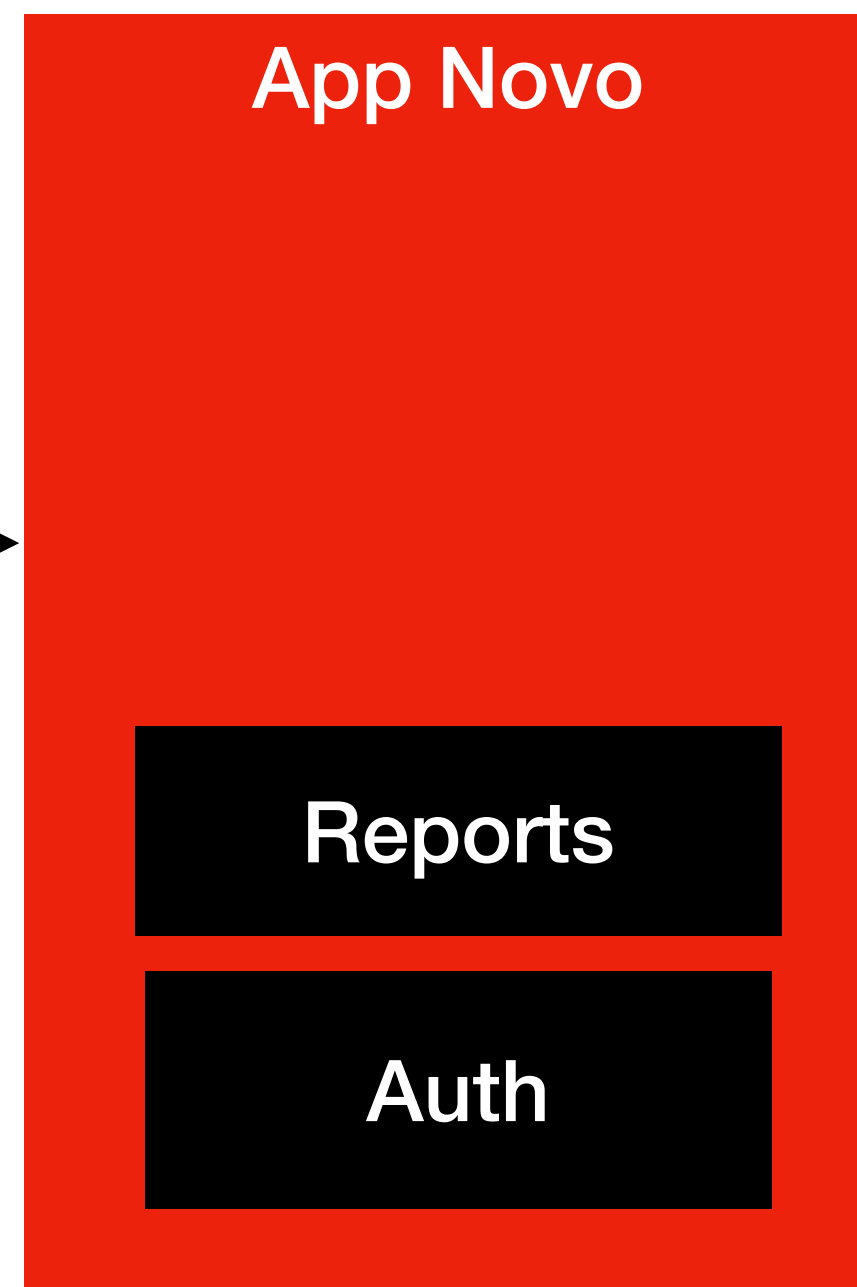
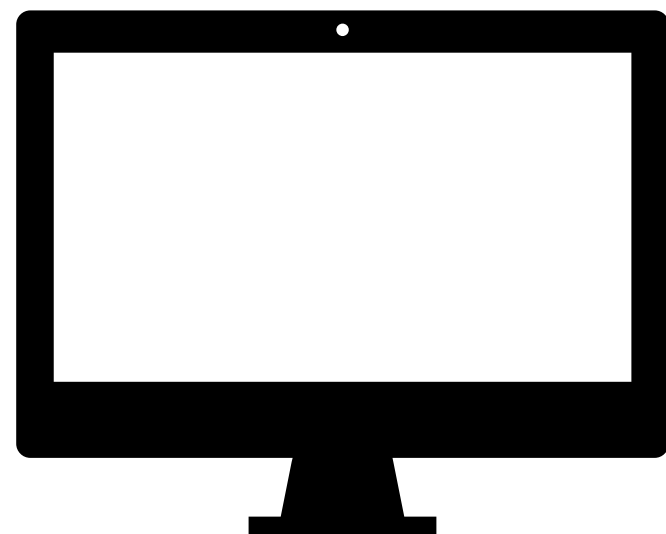
/logs
/checkout
/reports
/auth



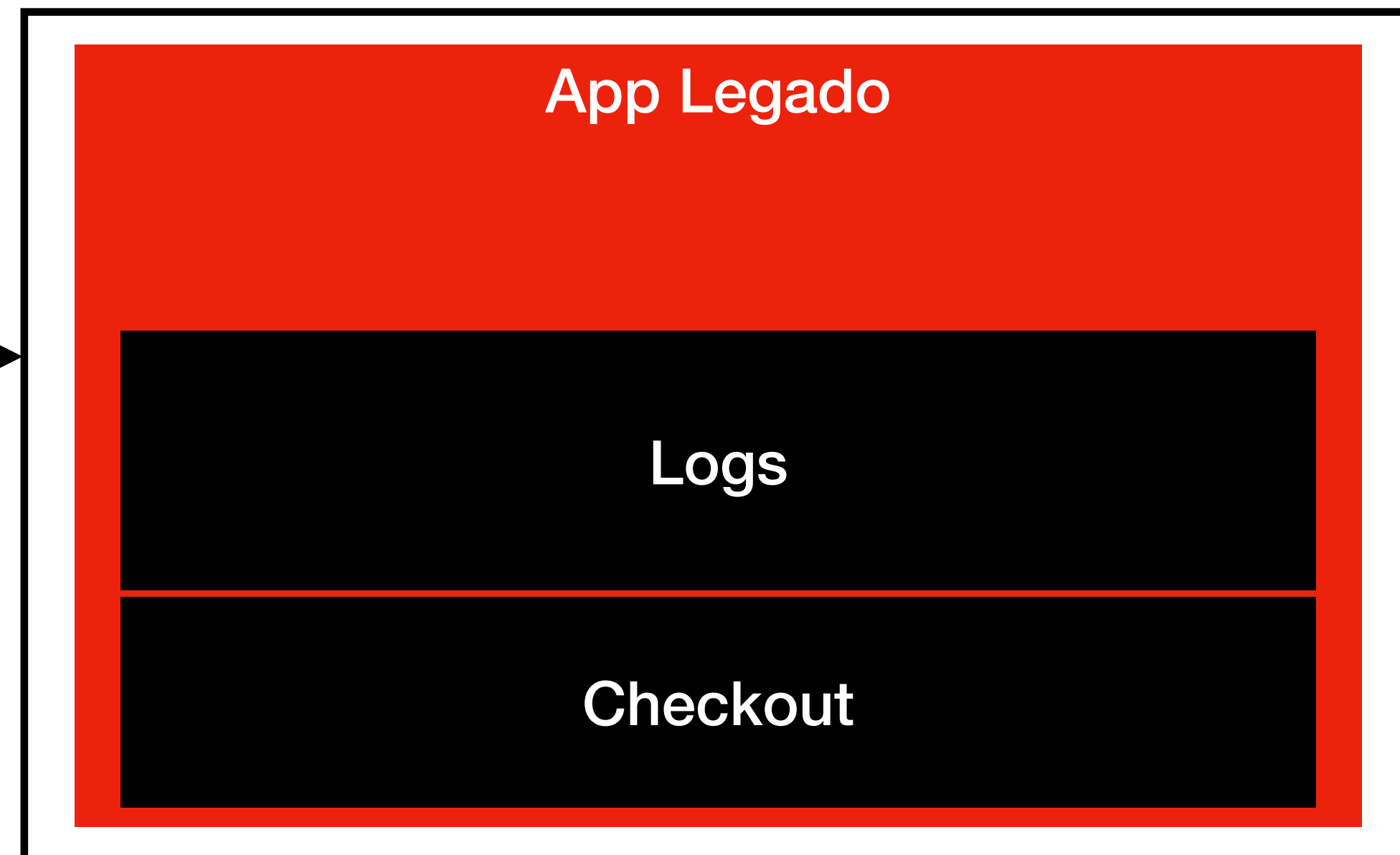


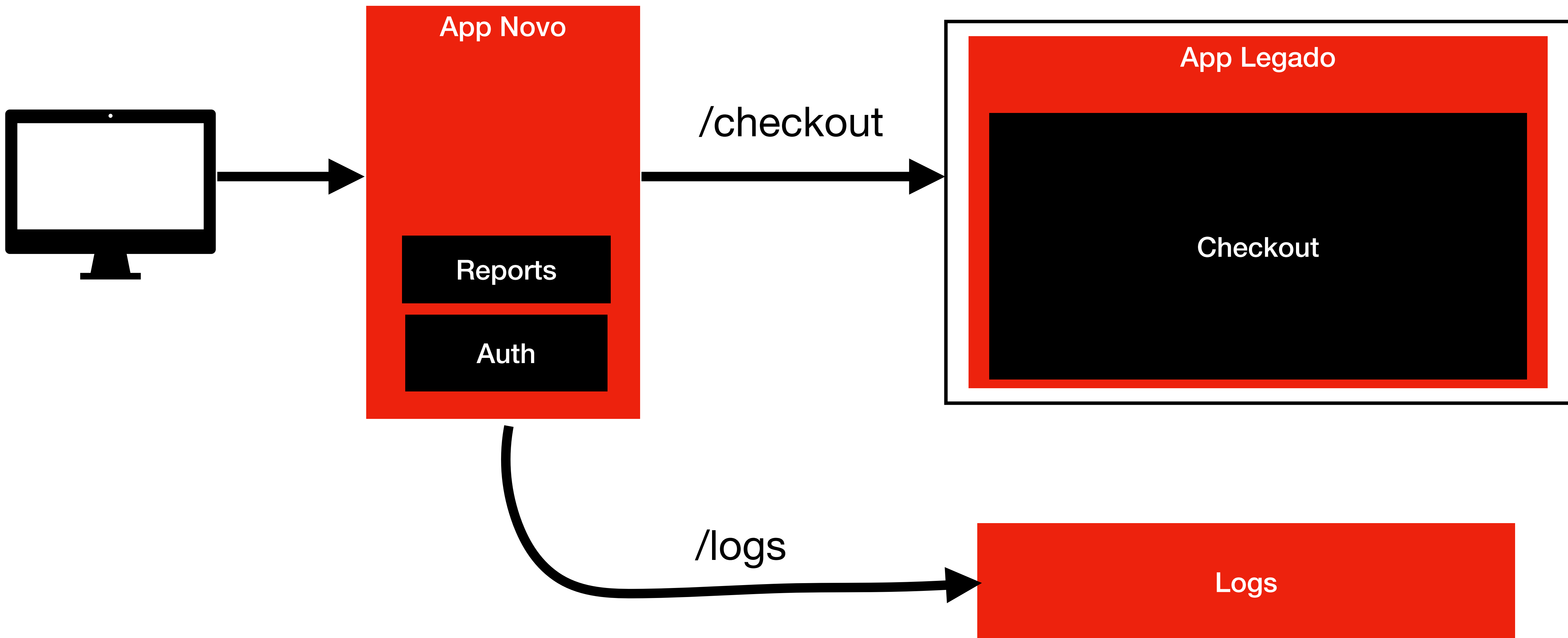
/logs
/checkout
/reports

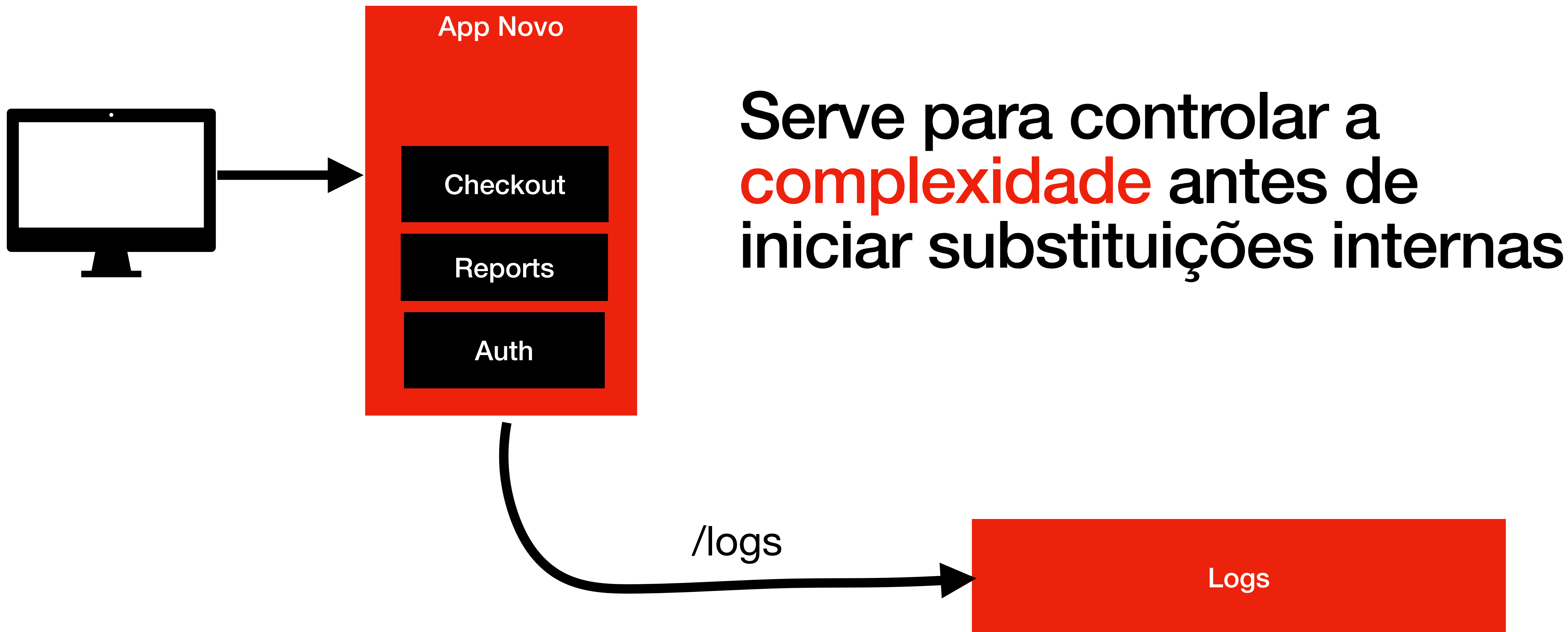




/logs
/checkout

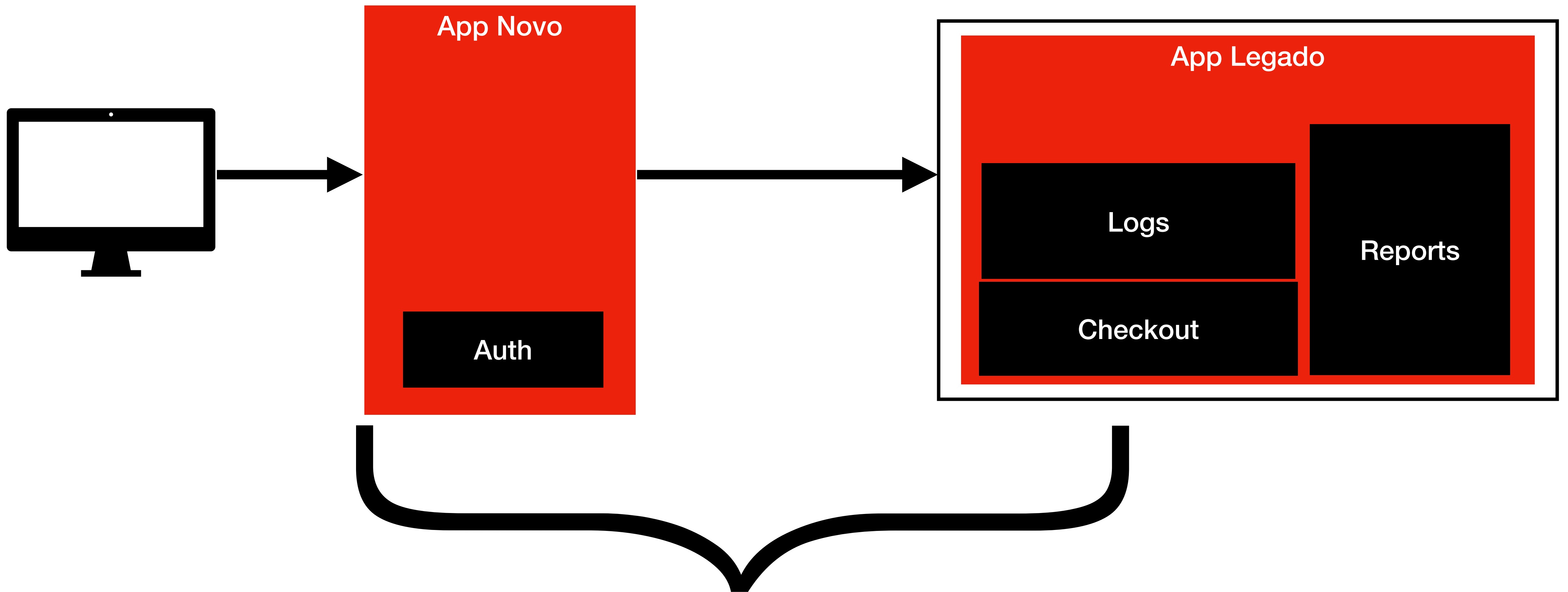






Pontos de Atenção

A dependência com o legado é muito forte durante a migração



App novo totalmente **dependente** do legado

Nem sempre as **interfaces** do
legado são boas e usuais

```
{
  "CodStatus": 7,
  "msg": "ok",
  "ret": {
    "p": "Aprovado",
    "vl": "129,90",
    "id": 991233,
    "t": "CC",
    "pagto_status": 1,
    "3DSv2result": "N/A",
    "erros": null,
    "Cliente": [
      "Luiz",
      "schons",
      "cpf:00011122233"
    ],
    "ENDERECO": {
      "CEP": 85000000,
      "cdd": "Guarapuava",
      "uf": "PR",
      "completo": 12
    },
    "FRAUDE_SCORE": "19%"
  }
}
```

```
{
  "status": "success",
  "payload": "{\"transaction\":991233,...}",
  "code": 200
}
```

```
{
  "status": "ERROR",
  "code": 57,
  "message": "Falha ao processar pagamento",
  "details": {
    "gateway_status": "DENIED",
    "reason": "cartao invalido",
    "attempt_id": null,
    "retry": "false"
  },
  "success": true
}
```

Pode criar **acoplamento** se o
design não for limpo


```
<?php

use Illuminate\Support\Facades\Http;

class CheckoutService
{
    private string $legacyApi;

    public function __construct(string $legacyApi)
    {
        $this->legacyApi = $legacyApi;
    }

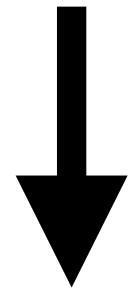
    public function processPayment(int $orderId): array
    {
        $response = Http::get($this->legacyApi . "/order/" . $orderId);

        $data = $response->json();

        if ($data['COD_STATUS'] === '7') {
            $status = 'APROVADO';
        } elseif ($data['COD_STATUS'] === '3') {
            $status = 'NEGADO';
        } else {
            $status = 'DESCONHECIDO';
        }

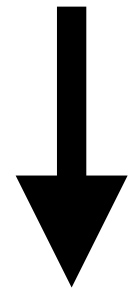
        return [
            'status' => $status,
            'amount' => str_replace(',', '.', $data['VALOR']),
            'extra' => $data['INF01'] . '|' . $data['INF02'],
        ];
    }
}
```

```
$response = Http::get($this->legacyApi . "/order/" . $orderId);
```



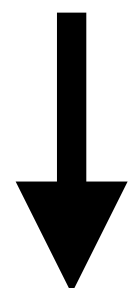
Acoplamento direto à API legado (dificulta testes e futuras trocas)


```
$response = Http::get($this->legacyApi . "/order/" . $orderId);
```



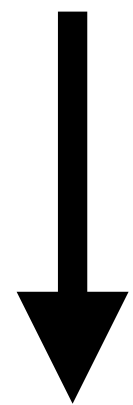
Acoplamento direto à API legado (dificulta testes e futuras trocas)

```
$data = $response->json( );
```



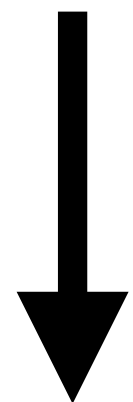
Nenhum tratamento de erro da API (timeout, 500, resposta inválida)

```
if ( $data[ 'COD_STATUS' ] === '7' ) {  
    $status = 'APROVADO';  
} elseif ( $data[ 'COD_STATUS' ] === '3' ) {  
    $status = 'NEGADO';  
} else {  
    $status = 'DESCONHECIDO';  
}
```



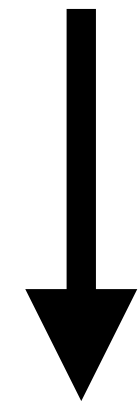
Reutilização de códigos mágicos do legado

```
if ( $data[ 'COD_STATUS' ] === '7' ) {  
    $status = 'APROVADO';  
} elseif ( $data[ 'COD_STATUS' ] === '3' ) {  
    $status = 'NEGADO';  
} else {  
    $status = 'DESCONHECIDO';  
}
```



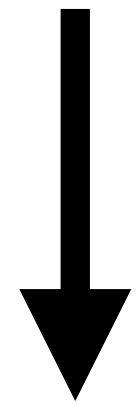
Falta de contrato forte: legado pode mudar o status sem aviso


```
return [  
    'status' => $status,  
    'amount' => str_replace( ',', ' ', ' ', $data[ 'VALOR' ] ),  
    'extra'   => $data[ 'INF01' ] . ' | ' . $data[ 'INF02' ],  
];
```



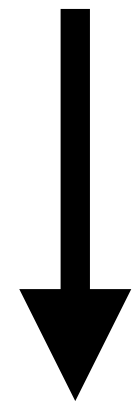
Novo sistema replicando regras antigas e pouco claras

```
return [  
    'status' => $status,  
    'amount' => str_replace( ',', ' ', ' . ', $data[ 'VALOR' ] ),  
    'extra'   => $data[ 'INF01' ] . ' | ' . $data[ 'INF02' ],  
];
```



Novo sistema replicando regras antigas e pouco claras
Gambiarra de normalização baseada em formato legado

```
return [  
    'status' => $status,  
    'amount' => str_replace( ',', ' ', ' . ', $data[ 'VALOR' ] ),  
    'extra'   => $data[ 'INF01' ] . ' | ' . $data[ 'INF02' ],  
];
```



Novo sistema replicando regras antigas e pouco claras
Gambiarra de normalização baseada em formato legado
Mantendo formato estranho


```
<?php

declare(strict_types=1);

namespace App\Service;

use App\Contracts\LegacyOrderClientInterface;
use App\DTO\OrderData;

class CheckoutService
{
    public function __construct(
        private LegacyOrderClientInterface $legacyClient
    ) {}

    public function processPayment(int $orderId): OrderData
    {
        $order = $this->legacyClient->fetchOrder($orderId);

        $status = match ($order->statusCode) {
            '7' => 'APROVADO',
            '3' => 'NEGADO',
            default => 'DESCONHECIDO',
        };

        $amount = $order->normalizedAmount();

        $extra = $order->formattedExtra();

        return new OrderData(
            status: $status,
            amount: $amount,
            extra: $extra
        );
    }
}
```

```
class CheckoutService
{
    public function __construct(
        private LegacyOrderClientInterface $legacyClient
    ) {}
}
```



Inversão de dependência

Inversão de dependência?

Módulos de alto nível **não devem**
depender de módulos de baixo nível.

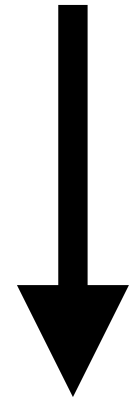
Ambos devem depender de **abstrações**.

```
class CheckoutService
{
    public function __construct(
        private LegacyOrderClientInterface $legacyClient
    ) {}

}
```

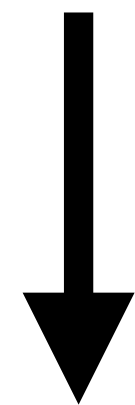


```
$order = $this->legacyClient->fetchOrder($orderId);
```



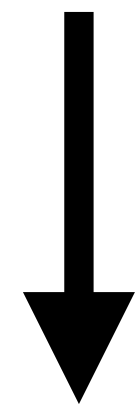
O client retorna um objeto forte (DTO)
ao invés de array solto

```
$status = match ($order->statusCode) {  
    '7' => 'APROVADO',  
    '3' => 'NEGADO',  
    default => 'DESCONHECIDO',  
};
```



Lógica de status **encapsulada** e padronizada

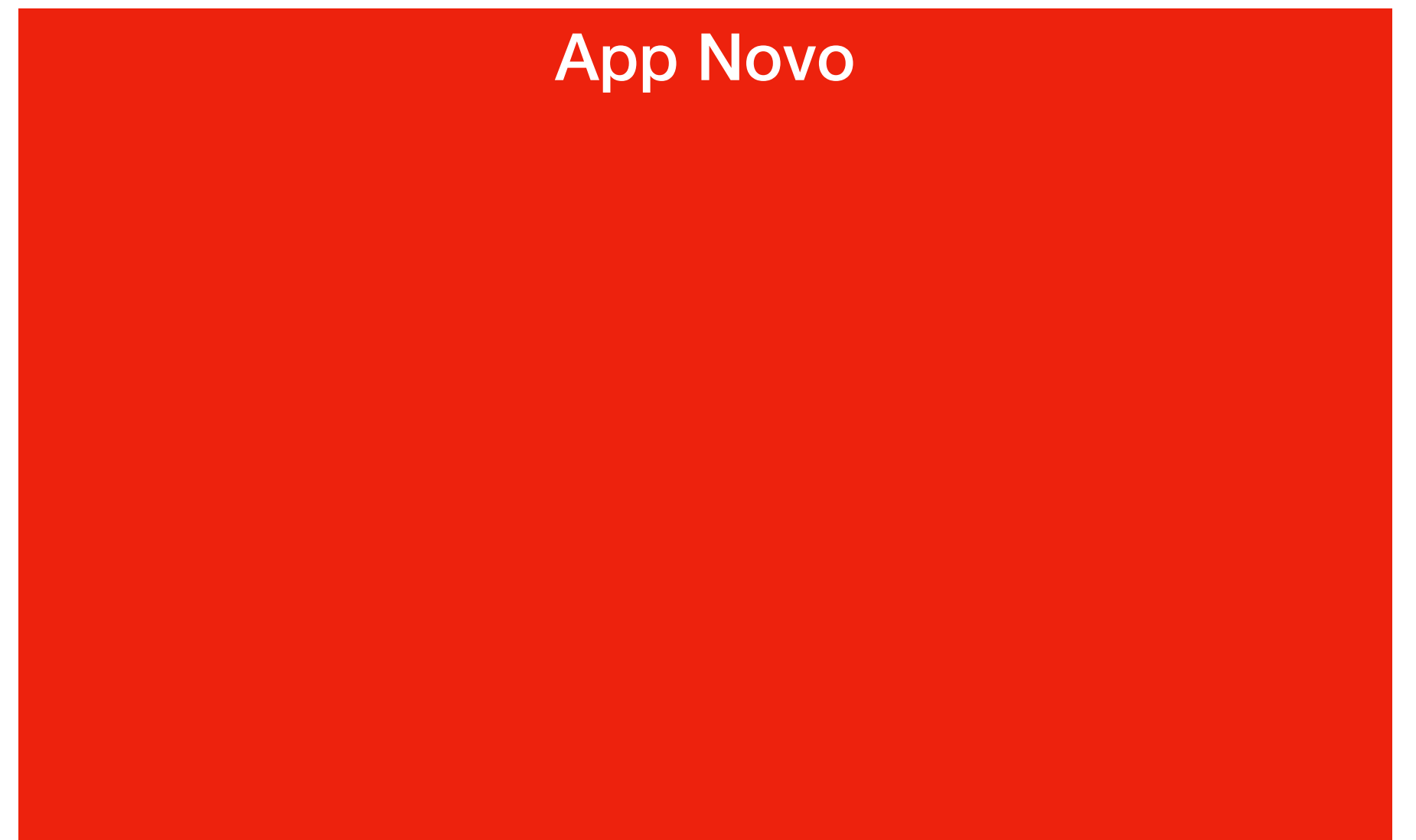
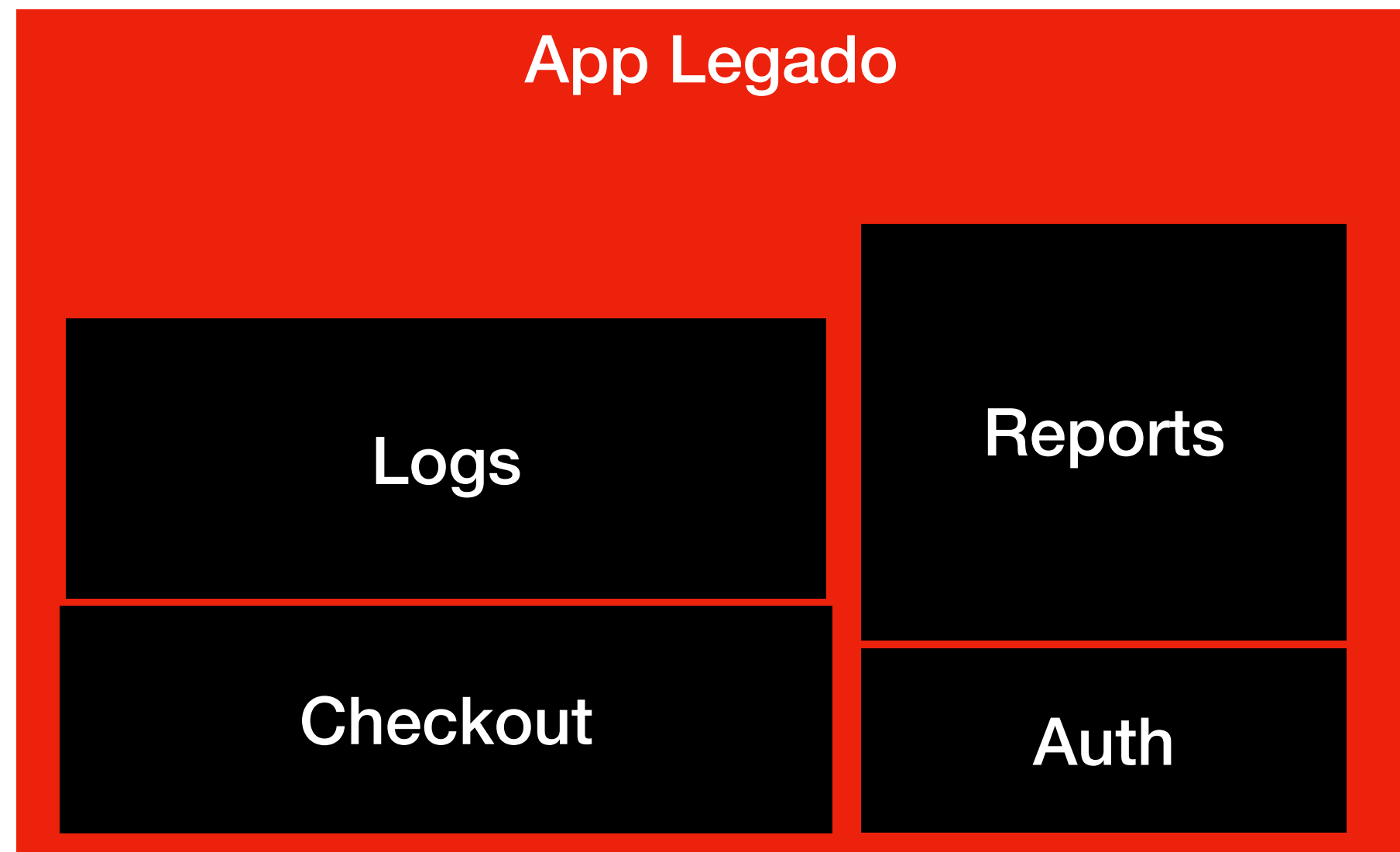
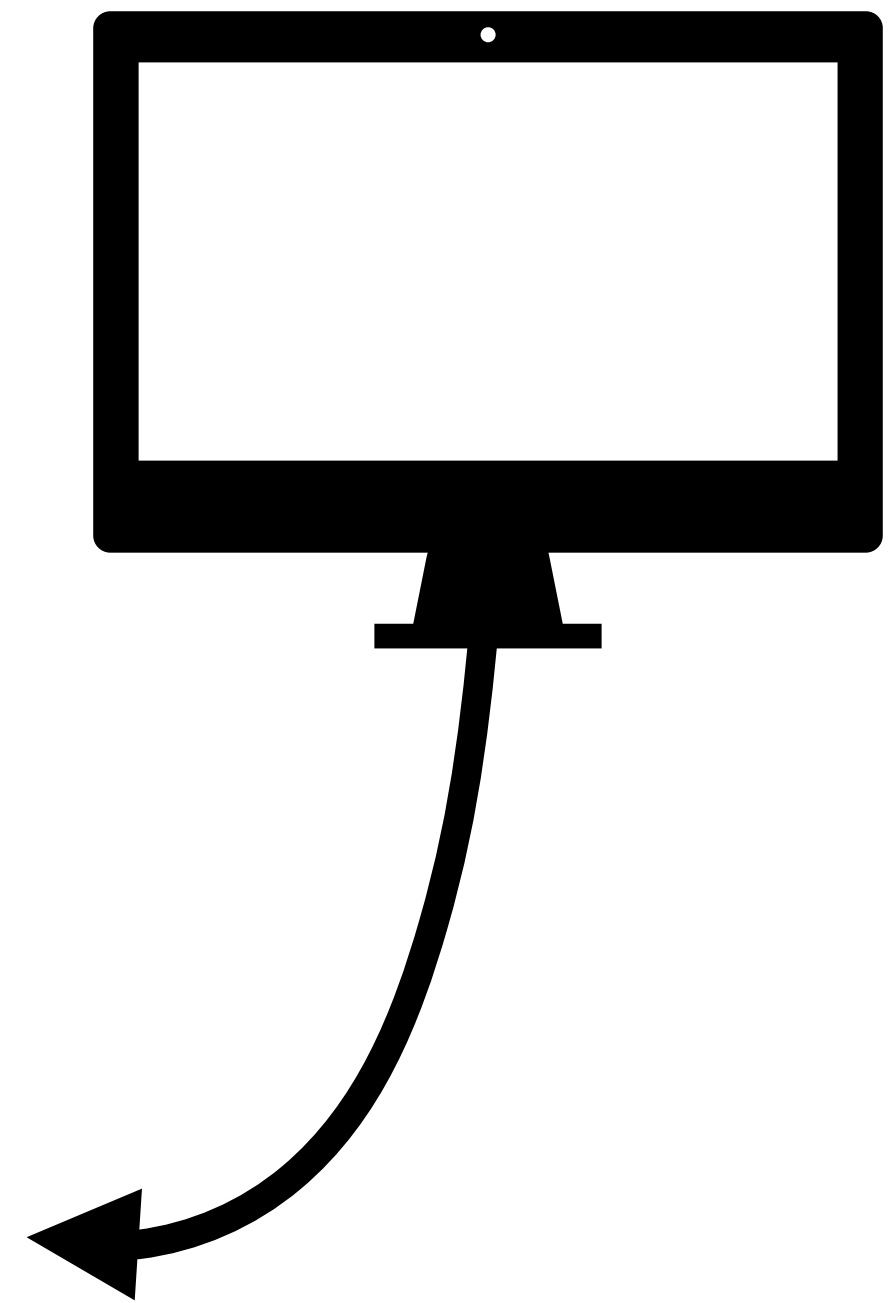
```
return new OrderData(  
    status: $status,  
    amount: $amount,  
    extra: $extra  
);
```

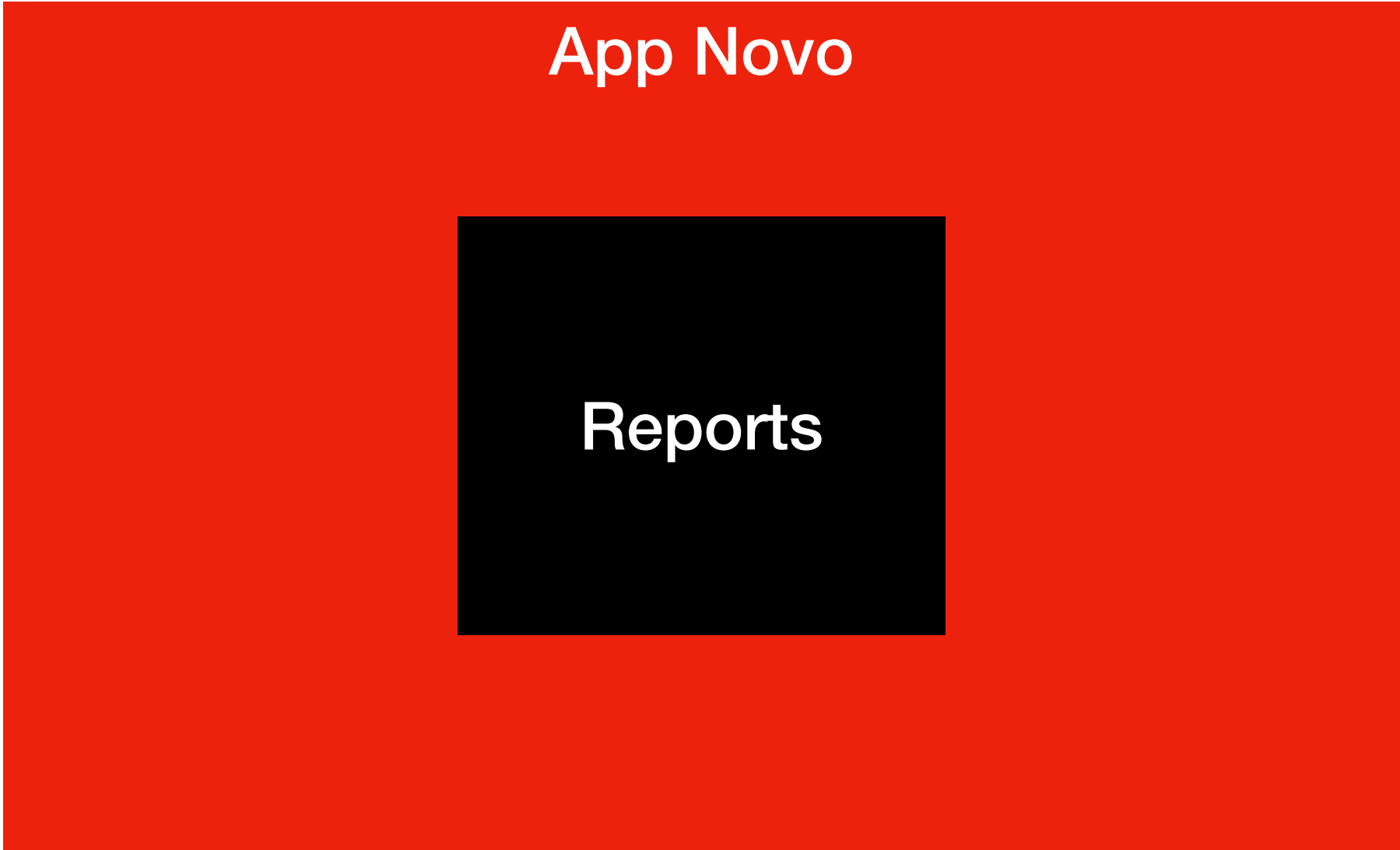
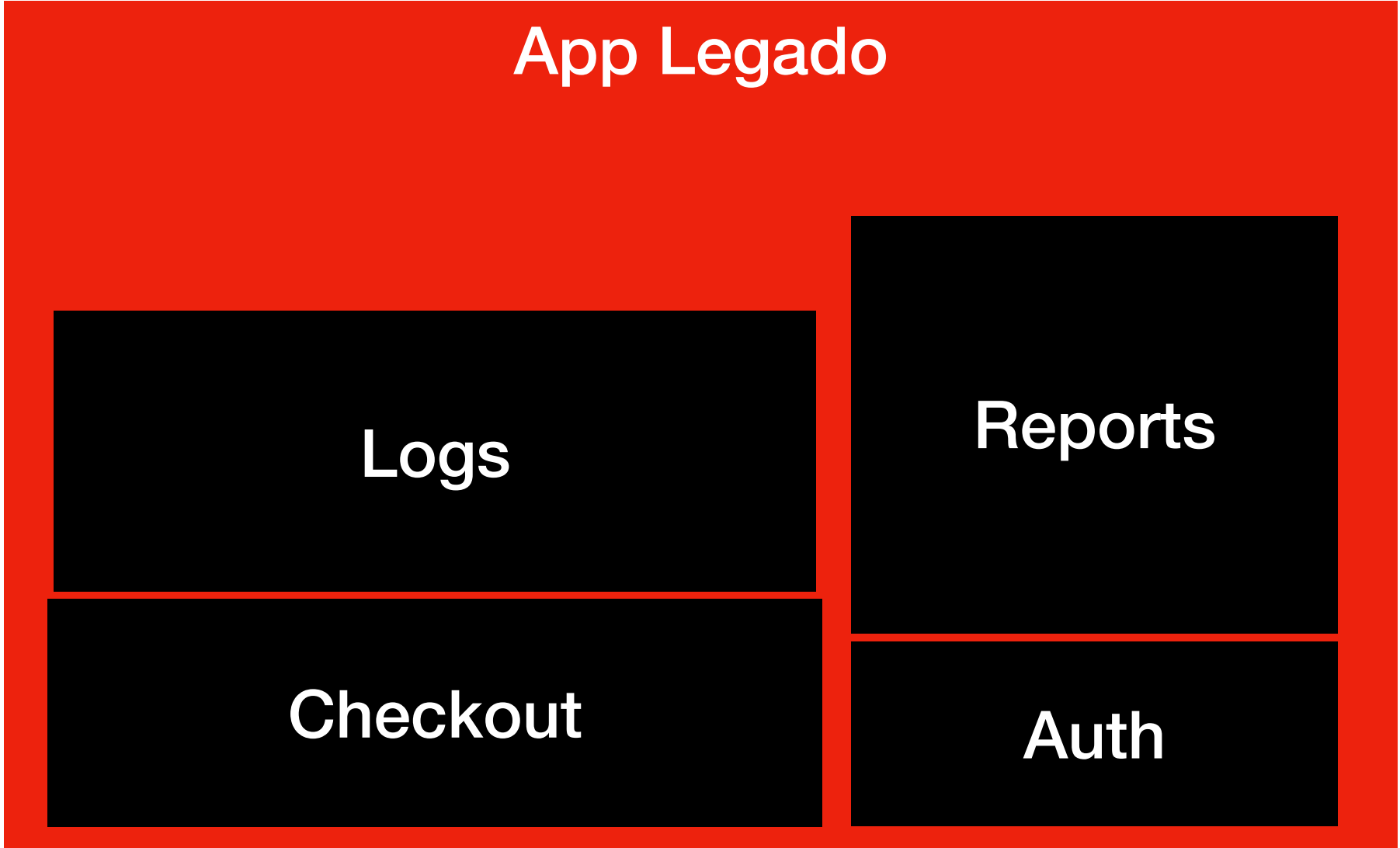
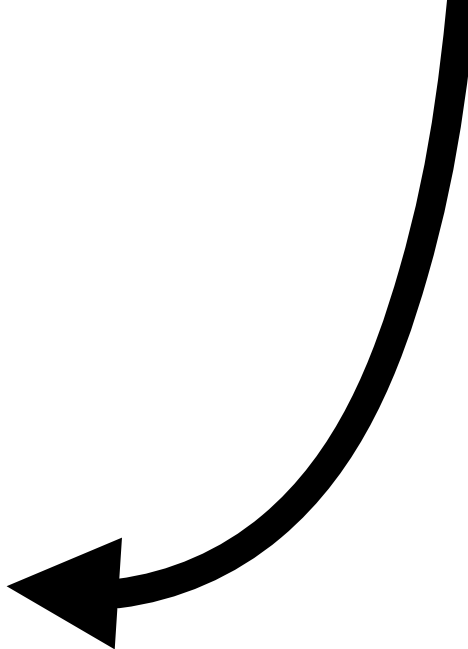
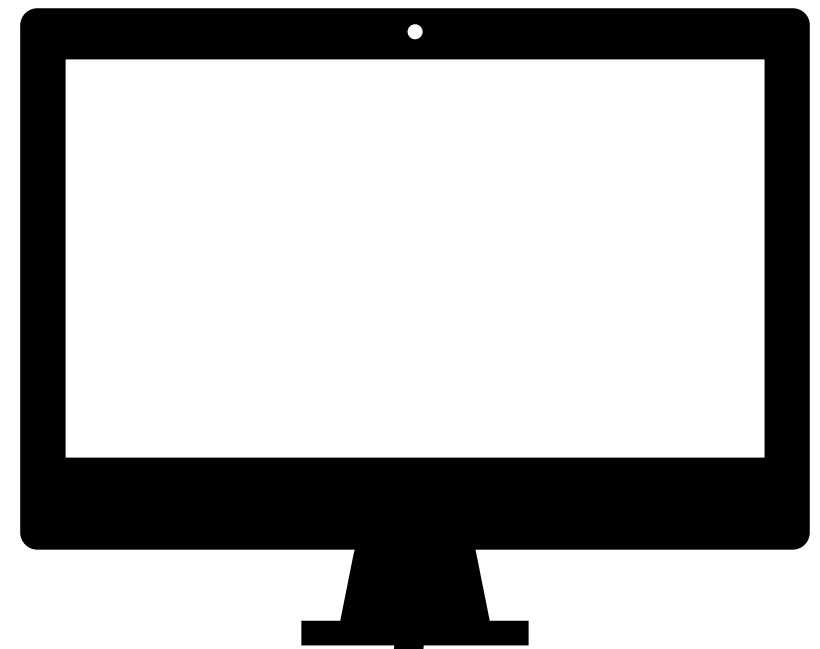


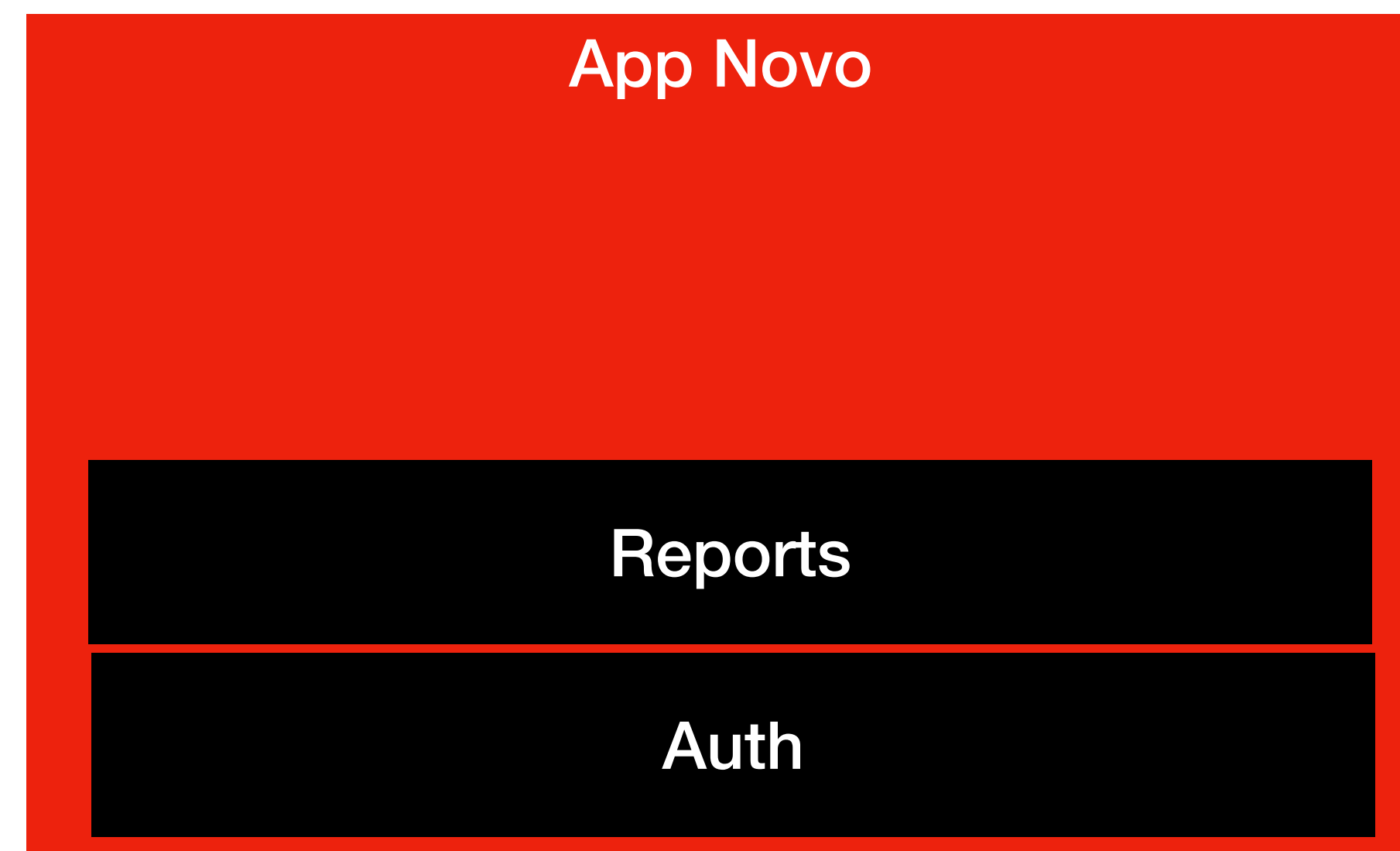
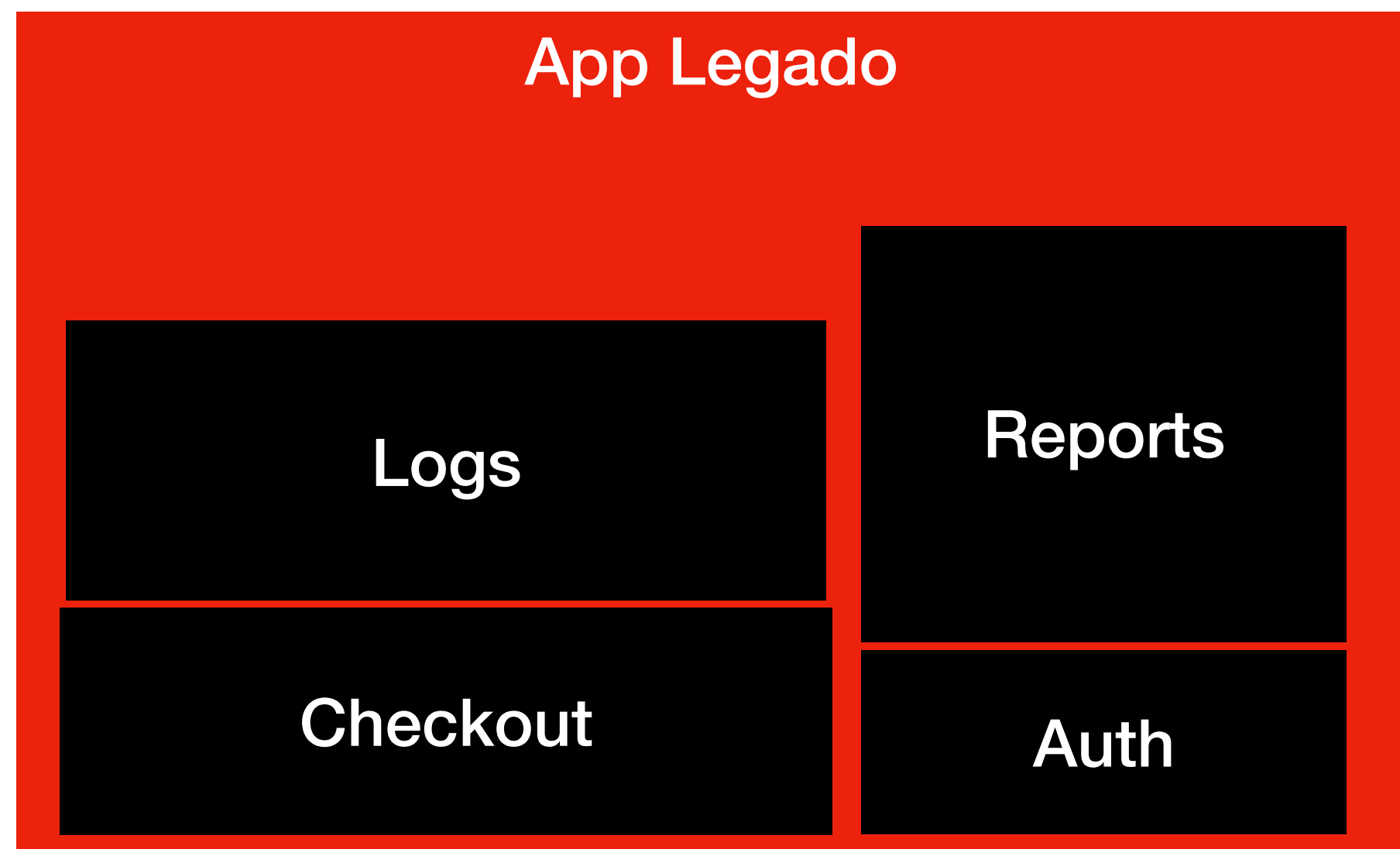
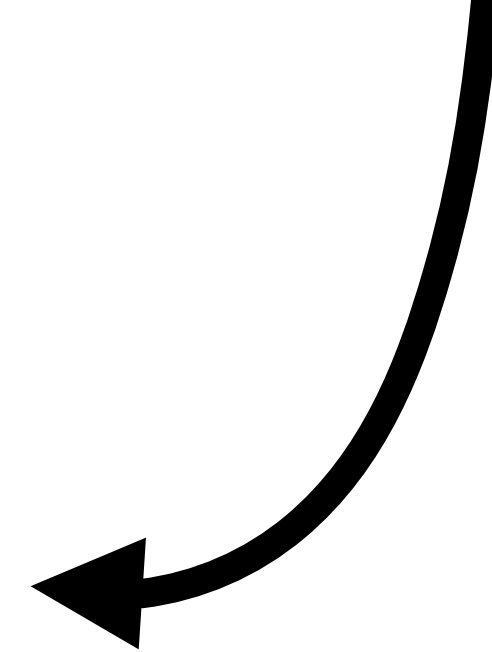
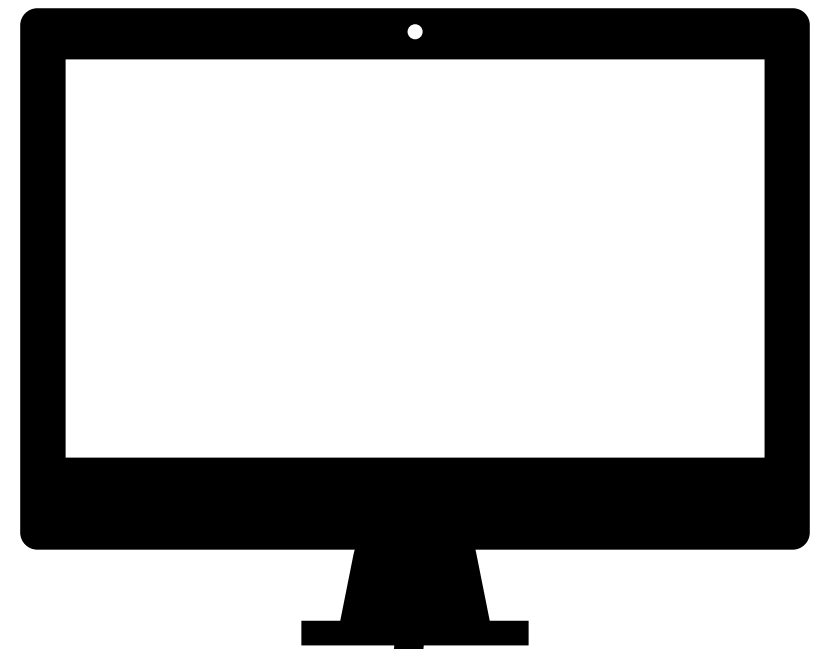
Service retorna um objeto de domínio (OrderData), **nunca a resposta bruta do legado**

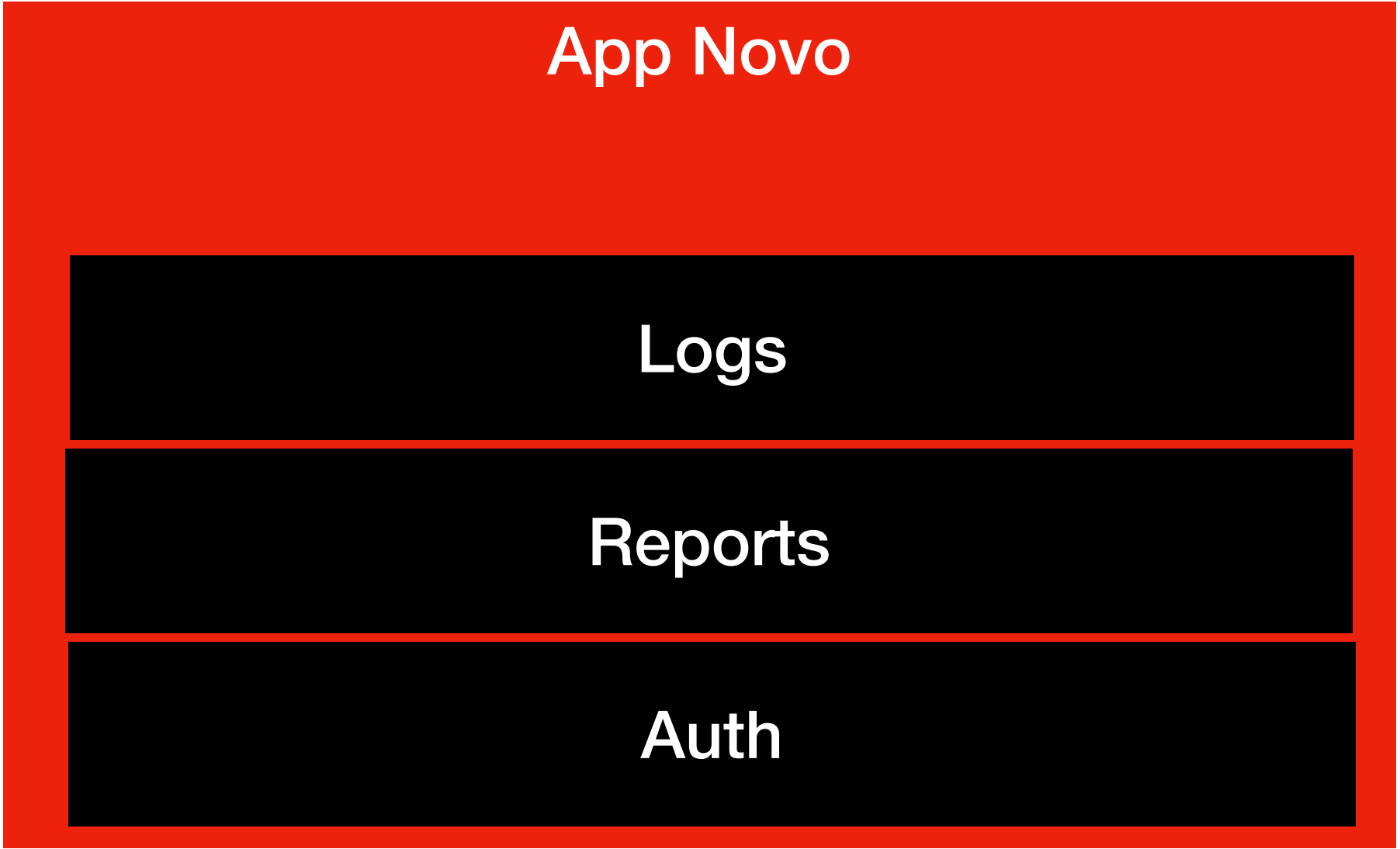
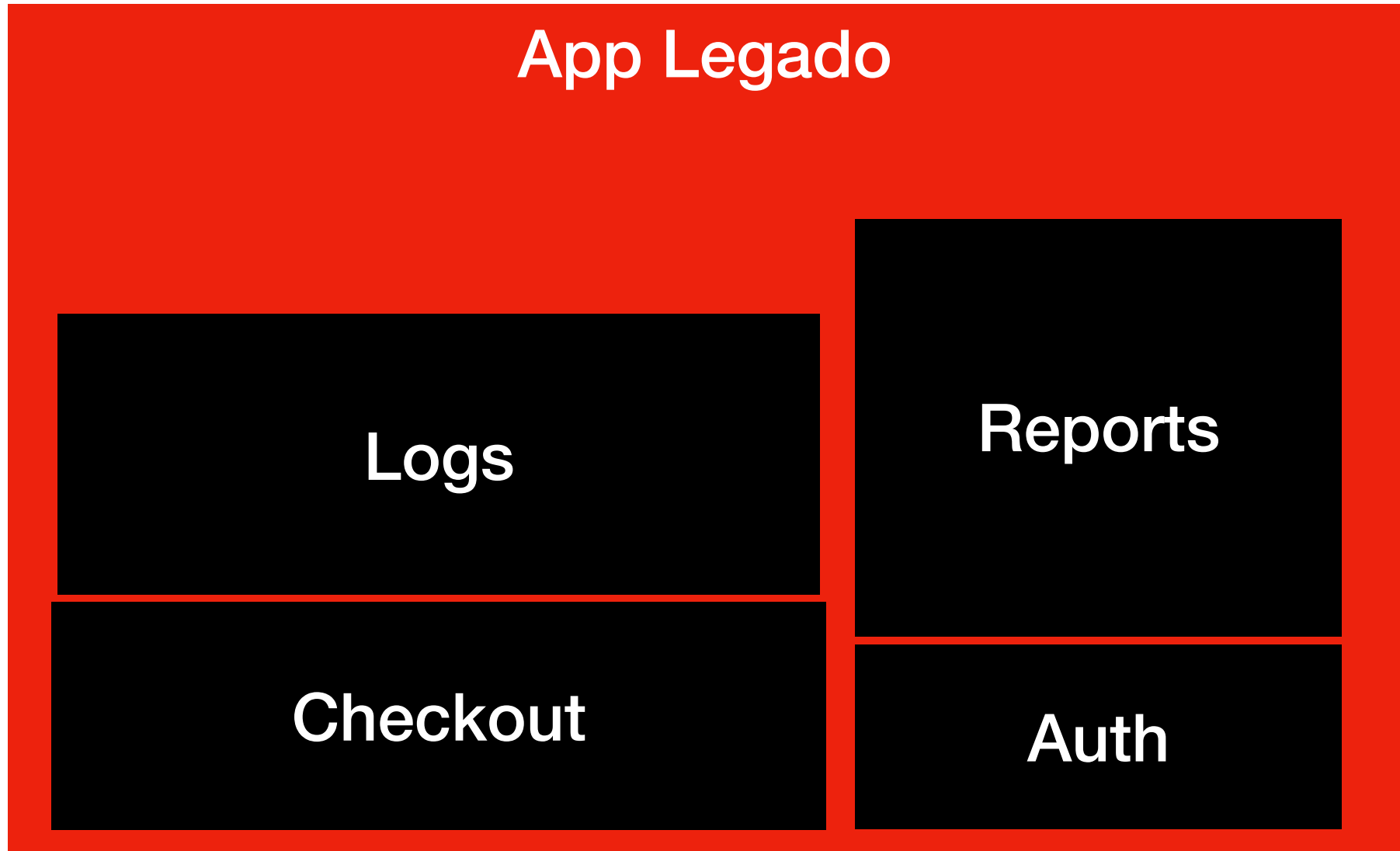
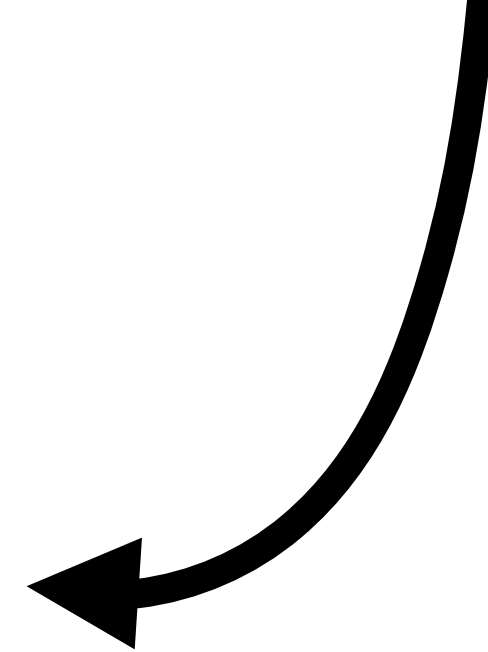
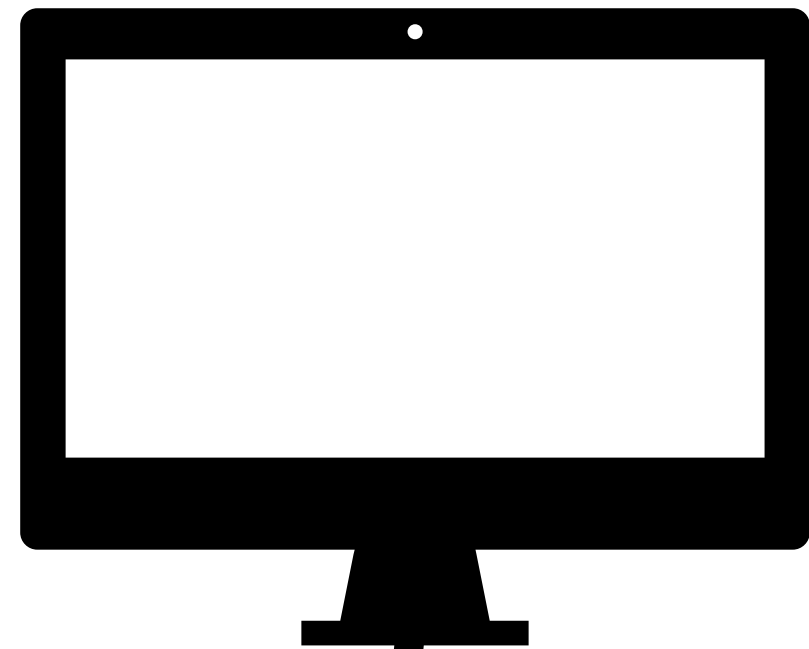
Nesses exemplos você **começa** a resolver o problema de **acoplamento** entre os serviço

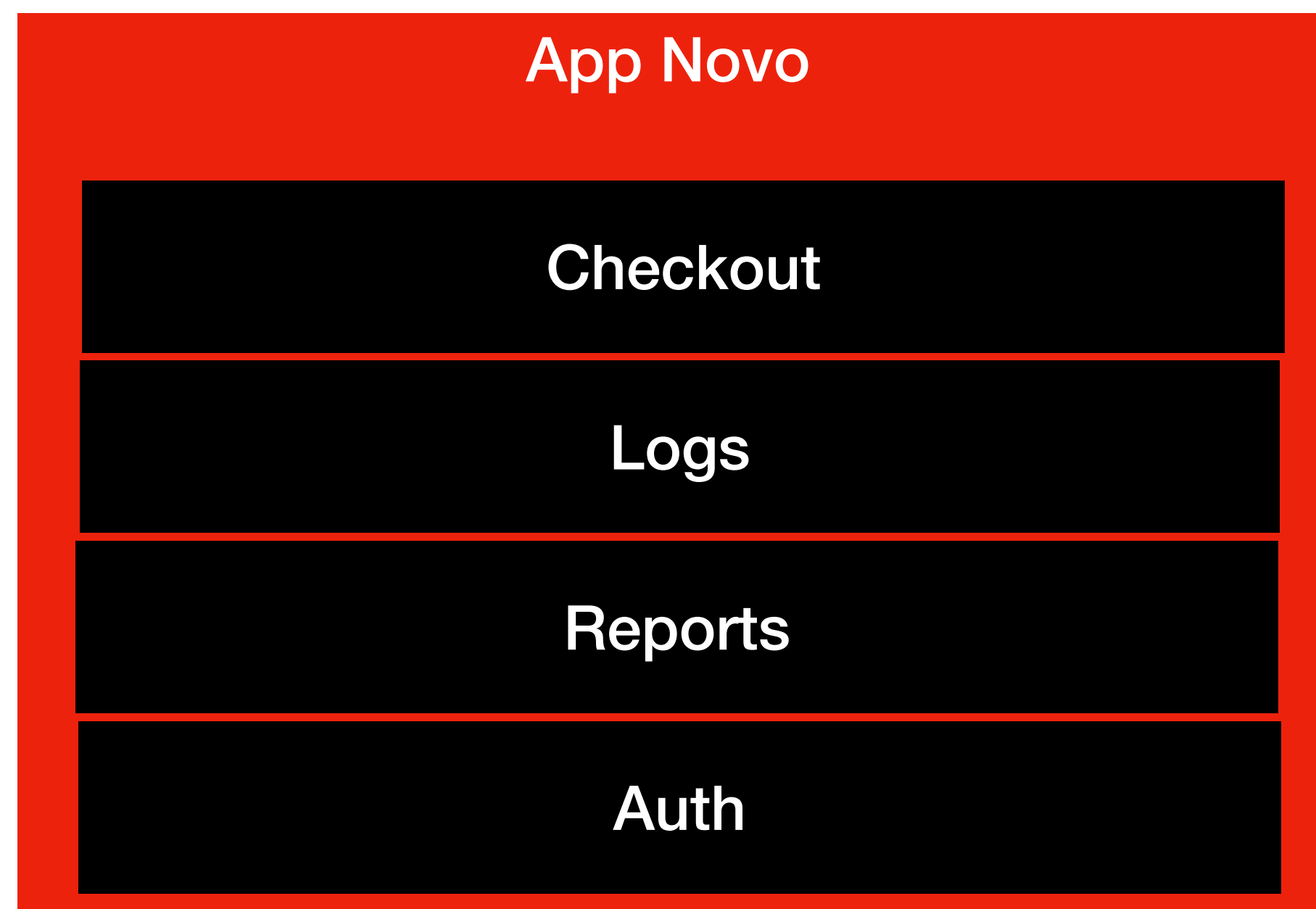
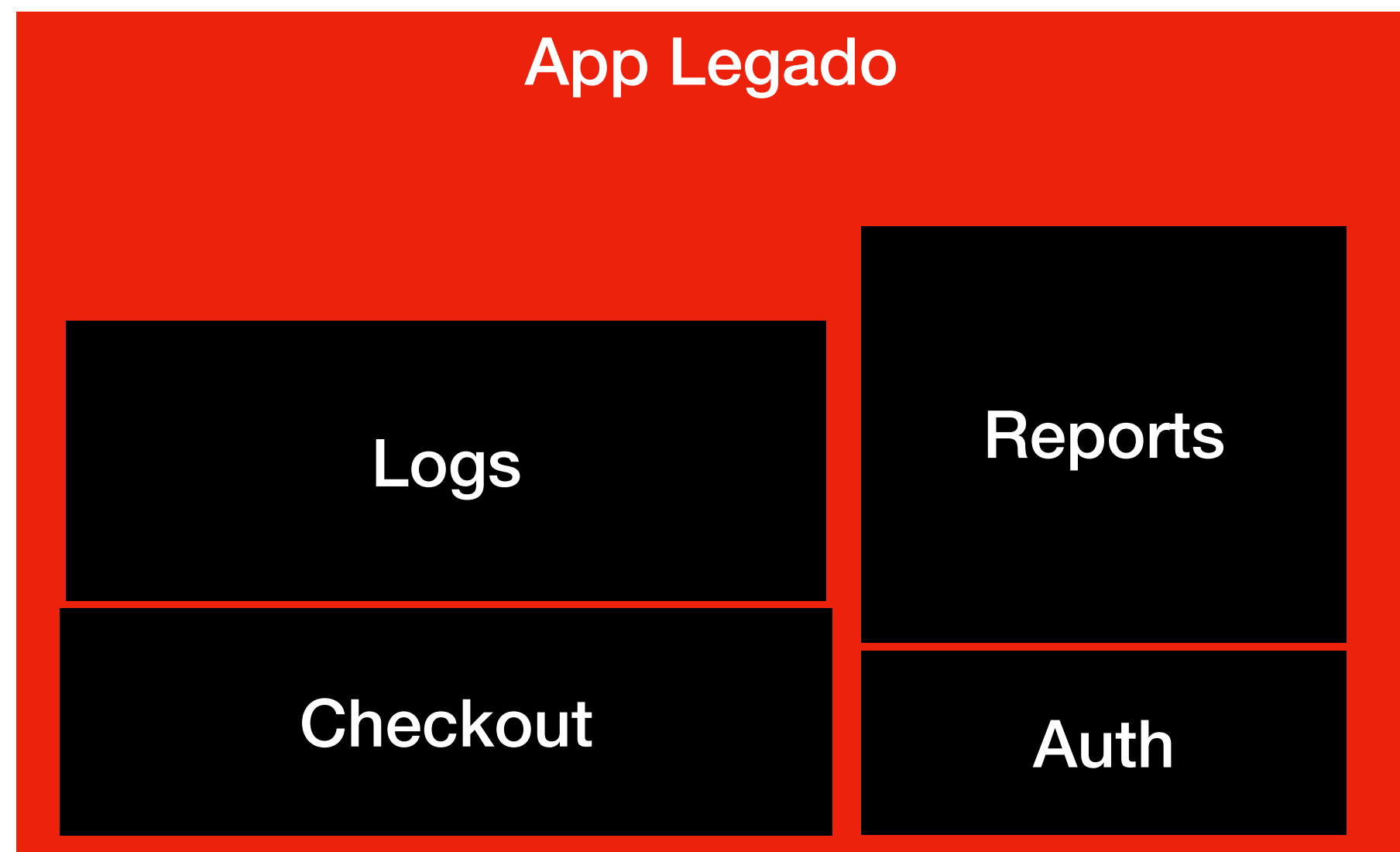
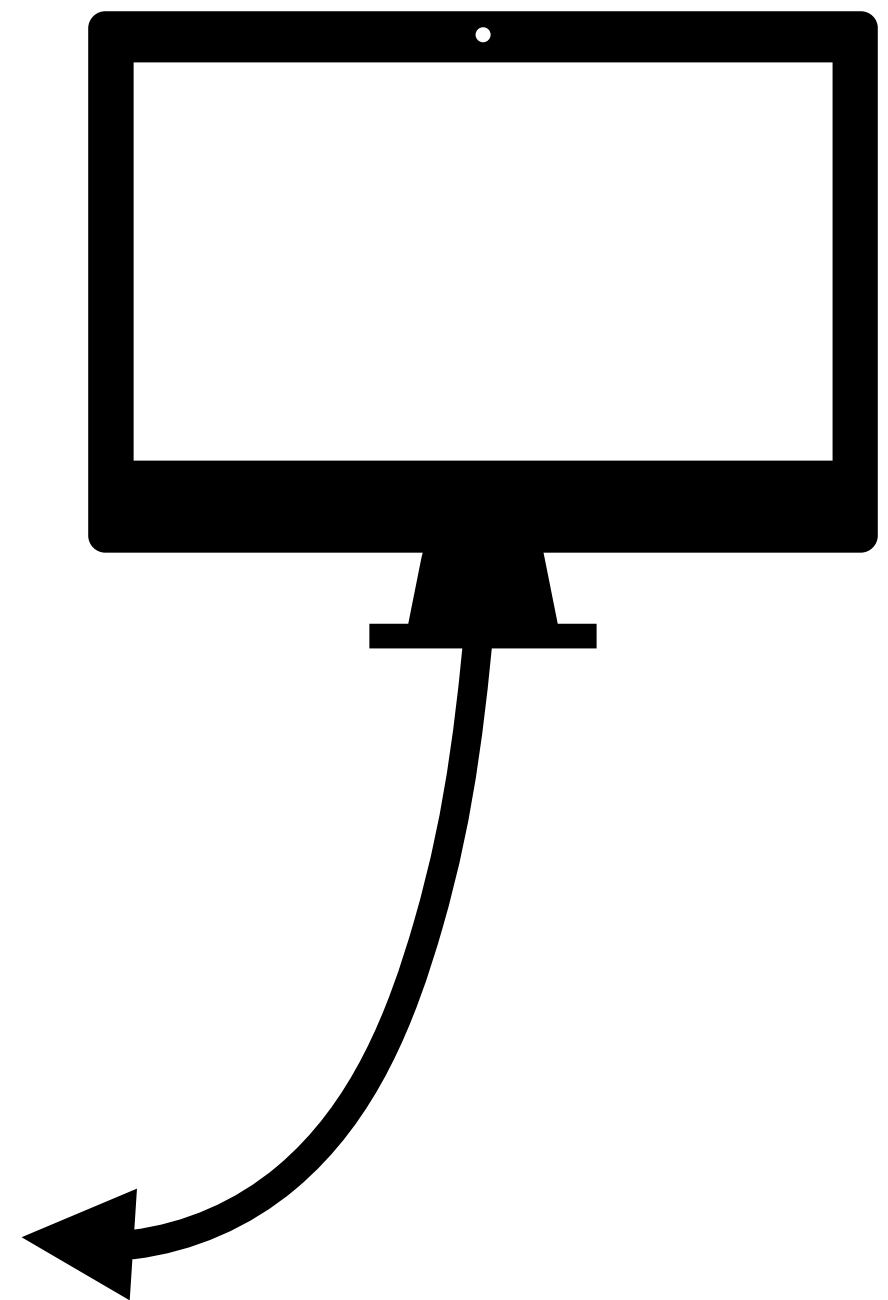
Big Bang / Full Rewrite

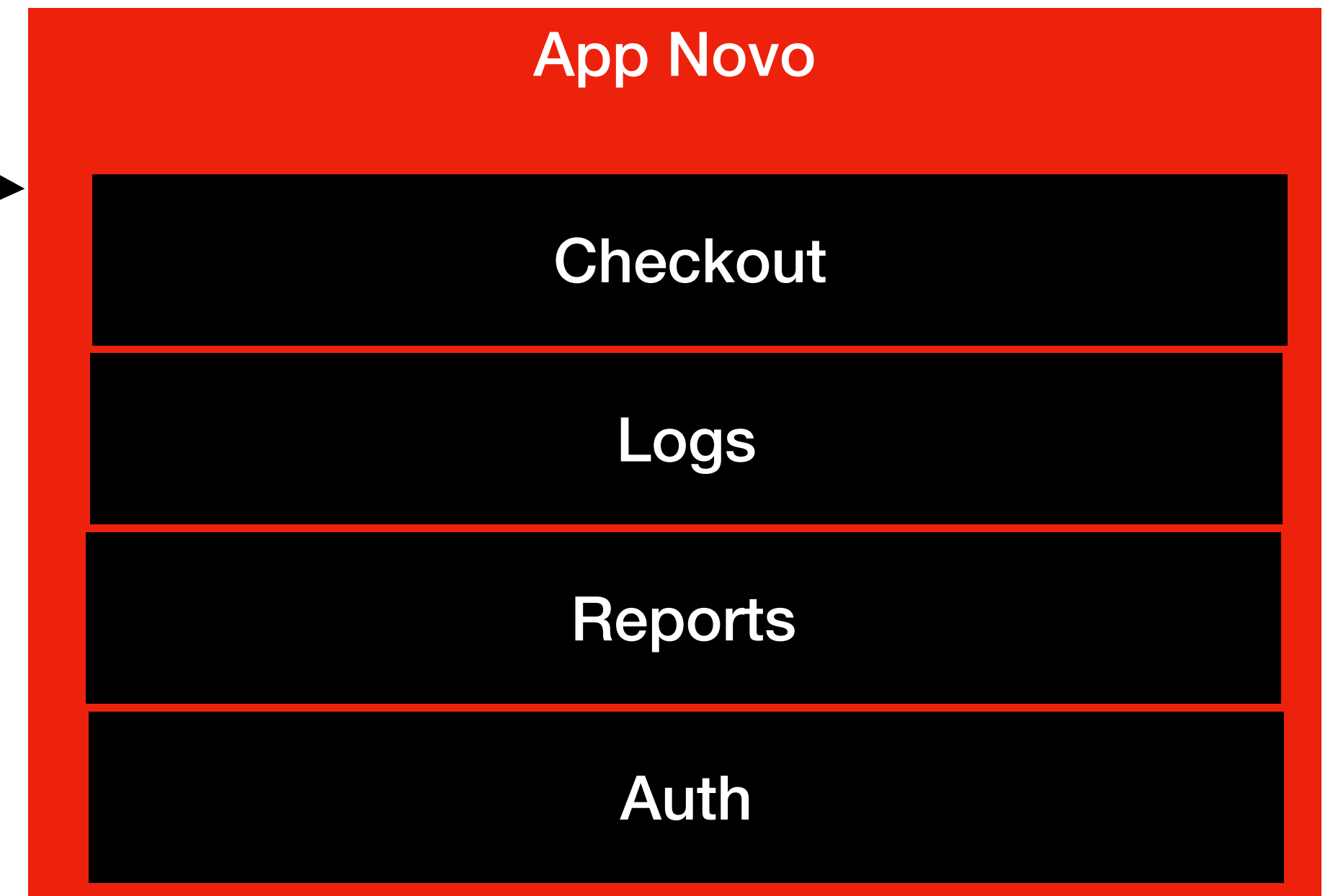
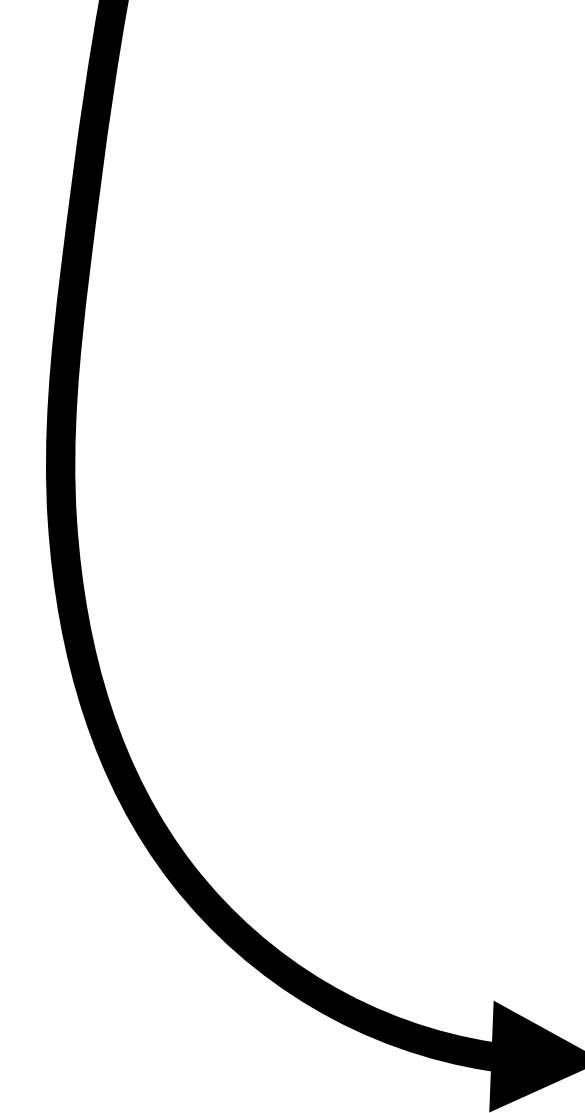
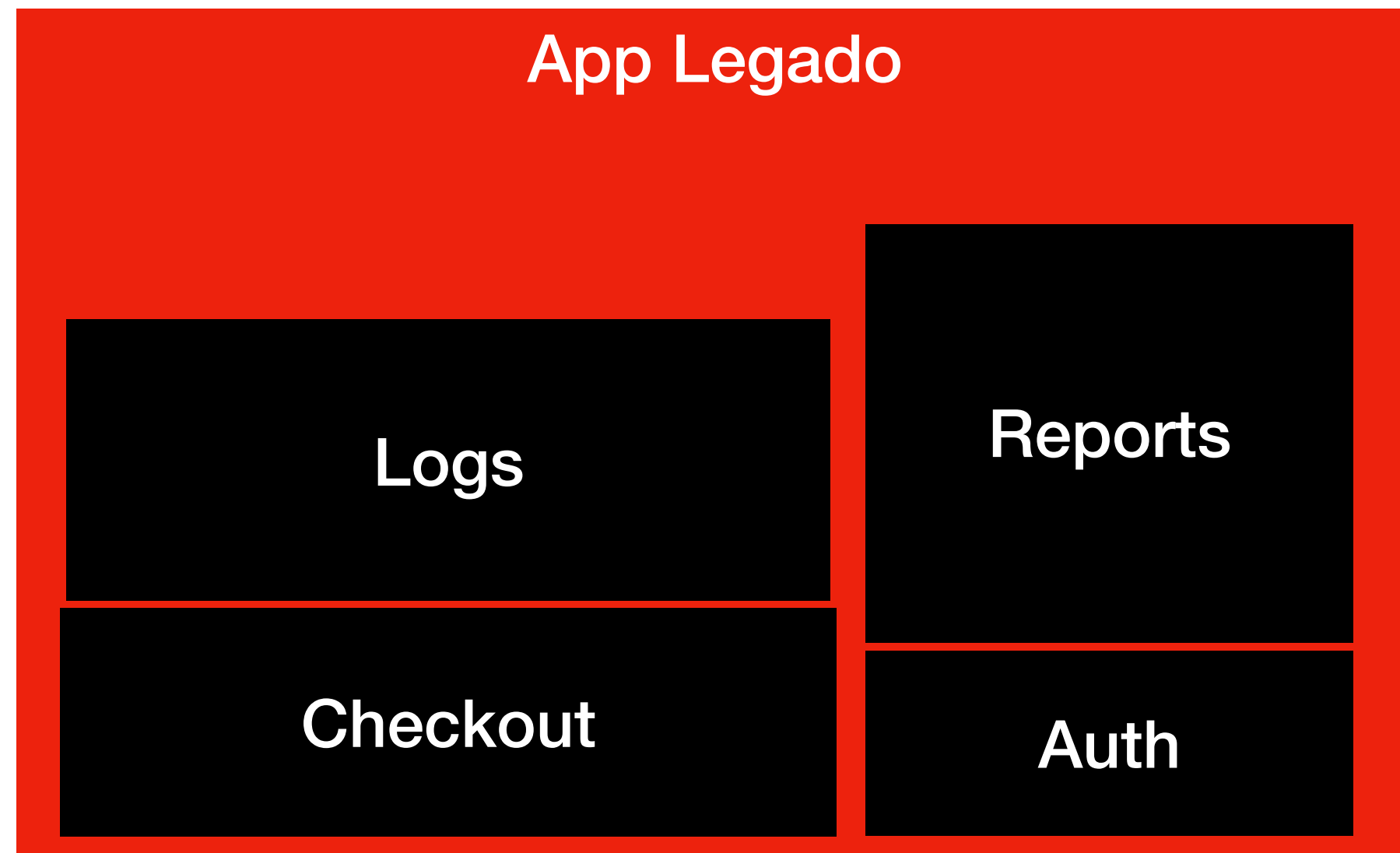
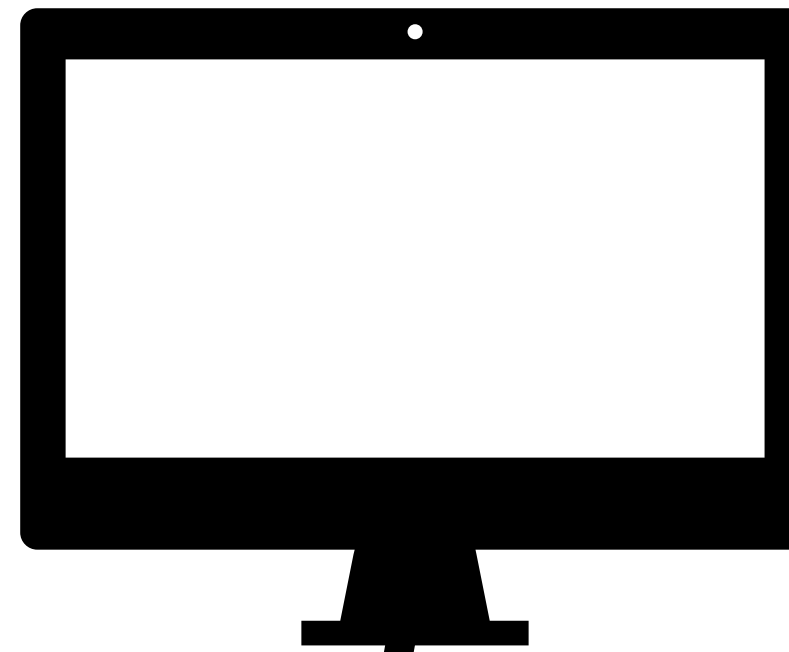












Se o novo sistema falhar no dia do corte
todo o negócio é impactado

Não existe **rollback** simples

Só gera valor **no lançamento**

Enquanto padrão como **Strangler** gera
valor incremental

Quando usar esse padrão?

O sistema atual é **insustentável**, impossível de manter

A arquitetura antiga *impede* qualquer evolução

O código é tão **acoplado** que não dá para aplicar Strangler ou Black Box

O **negócio aceita** um período de congelamento de features (freeze)

Existe **tempo e orçamento** para um grande projeto

**Objetivo final de qualquer
migração é melhorar**

**Se o usuário não reclamar,
melhor ainda**

Problemas sempre vão existir

Saiba escolher os problemas que
você **quer e pode** enfrentar

**Você *nunca* vai escolher a
melhor a arquitetura**

Você **nunca** vai escolher a
melhor arquitetura

Vai escolher a que vai te dar
menos dor de cabeça

Obrigado!

Perguntas?